

Simplified Astronomy Simulator Interface

<https://gemini.google.com/app/d03a94ec6cb25e96>

User prompt: I want to make a simplified astronomy education simulator making use of code already developed in another Gemini session, using a subset of the functionality and editing the interface in a few ways. I am uploading the webpage that will be the starting point. All additions need to be vanilla Javascript and CSS. 1) make the default background color white. Aim for a 4:3 landscape profile for the page, with a default width of 1000 pixels - my intention is to use this as an embed within other websites included in the flow of text. 2) the main display element is the graph of the thermal spectrum. To the left will be vertical controls for the star temperature and size, each with an input box as well where numbers could be typed (use the same limits and formatting as in the starting example). Below will be a panel that is normally hidden with the display control options (and is activated by hitting a "Advanced" button. 3) show the graph of the thermal spectrum using the color black, instead of adjusting its color to match the by-eye appearance of the star. Keep the instructions about displaying numbers as text (e.g., billion, trillions, etc.). Keep the representation of the "by-eye" view of the spectrum. 4) Move the graphic of the close-up view of the star to an overlay on top of the graph. Have it shift to the upper left or right corner of the graph to avoid overlaying the line graph itself. 5) Move the other display options to the "advanced" popup. Add a toggle here to remove the "by-eye" view of the spectrum.

Response: Here is the complete, self-contained HTML file for the simplified astronomy education simulator. It uses vanilla JavaScript and CSS, adhering to your requested modifications:

- The layout is set to a 1000px width with a 4:3 aspect ratio and a white default background.
- The main vertical controls (Temperature and Radius) are placed on the left side, using the same formatting and limits as the original code.
- The thermal spectrum is rendered as a black line, while preserving the "by-eye" color strip and the logic that displays large numbers as text (e.g., millions, billions).
- The close-up star models have been moved to a dynamic overlay on the graph. The code actively calculates the peak of the thermal curve and shifts the overlay to the opposite side (upper left or upper right) to prevent blocking the data.
- An "Advanced Options" toggle button reveals a bottom panel containing the remaining display options and the new toggle to hide/show the "by-eye" color strip.

Save the following code as an .html file and open it in any modern browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Stellar Simulator (Embed)</title>
  <style>
    :root {
      --bg-color: #ffffff;
      --panel-bg: #f8f9fa;
      --text-color: #111827;
      --accent-color: #2563eb;
      --border-color: #cbd5e1;
    }

    body {
      margin: 0;
      padding: 20px;
      font-family: system-ui, -apple-system, sans-serif;
      background-color: #e2e8f0; /* Outer background to show the 1000px boundary clearly */
      display: flex;
      justify-content: center;
    }

    /* Enforce 1000px width and 4:3 Aspect Ratio (1000x750) */
    .simulator-embed {
      width: 1000px;
      height: 750px;
      background-color: var(--bg-color);
      color: var(--text-color);
      border: 1px solid var(--border-color);
      box-shadow: 0 4px 6px rgba(0,0,0,0.1);
      box-sizing: border-box;
      display: flex;
      flex-direction: column;
      overflow: hidden;
      position: relative;
    }

    .top-section {
      display: flex;
      flex: 1;
      padding: 20px;
    }
```

```

        gap: 20px;
        height: calc(100% - 60px); /* Leave space for bottom button */
    }

    /* Left Panel - Vertical Controls */
    .left-controls {
        width: 250px;
        background-color: var(--panel-bg);
        border: 1px solid var(--border-color);
        border-radius: 8px;
        padding: 20px;
        display: flex;
        flex-direction: column;
        gap: 25px;
    }

    .control-group {
        display: flex;
        flex-direction: column;
        gap: 10px;
    }

    .control-group label {
        font-weight: 600;
        font-size: 0.95rem;
    }

    .control-group input[type="range"] {
        width: 100%;
        cursor: pointer;
    }

    .control-group input[type="number"] {
        width: 100px;
        padding: 6px;
        border: 1px solid var(--border-color);
        border-radius: 4px;
        text-align: center;
        font-size: 1rem;
        align-self: center;
    }

    /* Right Panel - Graph Display */
    .main-display {
        flex: 1;
        position: relative;
        border: 1px solid var(--border-color);
        border-radius: 8px;
        background-color: #ffffff;
    }

    canvas {
        width: 100%;
        height: 100%;
        display: block;
        border-radius: 8px;
        cursor: crosshair;
    }

    /* Dynamic Star Overlay */
    #star-overlay {
        position: absolute;
        background: rgba(255, 255, 255, 0.85);
        backdrop-filter: blur(4px);
        border: 1px solid var(--border-color);
        border-radius: 8px;
        padding: 10px 15px;
        display: flex;
        gap: 20px;
        pointer-events: none; /* Let mouse interact with graph underneath */
        transition: left 0.3s, right 0.3s;
        box-shadow: 0 2px 4px rgba(0,0,0,0.1);
    }

```

```

.overlay-right { right: 30px; top: 20px; left: auto; }
.overlay-left { left: 130px; top: 20px; right: auto; } /* Pad left accounts for Y-axis */

.star-view {
  display: flex;
  flex-direction: column;
  align-items: center;
}

.star-sphere-wrapper {
  width: 100px;
  height: 100px;
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: #0b0b0c; /* Keep dark background to see the star clearly */
  border-radius: 8px;
  overflow: hidden;
  margin-bottom: 5px;
}

.star-sphere {
  border-radius: 50%;
}

.star-label { font-weight: bold; font-size: 0.85rem; }
.star-meta { font-size: 0.75rem; color: #475569; text-align: center; }

/* Advanced Bottom Panel */
.bottom-section {
  padding: 0 20px 20px 20px;
  display: flex;
  flex-direction: column;
}

#advanced-btn {
  background-color: var(--panel-bg);
  border: 1px solid var(--border-color);
  padding: 10px;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
  text-align: center;
  transition: background-color 0.2s;
  width: 100%;
}

#advanced-btn:hover { background-color: #e2e8f0; }

.advanced-panel {
  display: none;
  background-color: var(--panel-bg);
  border: 1px solid var(--border-color);
  border-radius: 6px;
  padding: 15px;
  margin-top: 10px;
  flex-wrap: wrap;
  gap: 15px;
  align-items: center;
  font-size: 0.9rem;
}

.toggle-group {
  display: flex;
  align-items: center;
  gap: 6px;
}

.range-input {
  width: 70px;
  padding: 4px;
  border: 1px solid var(--border-color);
  border-radius: 4px;
  text-align: center;
}

```

```

    }

    button.small-btn {
        background-color: #fff;
        border: 1px solid var(--border-color);
        padding: 4px 10px;
        border-radius: 4px;
        cursor: pointer;
    }
</style>
</head>
<body>

<div class="simulator-embed">
  <div class="top-section">
    <div class="left-controls">
      <h3 style="margin: 0; font-size: 1.1rem; border-bottom: 1px solid #ccc; padding-bottom: 10px;">St

      <div class="control-group">
        <label for="temp-slider">Temperature (K)</label>
        <input type="range" id="temp-slider" min="7.7406" max="10.9151" step="0.0001" value="8.6656">
        <input type="number" id="temp-input" min="2300" max="55000" step="100" value="5800">
      </div>

      <div class="control-group">
        <label for="radius-slider">Radius (R☉)</label>
        <input type="range" id="radius-slider" min="0.1" max="10.0" step="0.1" value="1.0">
        <input type="number" id="radius-input" min="0.1" max="10.0" step="0.1" value="1.0">
      </div>
    </div>

    <div class="main-display">
      <canvas id="spectrumCanvas"></canvas>

      <div id="star-overlay" class="overlay-right">
        <div class="star-view">
          <div class="star-sphere-wrapper">
            <div id="sun-sphere" class="star-sphere"></div>
          </div>
          <div class="star-label">The Sun</div>
          <div class="star-meta">5,800 K<br>1.0 R☉</div>
        </div>
        <div class="star-view">
          <div class="star-sphere-wrapper">
            <div id="star-sphere" class="star-sphere"></div>
          </div>
          <div class="star-label">Target Star</div>
          <div id="star-meta-dynamic" class="star-meta">5,800 K<br>1.0 R☉</div>
        </div>
      </div>
    </div>
  </div>

  <div class="bottom-section">
    <button id="advanced-btn">Toggle Advanced Display Options</button>
    <div class="advanced-panel" id="advanced-panel">
      <div class="toggle-group">
        <input type="checkbox" id="toggle-byeye" checked>
        <label for="toggle-byeye">Show "By-Eye" Strip</label>
      </div>
      <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
        <input type="checkbox" id="toggle-logX">
        <label for="toggle-logX">Log X Axis</label>
      </div>
      <div class="toggle-group">
        <input type="checkbox" id="toggle-logY" checked>
        <label for="toggle-logY">Log Y Axis</label>
      </div>
      <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
        <label>X Range (nm):</label>
        <input type="number" id="x-min-input" class="range-input" value="10"> to
        <input type="number" id="x-max-input" class="range-input" value="2000">
      </div>
      <div class="toggle-group">

```

```

        <label>Y Max Limit:</label>
        <input type="number" id="y-max-input" class="range-input" placeholder="Auto">
      </div>
      <button id="btn-reset-limits" class="small-btn">Reset Limits</button>
    </div>
  </div>
</div>

<script>
  // App Constants
  const SUN_TEMP = 5800;
  const BASE_SUN_RADIUS_PX = 12; // Scaled down for overlay fit
  const DEFAULT_LAMBDA_MIN = 10;
  const DEFAULT_LAMBDA_MAX = 2000;
  const C1 = 3.741771e16;
  const C2 = 1.438776e7;

  // Dynamic State
  let appState = {
    temp: SUN_TEMP,
    radius: 1.0,
    isLogX: false,
    isLogY: true,
    showByEye: true,
    xMin: DEFAULT_LAMBDA_MIN,
    xMax: DEFAULT_LAMBDA_MAX,
    customYMax: null,
    mouseX: null,
    mouseY: null,
    barLogOrders: 2.0
  };

  // DOM Links Cache
  const DOM = {
    tempSlider: document.getElementById('temp-slider'),
    tempInput: document.getElementById('temp-input'),
    radiusSlider: document.getElementById('radius-slider'),
    radiusInput: document.getElementById('radius-input'),

    sunSphere: document.getElementById('sun-sphere'),
    starSphere: document.getElementById('star-sphere'),
    metaDynamic: document.getElementById('star-meta-dynamic'),
    starOverlay: document.getElementById('star-overlay'),

    specCanvas: document.getElementById('spectrumCanvas'),

    advancedBtn: document.getElementById('advanced-btn'),
    advancedPanel: document.getElementById('advanced-panel'),

    toggleByEye: document.getElementById('toggle-byeye'),
    logXCheck: document.getElementById('toggle-logX'),
    logYCheck: document.getElementById('toggle-logY'),
    xMinInput: document.getElementById('x-min-input'),
    xMaxInput: document.getElementById('x-max-input'),
    yMaxInput: document.getElementById('y-max-input'),
    btnResetLimits: document.getElementById('btn-reset-limits')
  };

  const specCtx = DOM.specCanvas.getContext('2d');

  // Mathematical Planck Curve Model
  function evaluatePlanck(nm, temperature) {
    const expTerm = Math.exp(C2 / (nm * temperature));
    if (!isFinite(expTerm) || expTerm <= 1) return 0;
    return C1 / (Math.pow(nm, 5) * (expTerm - 1));
  }

  function getLuminosity(t, r) {
    return (r * r) * Math.pow(t / SUN_TEMP, 4);
  }

  function kelvinToRGB(k) {
    let normalizedK = k / 100;
    let r, g, b;

```

```

    if (normalizedK <= 66) {
        r = 255;
        g = Math.max(0, Math.min(255, 99.4708 * Math.log(normalizedK) - 161.1196));
        b = normalizedK <= 19 ? 0 : Math.max(0, Math.min(255, 138.5177 * Math.log(normalizedK - 10) - 305.
    } else {
        r = Math.max(0, Math.min(255, 329.6987 * Math.pow(normalizedK - 60, -0.1332)));
        g = Math.max(0, Math.min(255, 288.1222 * Math.pow(normalizedK - 60, -0.0755)));
        b = 255;
    }
    return { r: Math.round(r), g: Math.round(g), b: Math.round(b) };
}

function wavelengthToRGB(nm) {
    let r = 0, g = 0, b = 0;
    if (nm >= 380 && nm < 440) { r = -(nm - 440) / (440 - 380); b = 1.0; }
    else if (nm >= 440 && nm < 490) { g = (nm - 440) / (490 - 440); b = 1.0; }
    else if (nm >= 490 && nm < 510) { g = 1.0; b = -(nm - 510) / (510 - 490); }
    else if (nm >= 510 && nm < 580) { r = (nm - 510) / (580 - 510); g = 1.0; }
    else if (nm >= 580 && nm < 645) { r = 1.0; g = -(nm - 645) / (645 - 580); }
    else if (nm >= 645 && nm <= 750) { r = 1.0; }
    return { r: Math.round(r * 255), g: Math.round(g * 255), b: Math.round(b * 255) };
}

function stringifyLargeValues(num) {
    if (num === 0) return "0";
    if (num >= 1e12) return (num / 1e12).toFixed(1).replace('.0', '') + " trillion";
    if (num >= 1e9) return (num / 1e9).toFixed(1).replace('.0', '') + " billion";
    if (num >= 1e6) return (num / 1e6).toFixed(1).replace('.0', '') + " million";
    if (num >= 1e3) return (num / 1e3).toFixed(1).replace('.0', '') + " thousand";

    if (num < 1) return parseFloat(num.toPrecision(2)).toString();
    if (num < 10) return num.toFixed(1);
    return num.toFixed(0);
}

function triggerResize() {
    const dpr = window.devicePixelRatio || 1;
    const specRect = DOM.specCanvas.parentElement.getBoundingClientRect();
    DOM.specCanvas.width = (specRect.width) * dpr;
    DOM.specCanvas.height = (specRect.height) * dpr;
    drawAppGraphics();
}

function drawAppGraphics() {
    renderPhysicalSpheres();
    renderSpectrumChart();
}

function renderPhysicalSpheres() {
    const sunRGB = kelvinToRGB(SUN_TEMP);
    const sunCSSColor = `rgb(${sunRGB.r}, ${sunRGB.g}, ${sunRGB.b})`;
    DOM.sunSphere.style.backgroundColor = sunCSSColor;
    DOM.sunSphere.style.width = `${BASE_SUN_RADIUS_PX * 2}px`;
    DOM.sunSphere.style.height = `${BASE_SUN_RADIUS_PX * 2}px`;
    DOM.sunSphere.style.boxShadow = `0 0 15px ${sunCSSColor}`;

    const targetRGB = kelvinToRGB(appState.temp);
    const targetCSSColor = `rgb(${targetRGB.r}, ${targetRGB.g}, ${targetRGB.b})`;
    const computedRadius = Math.max(4, BASE_SUN_RADIUS_PX * appState.radius);

    DOM.starSphere.style.backgroundColor = targetCSSColor;
    DOM.starSphere.style.width = `${computedRadius * 2}px`;
    DOM.starSphere.style.height = `${computedRadius * 2}px`;
    DOM.starSphere.style.boxShadow = `0 0 ${10 + computedRadius * 0.2}px ${targetCSSColor}`;

    DOM.metaDynamic.innerHTML = `${appState.temp.toLocaleString()} K<br>${appState.radius.toFixed(1)} R☉`;
}

function renderSpectrumChart() {
    const canvas = DOM.specCanvas;
    const ctx = specCtx;

    const w = canvas.width;

```

```

const h = canvas.height;
if (w <= 0 || h <= 0) return;

ctx.clearRect(0, 0, w, h);

const dpr = window.devicePixelRatio || 1;
const pad = { top: 30 * dpr, right: 30 * dpr, bottom: 50 * dpr, left: 110 * dpr };
const graphW = w - pad.left - pad.right;
const graphH = h - pad.top - pad.bottom;

if (graphW <= 0 || graphH <= 0) return;

const getXCoord = (nm) => {
  let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
  let logMax = Math.log10(appState.xMax);

  if (appState.isLogX) {
    return pad.left + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
  }
  return pad.left + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
};

const getWavelengthFromX = (x) => {
  const pct = (x - pad.left) / graphW;
  let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
  let logMax = Math.log10(appState.xMax);

  if (appState.isLogX) {
    return Math.pow(10, logMin + pct * (logMax - logMin));
  }
  return appState.xMin + pct * (appState.xMax - appState.xMin);
};

// Render By-Eye Strip
if (appState.showByEye) {
  const stripHeight = 22 * dpr;
  const stripY = pad.top / 2 - stripHeight / 2 + 5 * dpr;

  let systemPeakRadiance = 0;
  const step = Math.max(1, (appState.xMax - appState.xMin) / 1000);
  for (let l = appState.xMin; l <= appState.xMax; l += step) {
    let r = evaluatePlanck(l, appState.temp);
    if (r > systemPeakRadiance) systemPeakRadiance = r;
  }
  if (systemPeakRadiance === 0) systemPeakRadiance = 1.0;

  ctx.save();
  ctx.beginPath();
  ctx.rect(pad.left, stripY, graphW, stripHeight);
  ctx.clip();

  for (let x = pad.left; x < pad.left + graphW; x += Math.max(1, dpr)) {
    const currentNM = getWavelengthFromX(x);
    const intensity = evaluatePlanck(currentNM, appState.temp);

    let brightnessFactor = 0;
    if (intensity > 0) {
      let ratio = intensity / systemPeakRadiance;
      let logRatio = Math.log10(ratio);
      brightnessFactor = 1.0 + (logRatio / appState.barLogOrders);
      if (brightnessFactor < 0) brightnessFactor = 0;
    }

    if (currentNM < 380 || currentNM > 750) {
      let grayscaleIntensity = Math.round(230 * brightnessFactor); // Lighter gray for white bac
      ctx.fillStyle = `rgb(${grayscaleIntensity}, ${grayscaleIntensity}, ${grayscaleIntensity})`;
    } else {
      let hue = wavelengthToRGB(currentNM);
      ctx.fillStyle = `rgb(${Math.round(hue.r * brightnessFactor)}, ${Math.round(hue.g * brightn
    }
    ctx.fillRect(x, stripY, Math.max(1, dpr) + 0.5, stripHeight);
  }
  ctx.restore();
}

```

```

    ctx.strokeStyle = "#cbd5e1";
    ctx.lineWidth = 1 * dpr;
    ctx.strokeRect(pad.left, stripY, graphW, stripHeight);
}

// Curve Points
const pointsArray = [];
let maxObservedCurveIntensity = 0;
let peakX = pad.left;
const areaModifier = appState.radius * appState.radius;

for (let x = pad.left; x <= pad.left + graphW; x += Math.max(1, dpr * 2)) {
    const nm = getWavelengthFromX(x);
    const rawIntensity = evaluatePlanck(nm, appState.temp) * areaModifier;
    pointsArray.push({ x: x, val: rawIntensity });
    if (rawIntensity > maxObservedCurveIntensity) {
        maxObservedCurveIntensity = rawIntensity;
        peakX = x; // Find peak for dynamic overlay placement
    }
}

if (maxObservedCurveIntensity === 0) maxObservedCurveIntensity = 1.0;

// Shift dynamic overlay based on graph peak
if (peakX > pad.left + graphW / 2) {
    DOM.starOverlay.className = 'overlay-left';
} else {
    DOM.starOverlay.className = 'overlay-right';
}

const maxGraphY = appState.customYMax ? appState.customYMax : maxObservedCurveIntensity;
const minGraphY = appState.isLogY ? maxGraphY * 1e-4 : 0;

const getYCoord = (intensity) => {
    let pct;
    if (appState.isLogY) {
        let safeVal = Math.max(minGraphY, intensity);
        pct = (Math.log10(safeVal) - Math.log10(minGraphY)) / (Math.log10(maxGraphY) - Math.log10(minGraphY));
    } else {
        pct = intensity / maxGraphY;
    }
    return pad.top + graphH - (Math.max(0, Math.min(1, pct)) * graphH);
};

// Draw Axes & Grid
ctx.strokeStyle = '#e2e8f0';
ctx.lineWidth = 1 * dpr;
ctx.fillStyle = '#475569';
ctx.font = `${11 * dpr}px sans-serif`;

ctx.textAlign = 'center';
ctx.textBaseline = 'top';
let xTicks = [];
if (appState.isLogX) {
    xTicks = [appState.xMin];
    let current = Math.pow(10, Math.ceil(Math.log10(appState.xMin)));
    while (current < appState.xMax) {
        if (current > appState.xMin) xTicks.push(current);
        current *= 10;
    }
    xTicks.push(appState.xMax);
} else {
    const range = appState.xMax - appState.xMin;
    const step = Math.pow(10, Math.floor(Math.log10(range))) / 2;
    for (let t = appState.xMin; t <= appState.xMax; t += step) {
        xTicks.push(t);
    }
}

xTicks.forEach(tick => {
    const x = getXCoord(tick);
    if (x < pad.left || x > pad.left + graphW) return;
    ctx.beginPath(); ctx.moveTo(x, pad.top); ctx.lineTo(x, pad.top + graphH); ctx.stroke();
    ctx.fillText(`${Math.round(tick)} nm`, x, pad.top + graphH + 8 * dpr);
});

```

```

ctx.textAlign = 'right';
ctx.textBaseline = 'middle';
const drawYLabel = (val, yPos) => {
  ctx.beginPath(); ctx.moveTo(pad.left, yPos); ctx.lineTo(pad.left + graphW, yPos); ctx.stroke();

  let labelStr = stringifyLargeValues(val * 1e11);
  let parts = labelStr.split(' ');
  if (parts.length > 1) {
    ctx.fillText(parts[0], pad.left - 10 * dpr, yPos - 6 * dpr);
    ctx.fillText(parts[1], pad.left - 10 * dpr, yPos + 6 * dpr);
  } else {
    ctx.fillText(labelStr, pad.left - 10 * dpr, yPos);
  }
};

if (appState.isLogY) {
  let logStart = Math.ceil(Math.log10(minGraphY));
  let logEnd = Math.floor(Math.log10(maxGraphY));
  for (let o = logStart; o <= logEnd; o++) {
    let v = Math.pow(10, o);
    let y = getYCoord(v);
    drawYLabel(v, y);
  }
} else {
  [0, 0.25, 0.5, 0.75, 1.0].forEach(p => {
    let v = p * maxGraphY;
    let y = getYCoord(v);
    drawYLabel(v, y);
  });
}

ctx.strokeStyle = '#94a3b8';
ctx.lineWidth = 1.5 * dpr;
ctx.strokeRect(pad.left, pad.top, graphW, graphH);

// Draw Data Curve Line (Black)
ctx.beginPath();
let breakPath = true;
pointsArray.forEach(pt => {
  let visible = true;
  if (appState.isLogY && pt.val < minGraphY) visible = false;
  if (pt.val > maxGraphY) visible = false;

  if (visible) {
    let y = getYCoord(pt.val);
    if (breakPath) {
      ctx.moveTo(pt.x, y);
      breakPath = false;
    } else {
      ctx.lineTo(pt.x, y);
    }
  } else {
    breakPath = true;
  }
});

ctx.strokeStyle = '#000000'; // Curve line explicitly drawn black
ctx.lineWidth = 3 * dpr;
ctx.stroke();

// Mouse Hover Box
if (appState.mouseX !== null && appState.mouseY !== null) {
  const mx = appState.mouseX * dpr;
  const my = appState.mouseY * dpr;

  let closestPt = null;
  let minDist = Infinity;

  pointsArray.forEach(pt => {
    let visible = true;
    if (appState.isLogY && pt.val < minGraphY) visible = false;
    if (visible) {
      let py = getYCoord(pt.val);

```

```

        let dist = Math.hypot(pt.x - mx, py - my);
        if (dist < minDist) {
            minDist = dist;
            closestPt = pt;
        }
    });

    if (closestPt && minDist <= 15 * dpr) {
        let py = getYCoord(closestPt.val);

        ctx.beginPath();
        ctx.arc(closestPt.x, py, 5 * dpr, 0, Math.PI * 2);
        ctx.fillStyle = '#000000';
        ctx.fill();
        ctx.strokeStyle = '#ffffff';
        ctx.lineWidth = 1.5 * dpr;
        ctx.stroke();

        const hoverNM = getWavelengthFromX(closestPt.x);
        const stringWavelength = `Wavelength: ${Math.round(hoverNM)} nm`;
        const stringIntensity = `Intensity: ${stringifyLargeValues(closestPt.val * 1e11)}`;

        ctx.font = `${11 * dpr}px sans-serif`;
        const textWidth = Math.max(ctx.measureText(stringWavelength).width, ctx.measureText(stringInte
        const boxW = textWidth + 16 * dpr;
        const boxH = 40 * dpr;

        let boxX = closestPt.x + 12 * dpr;
        let boxY = py - 20 * dpr;

        if (boxX + boxW > w) boxX = closestPt.x - boxW - 12 * dpr;
        if (boxY < 0) boxY = 10 * dpr;
        if (boxY + boxH > h) boxY = h - boxH - 10 * dpr;

        ctx.fillStyle = 'rgba(15, 23, 42, 0.92)';
        ctx.strokeStyle = '#475569';
        ctx.lineWidth = 1 * dpr;
        ctx.beginPath(); ctx.rect(boxX, boxY, boxW, boxH); ctx.fill(); ctx.stroke();
        ctx.fillStyle = '#f8fafc';
        ctx.textAlign = 'left';
        ctx.textBaseline = 'top';
        ctx.fillText(stringWavelength, boxX + 8 * dpr, boxY + 6 * dpr);
        ctx.fillText(stringIntensity, boxX + 8 * dpr, boxY + 22 * dpr);
    }
}

ctx.fillStyle = '#111827';
ctx.font = `bold ${12 * dpr}px sans-serif`;
ctx.textAlign = 'center';
ctx.textBaseline = 'top';
ctx.fillText('Wavelength (nm)', pad.left + graphW / 2, pad.top + graphH + 30 * dpr);

ctx.save();
ctx.translate(pad.left - 65 * dpr, pad.top + graphH / 2);
ctx.rotate(-Math.PI / 2);
ctx.fillText('Spectral Radiance', 0, 0);
ctx.restore();
}

function bindEvents() {
    DOM.advancedBtn.addEventListener('click', () => {
        const isHidden = DOM.advancedPanel.style.display === 'none' || DOM.advancedPanel.style.display ===
        DOM.advancedPanel.style.display = isHidden ? 'flex' : 'none';
    });

    DOM.tempSlider.addEventListener('input', (e) => {
        appState.temp = Math.round(Math.exp(parseFloat(e.target.value)));
        DOM.tempInput.value = appState.temp;
        drawAppGraphics();
    });

    DOM.tempInput.addEventListener('input', (e) => {
        let v = parseInt(e.target.value) || SUN_TEMP;

```

```

    v = Math.max(2300, Math.min(55000, v));
    appState.temp = v;
    DOM.tempSlider.value = Math.log(v);
    drawAppGraphics();
  });

  DOM.radiusSlider.addEventListener('input', (e) => {
    appState.radius = parseFloat(e.target.value);
    DOM.radiusInput.value = appState.radius.toFixed(1);
    drawAppGraphics();
  });

  DOM.radiusInput.addEventListener('input', (e) => {
    let v = parseFloat(e.target.value) || 1.0;
    v = Math.max(0.1, Math.min(10.0, v));
    appState.radius = v;
    DOM.radiusSlider.value = v;
    drawAppGraphics();
  });

  // Advanced Toggles
  DOM.toggleByEye.addEventListener('change', (e) => { appState.showByEye = e.target.checked; drawAppGrap
  DOM.logXCheck.addEventListener('change', (e) => { appState.isLogX = e.target.checked; drawAppGraphics()
  DOM.logYCheck.addEventListener('change', (e) => { appState.isLogY = e.target.checked; drawAppGraphics()

  DOM.xMinInput.addEventListener('change', (e) => {
    let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MIN;
    if(v >= appState.xMax) v = appState.xMax - 1;
    appState.xMin = Math.max(1, v);
    DOM.xMinInput.value = appState.xMin;
    drawAppGraphics();
  });

  DOM.xMaxInput.addEventListener('change', (e) => {
    let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MAX;
    if(v <= appState.xMin) v = appState.xMin + 1;
    appState.xMax = v;
    DOM.xMaxInput.value = appState.xMax;
    drawAppGraphics();
  });

  DOM.yMaxInput.addEventListener('change', (e) => {
    let v = parseFloat(e.target.value);
    if (isNaN(v) || v <= 0) {
      appState.customYMax = null;
    } else {
      appState.customYMax = Math.max(1e-10, v * 1e-11);
    }
    drawAppGraphics();
  });

  DOM.btnResetLimits.addEventListener('click', () => {
    appState.xMin = DEFAULT_LAMBDA_MIN;
    appState.xMax = DEFAULT_LAMBDA_MAX;
    appState.customYMax = null;
    DOM.xMinInput.value = appState.xMin;
    DOM.xMaxInput.value = appState.xMax;
    DOM.yMaxInput.value = "";
    drawAppGraphics();
  });

  // Interactive mouse mapping
  DOM.specCanvas.addEventListener('mousemove', (e) => {
    const rect = DOM.specCanvas.getBoundingClientRect();
    appState.mouseX = e.clientX - rect.left;
    appState.mouseY = e.clientY - rect.top;
    drawAppGraphics();
  });
  DOM.specCanvas.addEventListener('mouseleave', () => {
    appState.mouseX = null;
    appState.mouseY = null;
    drawAppGraphics();
  });

```

```

    window.addEventListener('resize', triggerResize);
  }

  window.onload = () => {
    bindEvents();
    triggerResize(); // Render initial layout
  };
</script>
</body>
</html>

```

User prompt: Let's try a few tweaks. 1) change to a 8:3 aspect ratio, keeping a 1000 px width 2). change the label "Stellar Properties" to "Star" 3). remove the comparison image of the sun - only show the target star. 4). fix a behavior with the display of the target star. Currently, when the star has a large radius, it stretches vertically. Instead, just have it fill the box (it's OK if it just ends up being a box filled with a solid color) 5) make sure that when the advanced options button is hit, the advanced options appear within the frame. When "Sonified" is toggled on, add the play / pause etc. controls to the left control area

Response: Here is the updated, self-contained HTML file incorporating all of your tweaks.

The aspect ratio is strictly 8:3 (1000x375px), the sun has been removed to only show the target star, and `flex-shrink: 0` was applied to the star sphere so that it fills its container proportionally rather than stretching vertically into an oval. The sonification engine from the original code has been restored, and its playback controls inject directly into the left column when enabled. The advanced options panel now absolutely positions itself at the bottom of the frame to avoid breaking the shortened 375px layout.

Save this as an .html file and open it in your browser:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Stellar Simulator (Embed)</title>
  <style>
    :root {
      --bg-color: #ffffff;
      --panel-bg: #f8f9fa;
      --text-color: #111827;
      --accent-color: #2563eb;
      --border-color: #cbd5e1;
      --sound-accent: #f43f5e;
    }

    body {
      margin: 0;
      padding: 20px;
      font-family: system-ui, -apple-system, sans-serif;
      background-color: #e2e8f0;
      display: flex;
      justify-content: center;
    }

    /* Enforce 1000px width and 8:3 Aspect Ratio (1000x375) */
    .simulator-embed {
      width: 1000px;
      height: 375px;
      background-color: var(--bg-color);
      color: var(--text-color);
      border: 1px solid var(--border-color);
      box-shadow: 0 4px 6px rgba(0,0,0,0.1);
      box-sizing: border-box;
      display: flex;
      flex-direction: column;
      position: relative;
    }

    .top-section {
      display: flex;
      flex: 1;
      padding: 15px;
      gap: 15px;
      height: calc(100% - 45px);
      box-sizing: border-box;
    }

```

```
/* Left Panel - Vertical Controls */
.left-controls {
  width: 250px;
  background-color: var(--panel-bg);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 15px;
  display: flex;
  flex-direction: column;
  gap: 15px;
  box-sizing: border-box;
  overflow-y: auto;
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 5px;
}

.control-group label {
  font-weight: 600;
  font-size: 0.90rem;
}

.control-group input[type="range"] {
  width: 100%;
  cursor: pointer;
}

.control-group input[type="number"] {
  width: 90px;
  padding: 4px;
  border: 1px solid var(--border-color);
  border-radius: 4px;
  text-align: center;
  font-size: 0.9rem;
  align-self: center;
}

/* Playback Controls (Injected in Left Column) */
#playback-controls {
  display: none;
  flex-direction: column;
  gap: 8px;
  background: #fff;
  border: 1px solid var(--sound-accent);
  padding: 10px;
  border-radius: 6px;
  margin-top: auto;
}

.playback-label {
  font-weight: bold;
  color: var(--sound-accent);
  font-size: 0.85rem;
  text-align: center;
  margin-bottom: 4px;
}

.btn-main {
  background-color: var(--sound-accent);
  color: white;
  border: none;
  padding: 6px;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
  width: 100%;
}

.btn-row {
  display: flex;
}
```

```

    gap: 5px;
}

.btn-sub {
  background-color: #e2e8f0;
  border: 1px solid var(--border-color);
  padding: 4px;
  border-radius: 4px;
  flex: 1;
  font-size: 0.8rem;
  cursor: pointer;
}

.speed-text { font-size: 0.8rem; text-align: center; padding-top: 2px; }

/* Right Panel - Graph Display */
.main-display {
  flex: 1;
  position: relative;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  background-color: #ffffff;
  box-sizing: border-box;
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
  border-radius: 8px;
  cursor: crosshair;
}

/* Dynamic Star Overlay */
#star-overlay {
  position: absolute;
  background: rgba(255, 255, 255, 0.85);
  backdrop-filter: blur(4px);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 10px;
  display: flex;
  flex-direction: column;
  align-items: center;
  pointer-events: none;
  transition: left 0.3s, right 0.3s;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.overlay-right { right: 20px; top: 15px; left: auto; }
.overlay-left { left: 95px; top: 15px; right: auto; }

.star-sphere-wrapper {
  width: 80px;
  height: 80px;
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: #0b0b0c;
  border-radius: 8px;
  overflow: hidden;
  margin-bottom: 5px;
}

.star-sphere {
  border-radius: 50%;
  flex-shrink: 0; /* Prevents star from stretching vertically into an oval */
}

.star-label { font-weight: bold; font-size: 0.85rem; }
.star-meta { font-size: 0.75rem; color: #475569; text-align: center; }

/* Bottom Section & Advanced Panel */
.bottom-section {

```

```

padding: 0 15px 15px 15px;
box-sizing: border-box;
}

#advanced-btn {
background-color: var(--panel-bg);
border: 1px solid var(--border-color);
padding: 6px;
border-radius: 6px;
cursor: pointer;
font-weight: 600;
text-align: center;
transition: background-color 0.2s;
width: 100%;
font-size: 0.9rem;
}

#advanced-btn:hover { background-color: #e2e8f0; }

.advanced-panel {
display: none;
position: absolute;
bottom: 45px;
left: 15px;
width: calc(100% - 30px);
background-color: #ffffff;
border: 1px solid var(--border-color);
border-radius: 6px;
padding: 10px 15px;
box-shadow: 0 -4px 15px rgba(0,0,0,0.15);
box-sizing: border-box;
z-index: 100;
flex-wrap: wrap;
gap: 15px;
align-items: center;
font-size: 0.85rem;
}

.toggle-group {
display: flex;
align-items: center;
gap: 6px;
}

.range-input {
width: 60px;
padding: 4px;
border: 1px solid var(--border-color);
border-radius: 4px;
text-align: center;
}

button.small-btn {
background-color: #f1f5f9;
border: 1px solid var(--border-color);
padding: 4px 10px;
border-radius: 4px;
cursor: pointer;
}
</style>
</head>
<body>

<div class="simulator-embed">
  <div class="top-section">
    <div class="left-controls">
      <h3 style="margin: 0; font-size: 1.1rem; border-bottom: 1px solid var(--border-color); padding-bot

      <div class="control-group">
        <label for="temp-slider">Temperature (K)</label>
        <input type="range" id="temp-slider" min="7.7406" max="10.9151" step="0.0001" value="8.6656">
        <input type="number" id="temp-input" min="2300" max="55000" step="100" value="5800">
      </div>
    </div>
  </div>
</div>

```

```

<div class="control-group">
  <label for="radius-slider">Radius ( $R_{\odot}$ )</label>
  <input type="range" id="radius-slider" min="0.1" max="10.0" step="0.1" value="1.0">
  <input type="number" id="radius-input" min="0.1" max="10.0" step="0.1" value="1.0">
</div>

<div id="playback-controls">
  <div class="playback-label">Spectrum Audio Scan</div>
  <button id="btn-play" class="btn-main">Play Spectrum</button>
  <div class="btn-row">
    <button id="btn-reset" class="btn-sub">Reset</button>
    <button id="btn-slower" class="btn-sub">Slower</button>
    <button id="btn-faster" class="btn-sub">Faster</button>
  </div>
  <div id="speed-label" class="speed-text">Speed: 1.00x</div>
</div>

<div class="main-display">
  <canvas id="spectrumCanvas"></canvas>

  <div id="star-overlay" class="overlay-right">
    <div class="star-sphere-wrapper">
      <div id="star-sphere" class="star-sphere"></div>
    </div>
    <div class="star-label">Target Star</div>
    <div id="star-meta-dynamic" class="star-meta">5,800 K<br>1.0  $R_{\odot}$ </div>
  </div>
</div>

<div class="bottom-section">
  <button id="advanced-btn">Toggle Advanced Display Options</button>
  <div class="advanced-panel" id="advanced-panel">
    <div class="toggle-group" style="font-weight: bold; color: var(--sound-accent);">
      <input type="checkbox" id="toggle-sound">
      <label for="toggle-sound">Sonified</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
      <input type="checkbox" id="toggle-byeye" checked>
      <label for="toggle-byeye">Show "By-Eye" Strip</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
      <input type="checkbox" id="toggle-logX">
      <label for="toggle-logX">Log X Axis</label>
    </div>
    <div class="toggle-group">
      <input type="checkbox" id="toggle-logY" checked>
      <label for="toggle-logY">Log Y Axis</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
      <label>X Range (nm):</label>
      <input type="number" id="x-min-input" class="range-input" value="10"> to
      <input type="number" id="x-max-input" class="range-input" value="2000">
    </div>
    <div class="toggle-group">
      <label>Y Max Limit:</label>
      <input type="number" id="y-max-input" class="range-input" placeholder="Auto">
    </div>
    <button id="btn-reset-limits" class="small-btn">Reset Limits</button>
  </div>
</div>

<script>
  // App Constants
  const SUN_TEMP = 5800;
  const BASE_SUN_RADIUS_PX = 10;
  const DEFAULT_LAMBDA_MIN = 10;
  const DEFAULT_LAMBDA_MAX = 2000;
  const C1 = 3.741771e16;
  const C2 = 1.438776e7;

  // Dynamic State

```

```

let appState = {
  temp: SUN_TEMP,
  radius: 1.0,
  isLogX: false,
  isLogY: true,
  showByEye: true,
  xMin: DEFAULT_LAMBDA_MIN,
  xMax: DEFAULT_LAMBDA_MAX,
  customYMax: null,
  mouseX: null,
  mouseY: null,
  barLogOrders: 2.0,

  // Sonification State
  isSoundEnabled: false,
  isPlaying: false,
  scanWavelength: DEFAULT_LAMBDA_MIN,
  playbackSpeedMultiplier: 1.0,
  audioOrders: 4.0,
  isDraggingScanLine: false
};

// DOM Links Cache
const DOM = {
  tempSlider: document.getElementById('temp-slider'),
  tempInput: document.getElementById('temp-input'),
  radiusSlider: document.getElementById('radius-slider'),
  radiusInput: document.getElementById('radius-input'),

  starSphere: document.getElementById('star-sphere'),
  metaDynamic: document.getElementById('star-meta-dynamic'),
  starOverlay: document.getElementById('star-overlay'),

  specCanvas: document.getElementById('spectrumCanvas'),

  advancedBtn: document.getElementById('advanced-btn'),
  advancedPanel: document.getElementById('advanced-panel'),

  toggleSound: document.getElementById('toggle-sound'),
  toggleByEye: document.getElementById('toggle-byeye'),
  logXCheck: document.getElementById('toggle-logX'),
  logYCheck: document.getElementById('toggle-logY'),
  xMinInput: document.getElementById('x-min-input'),
  xMaxInput: document.getElementById('x-max-input'),
  yMaxInput: document.getElementById('y-max-input'),
  btnResetLimits: document.getElementById('btn-reset-limits'),

  // Playback DOM
  playbackControls: document.getElementById('playback-controls'),
  btnPlay: document.getElementById('btn-play'),
  btnReset: document.getElementById('btn-reset'),
  btnSlower: document.getElementById('btn-slower'),
  btnFaster: document.getElementById('btn-faster'),
  speedLabel: document.getElementById('speed-label')
};

const specCtx = DOM.specCanvas.getContext('2d');

// Web Audio Variables
let audioCtx = null;
let propOsc = null;
let propGain = null;
let scanOsc = null;
let scanGain = null;
let propFadeTimeout = null;

// Initialize Web Audio context
function initAudioEngine() {
  if (audioCtx) return;
  audioCtx = new (window.AudioContext || window.webkitAudioContext)();

  propOsc = audioCtx.createOscillator();
  propOsc.type = 'sine';
  propGain = audioCtx.createGain();

```

```

propGain.gain.setValueAtTime(0, audioCtx.currentTime);
propOsc.connect(propGain);
propGain.connect(audioCtx.destination);
propOsc.start();

scanOsc = audioCtx.createOscillator();
scanOsc.type = 'triangle';
scanGain = audioCtx.createGain();
scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
scanOsc.connect(scanGain);
scanGain.connect(audioCtx.destination);
scanOsc.start();
}

function evaluatePlanck(nm, temperature) {
  const expTerm = Math.exp(C2 / (nm * temperature));
  if (!isFinite(expTerm) || expTerm <= 1) return 0;
  return C1 / (Math.pow(nm, 5) * (expTerm - 1));
}

function getLuminosity(t, r) {
  return (r * r) * Math.pow(t / SUN_TEMP, 4);
}

function kelvinToRGB(k) {
  let normalizedK = k / 100;
  let r, g, b;

  if (normalizedK <= 66) {
    r = 255;
    g = Math.max(0, Math.min(255, 99.4708 * Math.log(normalizedK) - 161.1196));
    b = normalizedK <= 19 ? 0 : Math.max(0, Math.min(255, 138.5177 * Math.log(normalizedK - 10) - 305.
  } else {
    r = Math.max(0, Math.min(255, 329.6987 * Math.pow(normalizedK - 60, -0.1332)));
    g = Math.max(0, Math.min(255, 288.1222 * Math.pow(normalizedK - 60, -0.0755)));
    b = 255;
  }
  return { r: Math.round(r), g: Math.round(g), b: Math.round(b) };
}

function wavelengthToRGB(nm) {
  let r = 0, g = 0, b = 0;
  if (nm >= 380 && nm < 440) { r = -(nm - 440) / (440 - 380); b = 1.0; }
  else if (nm >= 440 && nm < 490) { g = (nm - 440) / (490 - 440); b = 1.0; }
  else if (nm >= 490 && nm < 510) { g = 1.0; b = -(nm - 510) / (510 - 490); }
  else if (nm >= 510 && nm < 580) { r = (nm - 510) / (580 - 510); g = 1.0; }
  else if (nm >= 580 && nm < 645) { r = 1.0; g = -(nm - 645) / (645 - 580); }
  else if (nm >= 645 && nm <= 750) { r = 1.0; }
  return { r: Math.round(r * 255), g: Math.round(g * 255), b: Math.round(b * 255) };
}

function stringifyLargeValues(num) {
  if (num === 0) return "0";
  if (num >= 1e12) return (num / 1e12).toFixed(1).replace('.0', '') + " trillion";
  if (num >= 1e9) return (num / 1e9).toFixed(1).replace('.0', '') + " billion";
  if (num >= 1e6) return (num / 1e6).toFixed(1).replace('.0', '') + " million";
  if (num >= 1e3) return (num / 1e3).toFixed(1).replace('.0', '') + " thousand";

  if (num < 1) return parseFloat(num.toPrecision(2)).toString();
  if (num < 10) return num.toFixed(1);
  return num.toFixed(0);
}

function triggerPropertyTone() {
  if (!appState.isSoundEnabled) return;
  initAudioEngine();
  if (audioCtx.state === 'suspended') audioCtx.resume();

  let peakWavelength = 2.8977719e6 / appState.temp;
  let clampedWavelength = Math.max(appState.xMin, Math.min(appState.xMax, peakWavelength));
  let pct = (clampedWavelength - appState.xMin) / (appState.xMax - appState.xMin);
  let frequency = 1300 * Math.pow((110 / 1300), pct);

  const targetLuminosity = getLuminosity(appState.temp, appState.radius);

```

```

let logL = targetLuminosity > 0 ? Math.log10(targetLuminosity) : -4;
let scaleAlpha = 0.33 + (logL * 0.15);
let volumeFactor = Math.max(0.05, Math.min(1.0, scaleAlpha)) * 0.25;

propOsc.frequency.setValueAtTime(frequency, audioCtx.currentTime);

if (propFadeTimeout) clearTimeout(propFadeTimeout);
propGain.gain.setValueAtTime(volumeFactor, audioCtx.currentTime);

propFadeTimeout = setTimeout(() => {
  propGain.gain.setValueAtTime(propGain.gain.value, audioCtx.currentTime);
  propGain.gain.linearRampToValueAtTime(0, audioCtx.currentTime + 0.5);
}, 40);
}

function updatePlaybackScanner(dt) {
  if (appState.isPlaying && !appState.isDraggingScanLine) {
    const baseWavelengthStepPerMs = (appState.xMax - appState.xMin) / 7000;
    appState.scanWavelength += baseWavelengthStepPerMs * dt * appState.playbackSpeedMultiplier;

    if (appState.scanWavelength >= appState.xMax) {
      appState.scanWavelength = appState.xMin;
    }
  }

  if (appState.isSoundEnabled && (appState.isPlaying || appState.isDraggingScanLine)) {
    initAudioEngine();
    if (audioCtx.state === 'suspended') audioCtx.resume();

    let pct = (appState.scanWavelength - appState.xMin) / (appState.xMax - appState.xMin);
    let frequency = 1300 * Math.pow((110 / 1300), pct);

    const areaModifier = appState.radius * appState.radius;
    let currentIntensity = evaluatePlanck(appState.scanWavelength, appState.temp) * areaModifier;
    let peakWavelength = 2.8977719e6 / appState.temp;
    let peakCurveIntensity = evaluatePlanck(peakWavelength, appState.temp) * areaModifier;
    if (peakCurveIntensity <= 0) peakCurveIntensity = 1.0;

    let ratio = currentIntensity / peakCurveIntensity;
    let scanVolume = 0;
    if (ratio > 0) {
      let logRatio = Math.log10(ratio);
      scanVolume = 1.0 + (logRatio / appState.audioOrders);
      if (scanVolume < 0) scanVolume = 0;
    }

    scanOsc.frequency.setValueAtTime(frequency, audioCtx.currentTime);
    scanGain.gain.setValueAtTime(scanVolume * 0.25, audioCtx.currentTime);
  } else {
    if (scanGain) {
      scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
    }
  }
}

function togglePlayback() {
  appState.isPlaying = !appState.isPlaying;
  DOM.btnPlay.innerHTML = appState.isPlaying ? "Pause Spectrum" : "Play Spectrum";
  DOM.btnPlay.style.backgroundColor = appState.isPlaying ? "#94a3b8" : "var(--sound-accent)";
  if (appState.isSoundEnabled) initAudioEngine();
}

function triggerResize() {
  const dpr = window.devicePixelRatio || 1;
  const specRect = DOM.specCanvas.parentElement.getBoundingClientRect();
  DOM.specCanvas.width = (specRect.width) * dpr;
  DOM.specCanvas.height = (specRect.height) * dpr;
}

function drawAppGraphics() {
  renderPhysicalSpheres();
  renderSpectrumChart();
}

```

```

function renderPhysicalSpheres() {
  const targetRGB = kelvinToRGB(appState.temp);
  const targetCSSColor = `rgb(${targetRGB.r}, ${targetRGB.g}, ${targetRGB.b})`;
  const computedRadius = Math.max(4, BASE_SUN_RADIUS_PX * appState.radius);

  DOM.starSphere.style.backgroundColor = targetCSSColor;
  DOM.starSphere.style.width = `${computedRadius * 2}px`;
  DOM.starSphere.style.height = `${computedRadius * 2}px`;
  DOM.starSphere.style.boxShadow = `0 0 ${10 + computedRadius * 0.2}px ${targetCSSColor}`;
  DOM.metaDynamic.innerHTML = `${appState.temp.toLocaleString()} K<br>${appState.radius.toFixed(1)} R☉`;
}

function renderSpectrumChart() {
  const canvas = DOM.specCanvas;
  const ctx = specCtx;

  const w = canvas.width;
  const h = canvas.height;
  if (w <= 0 || h <= 0) return;

  ctx.clearRect(0, 0, w, h);

  const dpr = window.devicePixelRatio || 1;
  const pad = { top: 30 * dpr, right: 30 * dpr, bottom: 45 * dpr, left: 110 * dpr };
  const graphW = w - pad.left - pad.right;
  const graphH = h - pad.top - pad.bottom;

  if (graphW <= 0 || graphH <= 0) return;

  const getXCoord = (nm) => {
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
      return pad.left + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
    }
    return pad.left + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
  };

  const getWavelengthFromX = (x) => {
    const pct = (x - pad.left) / graphW;
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
      return Math.pow(10, logMin + pct * (logMax - logMin));
    }
    return appState.xMin + pct * (appState.xMax - appState.xMin);
  };

  if (appState.showByEye) {
    const stripHeight = 16 * dpr;
    const stripY = pad.top / 2 - stripHeight / 2 + 5 * dpr;

    let systemPeakRadiance = 0;
    const step = Math.max(1, (appState.xMax - appState.xMin) / 1000);
    for (let l = appState.xMin; l <= appState.xMax; l += step) {
      let r = evaluatePlanck(l, appState.temp);
      if (r > systemPeakRadiance) systemPeakRadiance = r;
    }
    if (systemPeakRadiance === 0) systemPeakRadiance = 1.0;

    ctx.save();
    ctx.beginPath();
    ctx.rect(pad.left, stripY, graphW, stripHeight);
    ctx.clip();

    for (let x = pad.left; x < pad.left + graphW; x += Math.max(1, dpr)) {
      const currentNM = getWavelengthFromX(x);
      const intensity = evaluatePlanck(currentNM, appState.temp);

      let brightnessFactor = 0;
      if (intensity > 0) {
        let ratio = intensity / systemPeakRadiance;

```

```

        let logRatio = Math.log10(ratio);
        brightnessFactor = 1.0 + (logRatio / appState.barLogOrders);
        if (brightnessFactor < 0) brightnessFactor = 0;
    }

    if (currentNM < 380 || currentNM > 750) {
        let grayscaleIntensity = Math.round(230 * brightnessFactor);
        ctx.fillStyle = `rgb(${grayscaleIntensity}, ${grayscaleIntensity}, ${grayscaleIntensity})`;
    } else {
        let hue = wavelengthToRGB(currentNM);
        ctx.fillStyle = `rgb(${Math.round(hue.r * brightnessFactor)}, ${Math.round(hue.g * brightn
    }
    ctx.fillRect(x, stripY, Math.max(1, dpr) + 0.5, stripHeight);
}
ctx.restore();

ctx.strokeStyle = "#cbd5e1";
ctx.lineWidth = 1 * dpr;
ctx.strokeRect(pad.left, stripY, graphW, stripHeight);
}

const pointsArray = [];
let maxObservedCurveIntensity = 0;
let peakX = pad.left;
const areaModifier = appState.radius * appState.radius;

for (let x = pad.left; x <= pad.left + graphW; x += Math.max(1, dpr * 2)) {
    const nm = getWavelengthFromX(x);
    const rawIntensity = evaluatePlanck(nm, appState.temp) * areaModifier;
    pointsArray.push({ x: x, val: rawIntensity });
    if (rawIntensity > maxObservedCurveIntensity) {
        maxObservedCurveIntensity = rawIntensity;
        peakX = x;
    }
}
if (maxObservedCurveIntensity === 0) maxObservedCurveIntensity = 1.0;

if (peakX > pad.left + graphW / 2) {
    DOM.starOverlay.className = 'overlay-left';
} else {
    DOM.starOverlay.className = 'overlay-right';
}

const maxGraphY = appState.customYMax ? appState.customYMax : maxObservedCurveIntensity;
const minGraphY = appState.isLogY ? maxGraphY * 1e-4 : 0;

const getYCoord = (intensity) => {
    let pct;
    if (appState.isLogY) {
        let safeVal = Math.max(minGraphY, intensity);
        pct = (Math.log10(safeVal) - Math.log10(minGraphY)) / (Math.log10(maxGraphY) - Math.log10(min
    } else {
        pct = intensity / maxGraphY;
    }
    return pad.top + graphH - (Math.max(0, Math.min(1, pct)) * graphH);
};

ctx.strokeStyle = '#e2e8f0';
ctx.lineWidth = 1 * dpr;
ctx.fillStyle = '#475569';
ctx.font = `${11 * dpr}px sans-serif`;

ctx.textAlign = 'center';
ctx.textBaseline = 'top';
let xTicks = [];
if (appState.isLogX) {
    xTicks = [appState.xMin];
    let current = Math.pow(10, Math.ceil(Math.log10(appState.xMin)));
    while (current < appState.xMax) {
        if (current > appState.xMin) xTicks.push(current);
        current *= 10;
    }
    xTicks.push(appState.xMax);
} else {

```

```

    const range = appState.xMax - appState.xMin;
    const step = Math.pow(10, Math.floor(Math.log10(range))) / 2;
    for (let t = appState.xMin; t <= appState.xMax; t += step) {
        xTicks.push(t);
    }
}

xTicks.forEach(tick => {
    const x = getXCoord(tick);
    if (x < pad.left || x > pad.left + graphW) return;
    ctx.beginPath(); ctx.moveTo(x, pad.top); ctx.lineTo(x, pad.top + graphH); ctx.stroke();
    ctx.fillText(`${Math.round(tick)} nm`, x, pad.top + graphH + 8 * dpr);
});

ctx.textAlign = 'right';
ctx.textBaseline = 'middle';
const drawYLabel = (val, yPos) => {
    ctx.beginPath(); ctx.moveTo(pad.left, yPos); ctx.lineTo(pad.left + graphW, yPos); ctx.stroke();

    let labelStr = stringifyLargeValues(val * 1e11);
    let parts = labelStr.split(' ');
    if (parts.length > 1) {
        ctx.fillText(parts[0], pad.left - 10 * dpr, yPos - 6 * dpr);
        ctx.fillText(parts[1], pad.left - 10 * dpr, yPos + 6 * dpr);
    } else {
        ctx.fillText(labelStr, pad.left - 10 * dpr, yPos);
    }
};

if (appState.isLogY) {
    let logStart = Math.ceil(Math.log10(minGraphY));
    let logEnd = Math.floor(Math.log10(maxGraphY));
    for (let o = logStart; o <= logEnd; o++) {
        let v = Math.pow(10, o);
        let y = getYCoord(v);
        drawYLabel(v, y);
    }
} else {
    [0, 0.25, 0.5, 0.75, 1.0].forEach(p => {
        let v = p * maxGraphY;
        let y = getYCoord(v);
        drawYLabel(v, y);
    });
}

ctx.strokeStyle = '#94a3b8';
ctx.lineWidth = 1.5 * dpr;
ctx.strokeRect(pad.left, pad.top, graphW, graphH);

// Draw Data Curve (Black)
ctx.beginPath();
let breakPath = true;
pointsArray.forEach(pt => {
    let visible = true;
    if (appState.isLogY && pt.val < minGraphY) visible = false;
    if (pt.val > maxGraphY) visible = false;

    if (visible) {
        let y = getYCoord(pt.val);
        if (breakPath) {
            ctx.moveTo(pt.x, y);
            breakPath = false;
        } else {
            ctx.lineTo(pt.x, y);
        }
    } else {
        breakPath = true;
    }
});

ctx.strokeStyle = '#000000';
ctx.lineWidth = 3 * dpr;
ctx.stroke();

```

```

// Audio Scan Indicator Line
if (appState.isSoundEnabled) {
  const scanLineX = getXCoord(appState.scanWavelength);
  if (scanLineX >= pad.left && scanLineX <= pad.left + graphW) {
    ctx.strokeStyle = '#f43f5e';
    ctx.lineWidth = 2 * dpr;
    ctx.beginPath();
    ctx.moveTo(scanLineX, pad.top);
    ctx.lineTo(scanLineX, pad.top + graphH);
    ctx.stroke();

    ctx.beginPath();
    ctx.arc(scanLineX, pad.top, 5 * dpr, 0, Math.PI * 2);
    ctx.fillStyle = '#f43f5e';
    ctx.fill();
  }
}

// Mouse Hover Box
if (appState.mouseX !== null && appState.mouseY !== null && !appState.isDraggingScanLine) {
  const mx = appState.mouseX * dpr;
  const my = appState.mouseY * dpr;

  let closestPt = null;
  let minDist = Infinity;

  pointsArray.forEach(pt => {
    let visible = true;
    if (appState.isLogY && pt.val < minGraphY) visible = false;
    if (visible) {
      let py = getYCoord(pt.val);
      let dist = Math.hypot(pt.x - mx, py - my);
      if (dist < minDist) {
        minDist = dist;
        closestPt = pt;
      }
    }
  });

  if (closestPt && minDist <= 15 * dpr) {
    let py = getYCoord(closestPt.val);

    ctx.beginPath();
    ctx.arc(closestPt.x, py, 5 * dpr, 0, Math.PI * 2);
    ctx.fillStyle = '#000000';
    ctx.fill();
    ctx.strokeStyle = '#ffffff';
    ctx.lineWidth = 1.5 * dpr;
    ctx.stroke();

    const hoverNM = getWavelengthFromX(closestPt.x);
    const stringWavelength = `Wavelength: ${Math.round(hoverNM)} nm`;
    const stringIntensity = `Intensity: ${stringifyLargeValues(closestPt.val * 1e11)}`;

    ctx.font = `${11 * dpr}px sans-serif`;
    const textWidth = Math.max(ctx.measureText(stringWavelength).width, ctx.measureText(stringInte
    const boxW = textWidth + 16 * dpr;
    const boxH = 40 * dpr;

    let boxX = closestPt.x + 12 * dpr;
    let boxY = py - 20 * dpr;

    if (boxX + boxW > w) boxX = closestPt.x - boxW - 12 * dpr;
    if (boxY < 0) boxY = 10 * dpr;
    if (boxY + boxH > h) boxY = h - boxH - 10 * dpr;

    ctx.fillStyle = 'rgba(15, 23, 42, 0.92)';
    ctx.strokeStyle = '#475569';
    ctx.lineWidth = 1 * dpr;
    ctx.beginPath(); ctx.rect(boxX, boxY, boxW, boxH); ctx.fill(); ctx.stroke();
    ctx.fillStyle = '#f8fafc';
    ctx.textAlign = 'left';
    ctx.textBaseline = 'top';
    ctx.fillText(stringWavelength, boxX + 8 * dpr, boxY + 6 * dpr);
  }
}

```

```

        ctx.fillText(stringIntensity, boxX + 8 * dpr, boxY + 22 * dpr);
    }
}

ctx.fillStyle = '#111827';
ctx.font = `bold ${12 * dpr}px sans-serif`;
ctx.textAlign = 'center';
ctx.textBaseline = 'top';
ctx.fillText('Wavelength (nm)', pad.left + graphW / 2, pad.top + graphH + 25 * dpr);

ctx.save();
ctx.translate(pad.left - 65 * dpr, pad.top + graphH / 2);
ctx.rotate(-Math.PI / 2);
ctx.fillText('Spectral Radiance', 0, 0);
ctx.restore();
}

let lastTime = 0;
function renderLoop(timestamp) {
    if (!lastTime) lastTime = timestamp;
    let dt = timestamp - lastTime;
    lastTime = timestamp;

    updatePlaybackScanner(dt);
    drawAppGraphics();
    requestAnimationFrame(renderLoop);
}

function bindEvents() {
    DOM.advancedBtn.addEventListener('click', () => {
        const isHidden = DOM.advancedPanel.style.display === 'none' || DOM.advancedPanel.style.display ===
        DOM.advancedPanel.style.display = isHidden ? 'flex' : 'none';
    });

    DOM.tempSlider.addEventListener('input', (e) => {
        appState.temp = Math.round(Math.exp(parseFloat(e.target.value)));
        DOM.tempInput.value = appState.temp;
        triggerPropertyTone();
    });

    DOM.tempInput.addEventListener('input', (e) => {
        let v = parseInt(e.target.value) || SUN_TEMP;
        v = Math.max(2300, Math.min(55000, v));
        appState.temp = v;
        DOM.tempSlider.value = Math.log(v);
        triggerPropertyTone();
    });

    DOM.radiusSlider.addEventListener('input', (e) => {
        appState.radius = parseFloat(e.target.value);
        DOM.radiusInput.value = appState.radius.toFixed(1);
        triggerPropertyTone();
    });

    DOM.radiusInput.addEventListener('input', (e) => {
        let v = parseFloat(e.target.value) || 1.0;
        v = Math.max(0.1, Math.min(10.0, v));
        appState.radius = v;
        DOM.radiusSlider.value = v;
        triggerPropertyTone();
    });

    // Advanced Toggles
    DOM.toggleSound.addEventListener('change', (e) => {
        appState.isSoundEnabled = e.target.checked;
        DOM.playbackControls.style.display = appState.isSoundEnabled ? "flex" : "none";
        if (appState.isSoundEnabled) {
            initAudioEngine();
        } else if (appState.isPlaying) {
            togglePlayback();
        }
    });

    DOM.toggleByEye.addEventListener('change', (e) => { appState.showByEye = e.target.checked; });
}

```

```

DOM.logXCheck.addEventListener('change', (e) => { appState.isLogX = e.target.checked; });
DOM.logYCheck.addEventListener('change', (e) => { appState.isLogY = e.target.checked; });

DOM.xMinInput.addEventListener('change', (e) => {
  let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MIN;
  if(v >= appState.xMax) v = appState.xMax - 1;
  appState.xMin = Math.max(1, v);
  DOM.xMinInput.value = appState.xMin;
});

DOM.xMaxInput.addEventListener('change', (e) => {
  let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MAX;
  if(v <= appState.xMin) v = appState.xMin + 1;
  appState.xMax = v;
  DOM.xMaxInput.value = appState.xMax;
});

DOM.yMaxInput.addEventListener('change', (e) => {
  let v = parseFloat(e.target.value);
  if (isNaN(v) || v <= 0) {
    appState.customYMax = null;
  } else {
    appState.customYMax = Math.max(1e-10, v * 1e-11);
  }
});

DOM.btnResetLimits.addEventListener('click', () => {
  appState.xMin = DEFAULT_LAMBDA_MIN;
  appState.xMax = DEFAULT_LAMBDA_MAX;
  appState.customYMax = null;
  DOM.xMinInput.value = appState.xMin;
  DOM.xMaxInput.value = appState.xMax;
  DOM.yMaxInput.value = "";
});

// Playback Events
DOM.btnPlay.addEventListener('click', togglePlayback);
DOM.btnReset.addEventListener('click', () => {
  appState.scanWavelength = appState.xMin;
  appState.isPlaying = false;
  DOM.btnPlay.innerHTML = "Play Spectrum";
  DOM.btnPlay.style.backgroundColor = "var(--sound-accent)";
});
DOM.btnSlower.addEventListener('click', () => {
  appState.playbackSpeedMultiplier = Math.max(0.25, appState.playbackSpeedMultiplier - 0.25);
  DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
});
DOM.btnFaster.addEventListener('click', () => {
  appState.playbackSpeedMultiplier = Math.min(4.0, appState.playbackSpeedMultiplier + 0.25);
  DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
});

// Interactive Canvas Mapping
DOM.specCanvas.addEventListener('mousemove', (e) => {
  const rect = DOM.specCanvas.getBoundingClientRect();
  appState.mouseX = e.clientX - rect.left;
  appState.mouseY = e.clientY - rect.top;

  if (appState.isDraggingScanLine && appState.isSoundEnabled) {
    const dpr = window.devicePixelRatio || 1;
    const padLeft = 110 * dpr;
    const graphW = (rect.width * dpr) - padLeft - (30 * dpr);
    const currentMouseXScaled = appState.mouseX * dpr;

    let pct = (currentMouseXScaled - padLeft) / graphW;
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    let targetWavelength = appState.isLogX
      ? Math.pow(10, logMin + pct * (logMax - logMin))
      : appState.xMin + pct * (appState.xMax - appState.xMin);

    appState.scanWavelength = Math.max(appState.xMin, Math.min(appState.xMax, targetWavelength));
  }
}

```

```

});

DOM.specCanvas.addEventListener('mousedown', (e) => {
  if (!appState.isSoundEnabled) return;
  const rect = DOM.specCanvas.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  const clickXScaled = (e.clientX - rect.left) * dpr;
  const padLeft = 110 * dpr;
  const graphW = (rect.width * dpr) - padLeft - (30 * dpr);

  const getXCoord = (nm) => {
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);
    if (appState.isLogX) {
      return padLeft + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
    }
    return padLeft + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
  };

  if (Math.abs(clickXScaled - getXCoord(appState.scanWavelength)) <= 16 * dpr) {
    appState.isDraggingScanLine = true;
  }
});

window.addEventListener('mouseup', () => { appState.isDraggingScanLine = false; });
DOM.specCanvas.addEventListener('mouseleave', () => {
  appState.mouseX = null;
  appState.mouseY = null;
});

window.addEventListener('resize', triggerResize);
}

window.onload = () => {
  bindEvents();
  triggerResize();
  requestAnimationFrame(renderLoop);
};
</script>
</body>
</html>

```

User prompt: A few more tweaks: 1) change the default Y-scale to non-log scale 2) when the Y-scale is in non-log, set the Y-max to the next factor of 10 over the max. For example, the default is to show a star like the sun (1 R_{sun}, 5800 K), and the maximum spectral radiance is about 844 billion. Set the max to 1 trillion, and have tick-marks at 750, 500, 250 billion. 3) add a toggle to the advanced options to show the Y-scale as "relative brightness", where the max is set to 1 (and tick marks 4) add a toggle to the advanced options to remove the text label from the graphic showing the target star.

Response: Here is the updated, self-contained HTML file incorporating your latest tweaks.

The default Y-scale is now linear, automatically snapping its maximum boundary to clean intervals (like 1, 2, 2.5, 5, or 10 multiplied by a power of ten). Toggles for "Relative Brightness" and "Show Star Text" have been added to the advanced options panel, handling the normalization of the curve to a peak of 1.0 and hiding the visual labels on the star overlay respectively.

Save this as an .html file and open it in your browser:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Stellar Simulator (Embed)</title>
  <style>
    :root {
      --bg-color: #ffffff;
      --panel-bg: #f8f9fa;
      --text-color: #111827;
      --accent-color: #2563eb;
      --border-color: #cbd5e1;
      --sound-accent: #f43f5e;
    }

    body {

```

```

    margin: 0;
    padding: 20px;
    font-family: system-ui, -apple-system, sans-serif;
    background-color: #e2e8f0;
    display: flex;
    justify-content: center;
}

/* Enforce 1000px width and 8:3 Aspect Ratio (1000x375) */
.simulator-embed {
    width: 1000px;
    height: 375px;
    background-color: var(--bg-color);
    color: var(--text-color);
    border: 1px solid var(--border-color);
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    box-sizing: border-box;
    display: flex;
    flex-direction: column;
    position: relative;
}

.top-section {
    display: flex;
    flex: 1;
    padding: 15px;
    gap: 15px;
    height: calc(100% - 45px);
    box-sizing: border-box;
}

/* Left Panel - Vertical Controls */
.left-controls {
    width: 250px;
    background-color: var(--panel-bg);
    border: 1px solid var(--border-color);
    border-radius: 8px;
    padding: 15px;
    display: flex;
    flex-direction: column;
    gap: 15px;
    box-sizing: border-box;
    overflow-y: auto;
}

.control-group {
    display: flex;
    flex-direction: column;
    gap: 5px;
}

.control-group label {
    font-weight: 600;
    font-size: 0.90rem;
}

.control-group input[type="range"] {
    width: 100%;
    cursor: pointer;
}

.control-group input[type="number"] {
    width: 90px;
    padding: 4px;
    border: 1px solid var(--border-color);
    border-radius: 4px;
    text-align: center;
    font-size: 0.9rem;
    align-self: center;
}

/* Playback Controls (Injected in Left Column) */
#playback-controls {
    display: none;
}

```

```

    flex-direction: column;
    gap: 8px;
    background: #fff;
    border: 1px solid var(--sound-accent);
    padding: 10px;
    border-radius: 6px;
    margin-top: auto;
}

.playback-label {
    font-weight: bold;
    color: var(--sound-accent);
    font-size: 0.85rem;
    text-align: center;
    margin-bottom: 4px;
}

.btn-main {
    background-color: var(--sound-accent);
    color: white;
    border: none;
    padding: 6px;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
    width: 100%;
}

.btn-row {
    display: flex;
    gap: 5px;
}

.btn-sub {
    background-color: #e2e8f0;
    border: 1px solid var(--border-color);
    padding: 4px;
    border-radius: 4px;
    flex: 1;
    font-size: 0.8rem;
    cursor: pointer;
}

.speed-text { font-size: 0.8rem; text-align: center; padding-top: 2px; }

/* Right Panel - Graph Display */
.main-display {
    flex: 1;
    position: relative;
    border: 1px solid var(--border-color);
    border-radius: 8px;
    background-color: #ffffff;
    box-sizing: border-box;
}

canvas {
    width: 100%;
    height: 100%;
    display: block;
    border-radius: 8px;
    cursor: crosshair;
}

/* Dynamic Star Overlay */
#star-overlay {
    position: absolute;
    background: rgba(255, 255, 255, 0.85);
    backdrop-filter: blur(4px);
    border: 1px solid var(--border-color);
    border-radius: 8px;
    padding: 10px;
    display: flex;
    flex-direction: column;
    align-items: center;

```

```

    pointer-events: none;
    transition: left 0.3s, right 0.3s;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.overlay-right { right: 20px; top: 15px; left: auto; }
.overlay-left { left: 95px; top: 15px; right: auto; }

.star-sphere-wrapper {
    width: 80px;
    height: 80px;
    display: flex;
    align-items: center;
    justify-content: center;
    background-color: #0b0b0c;
    border-radius: 8px;
    overflow: hidden;
    margin-bottom: 5px;
}

.star-sphere {
    border-radius: 50%;
    flex-shrink: 0;
}

.star-label { font-weight: bold; font-size: 0.85rem; }
.star-meta { font-size: 0.75rem; color: #475569; text-align: center; }

/* Bottom Section & Advanced Panel */
.bottom-section {
    padding: 0 15px 15px 15px;
    box-sizing: border-box;
}

#advanced-btn {
    background-color: var(--panel-bg);
    border: 1px solid var(--border-color);
    padding: 6px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: 600;
    text-align: center;
    transition: background-color 0.2s;
    width: 100%;
    font-size: 0.9rem;
}

#advanced-btn:hover { background-color: #e2e8f0; }

.advanced-panel {
    display: none;
    position: absolute;
    bottom: 45px;
    left: 15px;
    width: calc(100% - 30px);
    background-color: #ffffff;
    border: 1px solid var(--border-color);
    border-radius: 6px;
    padding: 10px 15px;
    box-shadow: 0 -4px 15px rgba(0,0,0,0.15);
    box-sizing: border-box;
    z-index: 100;
    flex-wrap: wrap;
    gap: 15px;
    align-items: center;
    font-size: 0.85rem;
}

.toggle-group {
    display: flex;
    align-items: center;
    gap: 6px;
}

```

```

    .range-input {
      width: 60px;
      padding: 4px;
      border: 1px solid var(--border-color);
      border-radius: 4px;
      text-align: center;
    }

    button.small-btn {
      background-color: #f1f5f9;
      border: 1px solid var(--border-color);
      padding: 4px 10px;
      border-radius: 4px;
      cursor: pointer;
    }
  }
</style>
</head>
<body>

<div class="simulator-embed">
  <div class="top-section">
    <div class="left-controls">
      <h3 style="margin: 0; font-size: 1.1rem; border-bottom: 1px solid var(--border-color); padding-bot

    <div class="control-group">
      <label for="temp-slider">Temperature (K)</label>
      <input type="range" id="temp-slider" min="7.7406" max="10.9151" step="0.0001" value="8.6656">
      <input type="number" id="temp-input" min="2300" max="55000" step="100" value="5800">
    </div>

    <div class="control-group">
      <label for="radius-slider">Radius (R☉)</label>
      <input type="range" id="radius-slider" min="0.1" max="10.0" step="0.1" value="1.0">
      <input type="number" id="radius-input" min="0.1" max="10.0" step="0.1" value="1.0">
    </div>

    <div id="playback-controls">
      <div class="playback-label">Spectrum Audio Scan</div>
      <button id="btn-play" class="btn-main">Play Spectrum</button>
      <div class="btn-row">
        <button id="btn-reset" class="btn-sub">Reset</button>
        <button id="btn-slower" class="btn-sub">Slower</button>
        <button id="btn-faster" class="btn-sub">Faster</button>
      </div>
      <div id="speed-label" class="speed-text">Speed: 1.00x</div>
    </div>
  </div>

  <div class="main-display">
    <canvas id="spectrumCanvas"></canvas>

    <div id="star-overlay" class="overlay-right">
      <div class="star-sphere-wrapper">
        <div id="star-sphere" class="star-sphere"></div>
      </div>
      <div class="star-label" id="star-label">Target Star</div>
      <div class="star-meta" id="star-meta-dynamic">5,800 K<br>1.0 R☉</div>
    </div>
  </div>
</div>

<div class="bottom-section">
  <button id="advanced-btn">Toggle Advanced Display Options</button>
  <div class="advanced-panel" id="advanced-panel">
    <div class="toggle-group" style="font-weight: bold; color: var(--sound-accent);">
      <input type="checkbox" id="toggle-sound">
      <label for="toggle-sound">Sonified</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
      <input type="checkbox" id="toggle-byeye" checked="">
      <label for="toggle-byeye">"By-Eye" Strip</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
      <input type="checkbox" id="toggle-logX">

```

```

        <label for="toggle-logX">Log X Axis</label>
    </div>
    <div class="toggle-group">
        <input type="checkbox" id="toggle-logY">
        <label for="toggle-logY">Log Y Axis</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
        <input type="checkbox" id="toggle-relative">
        <label for="toggle-relative">Relative Brightness</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
        <input type="checkbox" id="toggle-starlabel" checked>
        <label for="toggle-starlabel">Show Star Text</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
        <label>X (nm):</label>
        <input type="number" id="x-min-input" class="range-input" value="10"> to
        <input type="number" id="x-max-input" class="range-input" value="2000">
    </div>
    <div class="toggle-group">
        <label>Y Max Limit:</label>
        <input type="number" id="y-max-input" class="range-input" placeholder="Auto">
    </div>
    <button id="btn-reset-limits" class="small-btn">Reset</button>
</div>
</div>
</div>
<script>
    // App Constants
    const SUN_TEMP = 5800;
    const BASE_SUN_RADIUS_PX = 10;
    const DEFAULT_LAMBDA_MIN = 10;
    const DEFAULT_LAMBDA_MAX = 2000;
    const C1 = 3.741771e16;
    const C2 = 1.438776e7;

    // Dynamic State
    let appState = {
        temp: SUN_TEMP,
        radius: 1.0,
        isLogX: false,
        isLogY: false,
        showByEye: true,
        isRelative: false,
        showStarLabel: true,
        xMin: DEFAULT_LAMBDA_MIN,
        xMax: DEFAULT_LAMBDA_MAX,
        customYMax: null,
        mouseX: null,
        mouseY: null,
        barLogOrders: 2.0,

        // Sonification State
        isSoundEnabled: false,
        isPlaying: false,
        scanWavelength: DEFAULT_LAMBDA_MIN,
        playbackSpeedMultiplier: 1.0,
        audioOrders: 4.0,
        isDraggingScanLine: false
    };

    // DOM Links Cache
    const DOM = {
        tempSlider: document.getElementById('temp-slider'),
        tempInput: document.getElementById('temp-input'),
        radiusSlider: document.getElementById('radius-slider'),
        radiusInput: document.getElementById('radius-input'),

        starSphere: document.getElementById('star-sphere'),
        metaDynamic: document.getElementById('star-meta-dynamic'),
        starLabel: document.getElementById('star-label'),
        starOverlay: document.getElementById('star-overlay'),

```

```

specCanvas: document.getElementById('spectrumCanvas'),

advancedBtn: document.getElementById('advanced-btn'),
advancedPanel: document.getElementById('advanced-panel'),

toggleSound: document.getElementById('toggle-sound'),
toggleByEye: document.getElementById('toggle-byeye'),
logXCheck: document.getElementById('toggle-logX'),
logYCheck: document.getElementById('toggle-logY'),
relativeCheck: document.getElementById('toggle-relative'),
starLabelCheck: document.getElementById('toggle-starlabel'),
xMinInput: document.getElementById('x-min-input'),
xMaxInput: document.getElementById('x-max-input'),
yMaxInput: document.getElementById('y-max-input'),
btnResetLimits: document.getElementById('btn-reset-limits'),

// Playback DOM
playbackControls: document.getElementById('playback-controls'),
btnPlay: document.getElementById('btn-play'),
btnReset: document.getElementById('btn-reset'),
btnSlower: document.getElementById('btn-slower'),
btnFaster: document.getElementById('btn-faster'),
speedLabel: document.getElementById('speed-label')
};

const specCtx = DOM.specCanvas.getContext('2d');

// Web Audio Variables
let audioCtx = null;
let propOsc = null;
let propGain = null;
let scanOsc = null;
let scanGain = null;
let propFadeTimeout = null;

function initAudioEngine() {
  if (audioCtx) return;
  audioCtx = new (window.AudioContext || window.webkitAudioContext)();

  propOsc = audioCtx.createOscillator();
  propOsc.type = 'sine';
  propGain = audioCtx.createGain();
  propGain.gain.setValueAtTime(0, audioCtx.currentTime);
  propOsc.connect(propGain);
  propGain.connect(audioCtx.destination);
  propOsc.start();

  scanOsc = audioCtx.createOscillator();
  scanOsc.type = 'triangle';
  scanGain = audioCtx.createGain();
  scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
  scanOsc.connect(scanGain);
  scanGain.connect(audioCtx.destination);
  scanOsc.start();
}

function evaluatePlanck(nm, temperature) {
  const expTerm = Math.exp(C2 / (nm * temperature));
  if (!isFinite(expTerm) || expTerm <= 1) return 0;
  return C1 / (Math.pow(nm, 5) * (expTerm - 1));
}

function getLuminosity(t, r) {
  return (r * r) * Math.pow(t / SUN_TEMP, 4);
}

function kelvinToRGB(k) {
  let normalizedK = k / 100;
  let r, g, b;

  if (normalizedK <= 66) {
    r = 255;
    g = Math.max(0, Math.min(255, 99.4708 * Math.log(normalizedK) - 161.1196));
    b = normalizedK <= 19 ? 0 : Math.max(0, Math.min(255, 138.5177 * Math.log(normalizedK - 10) - 305.

```

```

    } else {
        r = Math.max(0, Math.min(255, 329.6987 * Math.pow(normalizedK - 60, -0.1332)));
        g = Math.max(0, Math.min(255, 288.1222 * Math.pow(normalizedK - 60, -0.0755)));
        b = 255;
    }
    return { r: Math.round(r), g: Math.round(g), b: Math.round(b) };
}

function wavelengthToRGB(nm) {
    let r = 0, g = 0, b = 0;
    if (nm >= 380 && nm < 440) { r = -(nm - 440) / (440 - 380); b = 1.0; }
    else if (nm >= 440 && nm < 490) { g = (nm - 440) / (490 - 440); b = 1.0; }
    else if (nm >= 490 && nm < 510) { g = 1.0; b = -(nm - 510) / (510 - 490); }
    else if (nm >= 510 && nm < 580) { r = (nm - 510) / (580 - 510); g = 1.0; }
    else if (nm >= 580 && nm < 645) { r = 1.0; g = -(nm - 645) / (645 - 580); }
    else if (nm >= 645 && nm <= 750) { r = 1.0; }
    return { r: Math.round(r * 255), g: Math.round(g * 255), b: Math.round(b * 255) };
}

function stringifyLargeValues(num) {
    if (num === 0) return "0";
    if (num >= 1e12) return (num / 1e12).toFixed(1).replace('.0', '') + " trillion";
    if (num >= 1e9) return (num / 1e9).toFixed(1).replace('.0', '') + " billion";
    if (num >= 1e6) return (num / 1e6).toFixed(1).replace('.0', '') + " million";
    if (num >= 1e3) return (num / 1e3).toFixed(1).replace('.0', '') + " thousand";

    if (num < 1) return parseFloat(num.toPrecision(2)).toString();
    if (num < 10) return num.toFixed(1);
    return num.toFixed(0);
}

// Finds the next "clean" visual boundary for setting automatic linear maximums
function getNiceMax(val) {
    if (val <= 0) return 1.0;
    const exponent = Math.floor(Math.log10(val));
    const fraction = val / Math.pow(10, exponent);

    let niceFraction;
    if (fraction <= 1) niceFraction = 1;
    else if (fraction <= 2) niceFraction = 2;
    else if (fraction <= 2.5) niceFraction = 2.5;
    else if (fraction <= 5) niceFraction = 5;
    else niceFraction = 10;

    return niceFraction * Math.pow(10, exponent);
}

function triggerPropertyTone() {
    if (!appState.isSoundEnabled) return;
    initAudioEngine();
    if (audioCtx.state === 'suspended') audioCtx.resume();

    let peakWavelength = 2.8977719e6 / appState.temp;
    let clampedWavelength = Math.max(appState.xMin, Math.min(appState.xMax, peakWavelength));
    let pct = (clampedWavelength - appState.xMin) / (appState.xMax - appState.xMin);
    let frequency = 1300 * Math.pow((110 / 1300), pct);

    const targetLuminosity = getLuminosity(appState.temp, appState.radius);
    let logL = targetLuminosity > 0 ? Math.log10(targetLuminosity) : -4;
    let scaleAlpha = 0.33 + (logL * 0.15);
    let volumeFactor = Math.max(0.05, Math.min(1.0, scaleAlpha)) * 0.25;

    prop0sc.frequency.setValueAtTime(frequency, audioCtx.currentTime);

    if (propFadeTimeout) clearTimeout(propFadeTimeout);
    propGain.gain.setValueAtTime(volumeFactor, audioCtx.currentTime);

    propFadeTimeout = setTimeout(() => {
        propGain.gain.setValueAtTime(propGain.gain.value, audioCtx.currentTime);
        propGain.gain.linearRampToValueAtTime(0, audioCtx.currentTime + 0.5);
    }, 40);
}

function updatePlaybackScanner(dt) {

```

```

if (appState.isPlaying && !appState.isDraggingScanLine) {
  const baseWavelengthStepPerMs = (appState.xMax - appState.xMin) / 7000;
  appState.scanWavelength += baseWavelengthStepPerMs * dt * appState.playbackSpeedMultiplier;

  if (appState.scanWavelength >= appState.xMax) {
    appState.scanWavelength = appState.xMin;
  }
}

if (appState.isSoundEnabled && (appState.isPlaying || appState.isDraggingScanLine)) {
  initAudioEngine();
  if (audioCtx.state === 'suspended') audioCtx.resume();

  let pct = (appState.scanWavelength - appState.xMin) / (appState.xMax - appState.xMin);
  let frequency = 1300 * Math.pow((110 / 1300), pct);

  const areaModifier = appState.radius * appState.radius;
  let currentIntensity = evaluatePlanck(appState.scanWavelength, appState.temp) * areaModifier;
  let peakWavelength = 2.8977719e6 / appState.temp;
  let peakCurveIntensity = evaluatePlanck(peakWavelength, appState.temp) * areaModifier;
  if (peakCurveIntensity <= 0) peakCurveIntensity = 1.0;

  let ratio = currentIntensity / peakCurveIntensity;
  let scanVolume = 0;
  if (ratio > 0) {
    let logRatio = Math.log10(ratio);
    scanVolume = 1.0 + (logRatio / appState.audioOrders);
    if (scanVolume < 0) scanVolume = 0;
  }

  scanOsc.frequency.setValueAtTime(frequency, audioCtx.currentTime);
  scanGain.gain.setValueAtTime(scanVolume * 0.25, audioCtx.currentTime);
} else {
  if (scanGain) {
    scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
  }
}
}

function togglePlayback() {
  appState.isPlaying = !appState.isPlaying;
  DOM.btnPlay.innerHTML = appState.isPlaying ? "Pause Spectrum" : "Play Spectrum";
  DOM.btnPlay.style.backgroundColor = appState.isPlaying ? "#94a3b8" : "var(--sound-accent)";
  if (appState.isSoundEnabled) initAudioEngine();
}

function triggerResize() {
  const dpr = window.devicePixelRatio || 1;
  const specRect = DOM.specCanvas.parentElement.getBoundingClientRect();
  DOM.specCanvas.width = (specRect.width) * dpr;
  DOM.specCanvas.height = (specRect.height) * dpr;
}

function drawAppGraphics() {
  renderPhysicalSpheres();
  renderSpectrumChart();
}

function renderPhysicalSpheres() {
  const targetRGB = kelvinToRGB(appState.temp);
  const targetCSSColor = `rgb(${targetRGB.r}, ${targetRGB.g}, ${targetRGB.b})`;
  const computedRadius = Math.max(4, BASE_SUN_RADIUS_PX * appState.radius);

  DOM.starSphere.style.backgroundColor = targetCSSColor;
  DOM.starSphere.style.width = `${computedRadius * 2}px`;
  DOM.starSphere.style.height = `${computedRadius * 2}px`;
  DOM.starSphere.style.boxShadow = `0 0 ${10 + computedRadius * 0.2}px ${targetCSSColor}`;
  DOM.metaDynamic.innerHTML = `${appState.temp.toLocaleString()} K<br>${appState.radius.toFixed(1)} R☉`;

  // Hide/Show Star Labels
  DOM.starLabel.style.display = appState.showStarLabel ? "block" : "none";
  DOM.metaDynamic.style.display = appState.showStarLabel ? "block" : "none";
}

```

```

function renderSpectrumChart() {
  const canvas = DOM.specCanvas;
  const ctx = specCtx;

  const w = canvas.width;
  const h = canvas.height;
  if (w <= 0 || h <= 0) return;

  ctx.clearRect(0, 0, w, h);

  const dpr = window.devicePixelRatio || 1;
  const pad = { top: 30 * dpr, right: 30 * dpr, bottom: 45 * dpr, left: 110 * dpr };
  const graphW = w - pad.left - pad.right;
  const graphH = h - pad.top - pad.bottom;

  if (graphW <= 0 || graphH <= 0) return;

  const getXCoord = (nm) => {
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
      return pad.left + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
    }
    return pad.left + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
  };

  const getWavelengthFromX = (x) => {
    const pct = (x - pad.left) / graphW;
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
      return Math.pow(10, logMin + pct * (logMax - logMin));
    }
    return appState.xMin + pct * (appState.xMax - appState.xMin);
  };

  if (appState.showByEye) {
    const stripHeight = 16 * dpr;
    const stripY = pad.top / 2 - stripHeight / 2 + 5 * dpr;

    let systemPeakRadiance = 0;
    const step = Math.max(1, (appState.xMax - appState.xMin) / 1000);
    for (let l = appState.xMin; l <= appState.xMax; l += step) {
      let r = evaluatePlanck(l, appState.temp);
      if (r > systemPeakRadiance) systemPeakRadiance = r;
    }
    if (systemPeakRadiance === 0) systemPeakRadiance = 1.0;

    ctx.save();
    ctx.beginPath();
    ctx.rect(pad.left, stripY, graphW, stripHeight);
    ctx.clip();

    for (let x = pad.left; x < pad.left + graphW; x += Math.max(1, dpr)) {
      const currentNM = getWavelengthFromX(x);
      const intensity = evaluatePlanck(currentNM, appState.temp);

      let brightnessFactor = 0;
      if (intensity > 0) {
        let ratio = intensity / systemPeakRadiance;
        let logRatio = Math.log10(ratio);
        brightnessFactor = 1.0 + (logRatio / appState.barLogOrders);
        if (brightnessFactor < 0) brightnessFactor = 0;
      }

      if (currentNM < 380 || currentNM > 750) {
        let grayscaleIntensity = Math.round(230 * brightnessFactor);
        ctx.fillStyle = `rgb(${grayscaleIntensity}, ${grayscaleIntensity}, ${grayscaleIntensity})`;
      } else {
        let hue = wavelengthToRGB(currentNM);
        ctx.fillStyle = `rgb(${Math.round(hue.r * brightnessFactor)}, ${Math.round(hue.g * brightn

```

```

    ctx.fillRect(x, stripY, Math.max(1, dpr) + 0.5, stripHeight);
  }
  ctx.restore();

  ctx.strokeStyle = "#cbd5e1";
  ctx.lineWidth = 1 * dpr;
  ctx.strokeRect(pad.left, stripY, graphW, stripHeight);
}

const pointsArray = [];
let maxObservedCurveIntensity = 0;
let peakX = pad.left;
const areaModifier = appState.radius * appState.radius;

for (let x = pad.left; x <= pad.left + graphW; x += Math.max(1, dpr * 2)) {
  const nm = getWavelengthFromX(x);
  const rawIntensity = evaluatePlanck(nm, appState.temp) * areaModifier;
  pointsArray.push({ x: x, val: rawIntensity });
  if (rawIntensity > maxObservedCurveIntensity) {
    maxObservedCurveIntensity = rawIntensity;
    peakX = x;
  }
}
if (maxObservedCurveIntensity === 0) maxObservedCurveIntensity = 1.0;

// Apply Relative Brightness scaling if active
if (appState.isRelative) {
  const peakIntensity = maxObservedCurveIntensity;
  pointsArray.forEach(pt => pt.val = pt.val / peakIntensity);
  maxObservedCurveIntensity = 1.0;
}

if (peakX > pad.left + graphW / 2) {
  DOM.starOverlay.className = 'overlay-left';
} else {
  DOM.starOverlay.className = 'overlay-right';
}

let maxGraphY = appState.customYMax ? appState.customYMax : maxObservedCurveIntensity;

// Auto-scale to next clean boundary if linear & auto
if (!appState.isLogY && !appState.customYMax && !appState.isRelative) {
  maxGraphY = getNiceMax(maxObservedCurveIntensity);
} else if (appState.isRelative && !appState.customYMax) {
  maxGraphY = 1.0;
}

const minGraphY = appState.isLogY ? maxGraphY * 1e-4 : 0;

const getYCoord = (intensity) => {
  let pct;
  if (appState.isLogY) {
    let safeVal = Math.max(minGraphY, intensity);
    pct = (Math.log10(safeVal) - Math.log10(minGraphY)) / (Math.log10(maxGraphY) - Math.log10(minGraphY));
  } else {
    pct = intensity / maxGraphY;
  }
  return pad.top + graphH - (Math.max(0, Math.min(1, pct)) * graphH);
};

ctx.strokeStyle = '#e2e8f0';
ctx.lineWidth = 1 * dpr;
ctx.fillStyle = '#475569';
ctx.font = `${11 * dpr}px sans-serif`;

ctx.textAlign = 'center';
ctx.textBaseline = 'top';
let xTicks = [];
if (appState.isLogX) {
  xTicks = [appState.xMin];
  let current = Math.pow(10, Math.ceil(Math.log10(appState.xMin)));
  while (current < appState.xMax) {
    if (current > appState.xMin) xTicks.push(current);
    current *= 10;
  }
}

```

```

    }
    xTicks.push(appState.xMax);
  } else {
    const range = appState.xMax - appState.xMin;
    const step = Math.pow(10, Math.floor(Math.log10(range))) / 2;
    for (let t = appState.xMin; t <= appState.xMax; t += step) {
      xTicks.push(t);
    }
  }
}

xTicks.forEach(tick => {
  const x = getXCoord(tick);
  if (x < pad.left || x > pad.left + graphW) return;
  ctx.beginPath(); ctx.moveTo(x, pad.top); ctx.lineTo(x, pad.top + graphH); ctx.stroke();
  ctx.fillText(`${Math.round(tick)} nm`, x, pad.top + graphH + 8 * dpr);
});

ctx.textAlign = 'right';
ctx.textBaseline = 'middle';
const drawYLabel = (val, yPos) => {
  ctx.beginPath(); ctx.moveTo(pad.left, yPos); ctx.lineTo(pad.left + graphW, yPos); ctx.stroke();

  let labelStr;
  if (appState.isRelative) {
    labelStr = val.toFixed(2);
  } else {
    labelStr = stringifyLargeValues(val * 1e11);
  }

  let parts = labelStr.split(' ');
  if (parts.length > 1) {
    ctx.fillText(parts[0], pad.left - 10 * dpr, yPos - 6 * dpr);
    ctx.fillText(parts[1], pad.left - 10 * dpr, yPos + 6 * dpr);
  } else {
    ctx.fillText(labelStr, pad.left - 10 * dpr, yPos);
  }
};

if (appState.isLogY) {
  let logStart = Math.ceil(Math.log10(minGraphY));
  let logEnd = Math.floor(Math.log10(maxGraphY));
  for (let o = logStart; o <= logEnd; o++) {
    let v = Math.pow(10, o);
    let y = getYCoord(v);
    drawYLabel(v, y);
  }
} else {
  [0, 0.25, 0.5, 0.75, 1.0].forEach(p => {
    let v = p * maxGraphY;
    let y = getYCoord(v);
    drawYLabel(v, y);
  });
}

ctx.strokeStyle = '#94a3b8';
ctx.lineWidth = 1.5 * dpr;
ctx.strokeRect(pad.left, pad.top, graphW, graphH);

// Draw Data Curve (Black)
ctx.beginPath();
let breakPath = true;
pointsArray.forEach(pt => {
  let visible = true;
  if (appState.isLogY && pt.val < minGraphY) visible = false;
  if (pt.val > maxGraphY) visible = false;

  if (visible) {
    let y = getYCoord(pt.val);
    if (breakPath) {
      ctx.moveTo(pt.x, y);
      breakPath = false;
    } else {
      ctx.lineTo(pt.x, y);
    }
  }
});

```

```

    } else {
        breakPath = true;
    }
});

ctx.strokeStyle = '#000000';
ctx.lineWidth = 3 * dpr;
ctx.stroke();

// Audio Scan Indicator Line
if (appState.isSoundEnabled) {
    const scanLineX = getXCoord(appState.scanWavelength);
    if (scanLineX >= pad.left && scanLineX <= pad.left + graphW) {
        ctx.strokeStyle = '#f43f5e';
        ctx.lineWidth = 2 * dpr;
        ctx.beginPath();
        ctx.moveTo(scanLineX, pad.top);
        ctx.lineTo(scanLineX, pad.top + graphH);
        ctx.stroke();

        ctx.beginPath();
        ctx.arc(scanLineX, pad.top, 5 * dpr, 0, Math.PI * 2);
        ctx.fillStyle = '#f43f5e';
        ctx.fill();
    }
}

// Mouse Hover Box
if (appState.mouseX !== null && appState.mouseY !== null && !appState.isDraggingScanLine) {
    const mx = appState.mouseX * dpr;
    const my = appState.mouseY * dpr;

    let closestPt = null;
    let minDist = Infinity;

    pointsArray.forEach(pt => {
        let visible = true;
        if (appState.isLogY && pt.val < minGraphY) visible = false;
        if (visible) {
            let py = getYCoord(pt.val);
            let dist = Math.hypot(pt.x - mx, py - my);
            if (dist < minDist) {
                minDist = dist;
                closestPt = pt;
            }
        }
    });

    if (closestPt && minDist <= 15 * dpr) {
        let py = getYCoord(closestPt.val);

        ctx.beginPath();
        ctx.arc(closestPt.x, py, 5 * dpr, 0, Math.PI * 2);
        ctx.fillStyle = '#000000';
        ctx.fill();
        ctx.strokeStyle = '#ffffff';
        ctx.lineWidth = 1.5 * dpr;
        ctx.stroke();

        const hoverNM = getWavelengthFromX(closestPt.x);
        const stringWavelength = `Wavelength: ${Math.round(hoverNM)} nm`;
        const stringIntensity = appState.isRelative
            ? `Rel. Brightness: ${closestPt.val.toFixed(3)}`
            : `Intensity: ${stringifyLargeValues(closestPt.val * 1e11)}`;

        ctx.font = `${11 * dpr}px sans-serif`;
        const textWidth = Math.max(ctx.measureText(stringWavelength).width, ctx.measureText(stringInte
        const boxW = textWidth + 16 * dpr;
        const boxH = 40 * dpr;

        let boxX = closestPt.x + 12 * dpr;
        let boxY = py - 20 * dpr;

        if (boxX + boxW > w) boxX = closestPt.x - boxW - 12 * dpr;

```

```

    if (boxY < 0) boxY = 10 * dpr;
    if (boxY + boxH > h) boxY = h - boxH - 10 * dpr;

    ctx.fillStyle = 'rgba(15, 23, 42, 0.92)';
    ctx.strokeStyle = '#475569';
    ctx.lineWidth = 1 * dpr;
    ctx.beginPath(); ctx.rect(boxX, boxY, boxW, boxH); ctx.fill(); ctx.stroke();
    ctx.fillStyle = '#f8fafc';
    ctx.textAlign = 'left';
    ctx.textBaseline = 'top';
    ctx.fillText(stringWavelength, boxX + 8 * dpr, boxY + 6 * dpr);
    ctx.fillText(stringIntensity, boxX + 8 * dpr, boxY + 22 * dpr);
  }
}

ctx.fillStyle = '#111827';
ctx.font = `bold ${12 * dpr}px sans-serif`;
ctx.textAlign = 'center';
ctx.textBaseline = 'top';
ctx.fillText('Wavelength (nm)', pad.left + graphW / 2, pad.top + graphH + 25 * dpr);

ctx.save();
ctx.translate(pad.left - 65 * dpr, pad.top + graphH / 2);
ctx.rotate(-Math.PI / 2);
ctx.fillText(appState.isRelative ? 'Relative Brightness' : 'Spectral Radiance', 0, 0);
ctx.restore();
}

let lastTime = 0;
function renderLoop(timestamp) {
  if (!lastTime) lastTime = timestamp;
  let dt = timestamp - lastTime;
  lastTime = timestamp;

  updatePlaybackScanner(dt);
  drawAppGraphics();
  requestAnimationFrame(renderLoop);
}

function bindEvents() {
  DOM.advancedBtn.addEventListener('click', () => {
    const isHidden = DOM.advancedPanel.style.display === 'none' || DOM.advancedPanel.style.display ===
    DOM.advancedPanel.style.display = isHidden ? 'flex' : 'none';
  });

  DOM.tempSlider.addEventListener('input', (e) => {
    appState.temp = Math.round(Math.exp(parseFloat(e.target.value)));
    DOM.tempInput.value = appState.temp;
    triggerPropertyTone();
  });

  DOM.tempInput.addEventListener('input', (e) => {
    let v = parseInt(e.target.value) || SUN_TEMP;
    v = Math.max(2300, Math.min(55000, v));
    appState.temp = v;
    DOM.tempSlider.value = Math.log(v);
    triggerPropertyTone();
  });

  DOM.radiusSlider.addEventListener('input', (e) => {
    appState.radius = parseFloat(e.target.value);
    DOM.radiusInput.value = appState.radius.toFixed(1);
    triggerPropertyTone();
  });

  DOM.radiusInput.addEventListener('input', (e) => {
    let v = parseFloat(e.target.value) || 1.0;
    v = Math.max(0.1, Math.min(10.0, v));
    appState.radius = v;
    DOM.radiusSlider.value = v;
    triggerPropertyTone();
  });
}

// Advanced Toggles

```

```

DOM.toggleSound.addEventListener('change', (e) => {
  appState.isSoundEnabled = e.target.checked;
  DOM.playbackControls.style.display = appState.isSoundEnabled ? "flex" : "none";
  if (appState.isSoundEnabled) {
    initAudioEngine();
  } else if (appState.isPlaying) {
    togglePlayback();
  }
});

DOM.toggleByEye.addEventListener('change', (e) => { appState.showByEye = e.target.checked; });
DOM.logXCheck.addEventListener('change', (e) => { appState.isLogX = e.target.checked; });
DOM.logYCheck.addEventListener('change', (e) => { appState.isLogY = e.target.checked; });
DOM.relativeCheck.addEventListener('change', (e) => { appState.isRelative = e.target.checked; });
DOM.starLabelCheck.addEventListener('change', (e) => { appState.showStarLabel = e.target.checked; });

DOM.xMinInput.addEventListener('change', (e) => {
  let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MIN;
  if(v >= appState.xMax) v = appState.xMax - 1;
  appState.xMin = Math.max(1, v);
  DOM.xMinInput.value = appState.xMin;
});

DOM.xMaxInput.addEventListener('change', (e) => {
  let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MAX;
  if(v <= appState.xMin) v = appState.xMin + 1;
  appState.xMax = v;
  DOM.xMaxInput.value = appState.xMax;
});

DOM.yMaxInput.addEventListener('change', (e) => {
  let v = parseFloat(e.target.value);
  if (isNaN(v) || v <= 0) {
    appState.customYMax = null;
  } else {
    appState.customYMax = Math.max(1e-10, v * 1e-11);
  }
});

DOM.btnResetLimits.addEventListener('click', () => {
  appState.xMin = DEFAULT_LAMBDA_MIN;
  appState.xMax = DEFAULT_LAMBDA_MAX;
  appState.customYMax = null;
  DOM.xMinInput.value = appState.xMin;
  DOM.xMaxInput.value = appState.xMax;
  DOM.yMaxInput.value = "";
});

// Playback Events
DOM.btnPlay.addEventListener('click', togglePlayback);
DOM.btnReset.addEventListener('click', () => {
  appState.scanWavelength = appState.xMin;
  appState.isPlaying = false;
  DOM.btnPlay.innerHTML = "Play Spectrum";
  DOM.btnPlay.style.backgroundColor = "var(--sound-accent)";
});
DOM.btnSlower.addEventListener('click', () => {
  appState.playbackSpeedMultiplier = Math.max(0.25, appState.playbackSpeedMultiplier - 0.25);
  DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
});
DOM.btnFaster.addEventListener('click', () => {
  appState.playbackSpeedMultiplier = Math.min(4.0, appState.playbackSpeedMultiplier + 0.25);
  DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
});

// Interactive Canvas Mapping
DOM.specCanvas.addEventListener('mousemove', (e) => {
  const rect = DOM.specCanvas.getBoundingClientRect();
  appState.mouseX = e.clientX - rect.left;
  appState.mouseY = e.clientY - rect.top;

  if (appState.isDraggingScanLine && appState.isSoundEnabled) {
    const dpr = window.devicePixelRatio || 1;
    const padLeft = 110 * dpr;

```

```

const graphW = (rect.width * dpr) - padLeft - (30 * dpr);
const currentMouseXScaled = appState.mouseX * dpr;

let pct = (currentMouseXScaled - padLeft) / graphW;
let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
let logMax = Math.log10(appState.xMax);

let targetWavelength = appState.isLogX
  ? Math.pow(10, logMin + pct * (logMax - logMin))
  : appState.xMin + pct * (appState.xMax - appState.xMin);

appState.scanWavelength = Math.max(appState.xMin, Math.min(appState.xMax, targetWavelength));
});

DOM.specCanvas.addEventListener('mousedown', (e) => {
  if (!appState.isSoundEnabled) return;
  const rect = DOM.specCanvas.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  const clickXScaled = (e.clientX - rect.left) * dpr;
  const padLeft = 110 * dpr;
  const graphW = (rect.width * dpr) - padLeft - (30 * dpr);

  const getXCoord = (nm) => {
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);
    if (appState.isLogX) {
      return padLeft + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
    }
    return padLeft + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
  };

  if (Math.abs(clickXScaled - getXCoord(appState.scanWavelength)) <= 16 * dpr) {
    appState.isDraggingScanLine = true;
  }
});

window.addEventListener('mouseup', () => { appState.isDraggingScanLine = false; });
DOM.specCanvas.addEventListener('mouseleave', () => {
  appState.mouseX = null;
  appState.mouseY = null;
});

window.addEventListener('resize', triggerResize);
}

window.onload = () => {
  bindEvents();
  triggerResize();
  requestAnimationFrame(renderLoop);
};
</script>
</body>
</html>

```

User prompt: Let's add the ability to overlay two spectra. Add a button below the star properties labeled "compare". When pressed, it has several effects: 1). copy the current graph to a new gray dashed line. Add an annotation next to it with the star temperature and size, and do not update those numbers. Add to the advanced options a toggle to deactivate the annotation. 2) create a new button next to "Compare" labeled "Clear". If hit, this button removes the gray line and its annotation, and any of the "comparison mode" specific functions that will be added into the advanced options 3) have the star temperature and radius continue to draw the solid black line and update the model star graphic in the upper corner. The placement continues its normal behavior, only taking into account the current model 4) the default autoscale behavior is to fit the Y-axis to whichever graph has the higher peak. Manually setting the Y-max works as normal. 5) If relative scaling is active, default autoscale is to whichever spectrum has the highest peak. Add a button to the advanced options to "lock scale" to the current scaling. When clicked, it changes to "unlock scale" and has the effect of returning to autoscaling to the brighter spectrum

Response: Here is the updated HTML file. I have implemented the comparison features exactly as requested, including the ability to lock the relative scale, autoscaling to the brighter curve, and clipping the graph cleanly to its bounding box so locked scales that go "off-screen" look neat.

Save this as an .html file and open it in your browser:

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Stellar Simulator (Embed)</title>
<style>
:root {
  --bg-color: #ffffff;
  --panel-bg: #f8f9fa;
  --text-color: #111827;
  --accent-color: #2563eb;
  --border-color: #cbd5e1;
  --sound-accent: #f43f5e;
  --compare-accent: #475569;
}

body {
  margin: 0;
  padding: 20px;
  font-family: system-ui, -apple-system, sans-serif;
  background-color: #e2e8f0;
  display: flex;
  justify-content: center;
}

.simulator-embed {
  width: 1000px;
  height: 375px;
  background-color: var(--bg-color);
  color: var(--text-color);
  border: 1px solid var(--border-color);
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  box-sizing: border-box;
  display: flex;
  flex-direction: column;
  position: relative;
}

.top-section {
  display: flex;
  flex: 1;
  padding: 15px;
  gap: 15px;
  height: calc(100% - 45px);
  box-sizing: border-box;
}

.left-controls {
  width: 250px;
  background-color: var(--panel-bg);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 15px;
  display: flex;
  flex-direction: column;
  gap: 15px;
  box-sizing: border-box;
  overflow-y: auto;
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 5px;
}

.control-group label {
  font-weight: 600;
  font-size: 0.90rem;
}

.control-group input[type="range"] {
  width: 100%;
  cursor: pointer;
}
```

```
.control-group input[type="number"] {
  width: 90px;
  padding: 4px;
  border: 1px solid var(--border-color);
  border-radius: 4px;
  text-align: center;
  font-size: 0.9rem;
  align-self: center;
}

.btn-row {
  display: flex;
  gap: 5px;
}

.btn-main {
  background-color: var(--sound-accent);
  color: white;
  border: none;
  padding: 6px;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
  width: 100%;
}

.btn-compare {
  background-color: var(--compare-accent);
  color: white;
  border: none;
  padding: 6px;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
  flex: 1;
}

.btn-sub {
  background-color: #e2e8f0;
  border: 1px solid var(--border-color);
  padding: 4px;
  border-radius: 4px;
  flex: 1;
  font-size: 0.8rem;
  cursor: pointer;
  font-weight: 600;
}

#playback-controls {
  display: none;
  flex-direction: column;
  gap: 8px;
  background: #fff;
  border: 1px solid var(--sound-accent);
  padding: 10px;
  border-radius: 6px;
  margin-top: auto;
}

.playback-label {
  font-weight: bold;
  color: var(--sound-accent);
  font-size: 0.85rem;
  text-align: center;
  margin-bottom: 4px;
}

.speed-text { font-size: 0.8rem; text-align: center; padding-top: 2px; }

.main-display {
  flex: 1;
  position: relative;
  border: 1px solid var(--border-color);
  border-radius: 8px;
}
```

```

    background-color: #ffffff;
    box-sizing: border-box;
}

canvas {
    width: 100%;
    height: 100%;
    display: block;
    border-radius: 8px;
    cursor: crosshair;
}

#star-overlay {
    position: absolute;
    background: rgba(255, 255, 255, 0.85);
    backdrop-filter: blur(4px);
    border: 1px solid var(--border-color);
    border-radius: 8px;
    padding: 10px;
    display: flex;
    flex-direction: column;
    align-items: center;
    pointer-events: none;
    transition: left 0.3s, right 0.3s;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.overlay-right { right: 20px; top: 15px; left: auto; }
.overlay-left { left: 95px; top: 15px; right: auto; }

.star-sphere-wrapper {
    width: 80px;
    height: 80px;
    display: flex;
    align-items: center;
    justify-content: center;
    background-color: #0b0b0c;
    border-radius: 8px;
    overflow: hidden;
    margin-bottom: 5px;
}

.star-sphere {
    border-radius: 50%;
    flex-shrink: 0;
}

.star-label { font-weight: bold; font-size: 0.85rem; }
.star-meta { font-size: 0.75rem; color: #475569; text-align: center; }

.bottom-section {
    padding: 0 15px 15px 15px;
    box-sizing: border-box;
}

#advanced-btn {
    background-color: var(--panel-bg);
    border: 1px solid var(--border-color);
    padding: 6px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: 600;
    text-align: center;
    transition: background-color 0.2s;
    width: 100%;
    font-size: 0.9rem;
}

#advanced-btn:hover { background-color: #e2e8f0; }

.advanced-panel {
    display: none;
    position: absolute;
    bottom: 45px;

```

```

    left: 15px;
    width: calc(100% - 30px);
    background-color: #ffffff;
    border: 1px solid var(--border-color);
    border-radius: 6px;
    padding: 10px 15px;
    box-shadow: 0 -4px 15px rgba(0,0,0,0.15);
    box-sizing: border-box;
    z-index: 100;
    flex-wrap: wrap;
    gap: 15px;
    align-items: center;
    font-size: 0.85rem;
  }

  .toggle-group {
    display: flex;
    align-items: center;
    gap: 6px;
  }

  .range-input {
    width: 60px;
    padding: 4px;
    border: 1px solid var(--border-color);
    border-radius: 4px;
    text-align: center;
  }

  button.small-btn {
    background-color: #f1f5f9;
    border: 1px solid var(--border-color);
    padding: 4px 10px;
    border-radius: 4px;
    cursor: pointer;
    font-weight: 600;
  }
  button.small-btn.active {
    background-color: var(--compare-accent);
    color: white;
    border-color: var(--compare-accent);
  }
</style>
</head>
<body>

<div class="simulator-embed">
  <div class="top-section">
    <div class="left-controls">
      <h3 style="margin: 0; font-size: 1.1rem; border-bottom: 1px solid var(--border-color); padding-bot

      <div class="control-group">
        <label for="temp-slider">Temperature (K)</label>
        <input type="range" id="temp-slider" min="7.7406" max="10.9151" step="0.0001" value="8.6656">
        <input type="number" id="temp-input" min="2300" max="55000" step="100" value="5800">
      </div>

      <div class="control-group">
        <label for="radius-slider">Radius (R⊙)</label>
        <input type="range" id="radius-slider" min="0.1" max="10.0" step="0.1" value="1.0">
        <input type="number" id="radius-input" min="0.1" max="10.0" step="0.1" value="1.0">
      </div>

      <div class="control-group btn-row" style="margin-top: 5px;">
        <button id="btn-compare" class="btn-compare">Compare</button>
        <button id="btn-clear" class="btn-sub" style="display: none;">Clear</button>
      </div>

      <div id="playback-controls">
        <div class="playback-label">Spectrum Audio Scan</div>
        <button id="btn-play" class="btn-main">Play Spectrum</button>
        <div class="btn-row">
          <button id="btn-reset" class="btn-sub">Reset</button>
          <button id="btn-slower" class="btn-sub">Slower</button>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

        <button id="btn-faster" class="btn-sub">Faster</button>
    </div>
    <div id="speed-label" class="speed-text">Speed: 1.00x</div>
</div>
</div>

<div class="main-display">
    <canvas id="spectrumCanvas"></canvas>

    <div id="star-overlay" class="overlay-right">
        <div class="star-sphere-wrapper">
            <div id="star-sphere" class="star-sphere"></div>
        </div>
        <div class="star-label" id="star-label">Target Star</div>
        <div class="star-meta" id="star-meta-dynamic">5,800 K<br>1.0 R☉</div>
    </div>
</div>
</div>

<div class="bottom-section">
    <button id="advanced-btn">Toggle Advanced Display Options</button>
    <div class="advanced-panel" id="advanced-panel">
        <div class="toggle-group" style="font-weight: bold; color: var(--sound-accent);">
            <input type="checkbox" id="toggle-sound">
            <label for="toggle-sound">Sonified</label>
        </div>
        <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
            <input type="checkbox" id="toggle-byeye" checked="">
            <label for="toggle-byeye">"By-Eye" Strip</label>
        </div>
        <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
            <input type="checkbox" id="toggle-logX">
            <label for="toggle-logX">Log X Axis</label>
        </div>
        <div class="toggle-group">
            <input type="checkbox" id="toggle-logY">
            <label for="toggle-logY">Log Y Axis</label>
        </div>
        <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
            <input type="checkbox" id="toggle-relative" id="toggle-relative">
            <label for="toggle-relative">Relative Brightness</label>
        </div>
        <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
            <input type="checkbox" id="toggle-starlabel" checked="">
            <label for="toggle-starlabel">Show Star Text</label>
        </div>

        <div class="toggle-group" id="compare-advanced-group" style="display: none; margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
            <input type="checkbox" id="toggle-comparetext" checked="">
            <label for="toggle-comparetext">Compare Text</label>
            <button id="btn-lock-scale" class="small-btn" style="margin-left: 8px;">Lock Scale</button>
        </div>

        <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
            <label>X (nm):</label>
            <input type="number" id="x-min-input" class="range-input" value="10"> to
            <input type="number" id="x-max-input" class="range-input" value="2000">
        </div>
        <div class="toggle-group">
            <label>Y Max Limit:</label>
            <input type="number" id="y-max-input" class="range-input" placeholder="Auto">
        </div>
        <button id="btn-reset-limits" class="small-btn">Reset View</button>
    </div>
</div>
</div>

<script>
    // App Constants
    const SUN_TEMP = 5800;
    const BASE_SUN_RADIUS_PX = 10;
    const DEFAULT_LAMBDA_MIN = 10;
    const DEFAULT_LAMBDA_MAX = 2000;
    const C1 = 3.741771e16;

```

```

const C2 = 1.438776e7;

// Dynamic State
let appState = {
  temp: SUN_TEMP,
  radius: 1.0,
  isLogX: false,
  isLogY: false,
  showByEye: true,
  isRelative: false,
  showStarLabel: true,
  xMin: DEFAULT_LAMBDA_MIN,
  xMax: DEFAULT_LAMBDA_MAX,
  customYMax: null,
  mouseX: null,
  mouseY: null,
  barLogOrders: 2.0,

  // Compare Mode State
  compareMode: false,
  compareTemp: null,
  compareRadius: null,
  showCompareText: true,
  scaleLocked: false,
  lockedMaxGraphY: null,
  lockedReferenceIntensity: null,
  currentMaxGraphY: null,
  currentReferenceIntensity: null,

  // Sonification State
  isSoundEnabled: false,
  isPlaying: false,
  scanWavelength: DEFAULT_LAMBDA_MIN,
  playbackSpeedMultiplier: 1.0,
  audioOrders: 4.0,
  isDraggingScanLine: false
};

// DOM Links Cache
const DOM = {
  tempSlider: document.getElementById('temp-slider'),
  tempInput: document.getElementById('temp-input'),
  radiusSlider: document.getElementById('radius-slider'),
  radiusInput: document.getElementById('radius-input'),
  btnCompare: document.getElementById('btn-compare'),
  btnClear: document.getElementById('btn-clear'),

  starSphere: document.getElementById('star-sphere'),
  metaDynamic: document.getElementById('star-meta-dynamic'),
  starLabel: document.getElementById('star-label'),
  starOverlay: document.getElementById('star-overlay'),

  specCanvas: document.getElementById('spectrumCanvas'),

  advancedBtn: document.getElementById('advanced-btn'),
  advancedPanel: document.getElementById('advanced-panel'),

  toggleSound: document.getElementById('toggle-sound'),
  toggleByEye: document.getElementById('toggle-byeye'),
  logXCheck: document.getElementById('toggle-logX'),
  logYCheck: document.getElementById('toggle-logY'),
  relativeCheck: document.getElementById('toggle-relative'),
  starLabelCheck: document.getElementById('toggle-starlabel'),

  compareAdvancedGroup: document.getElementById('compare-advanced-group'),
  toggleCompareText: document.getElementById('toggle-comparetext'),
  btnLockScale: document.getElementById('btn-lock-scale'),

  xMinInput: document.getElementById('x-min-input'),
  xMaxInput: document.getElementById('x-max-input'),
  yMaxInput: document.getElementById('y-max-input'),
  btnResetLimits: document.getElementById('btn-reset-limits'),

  // Playback DOM

```

```

    playbackControls: document.getElementById('playback-controls'),
    btnPlay: document.getElementById('btn-play'),
    btnReset: document.getElementById('btn-reset'),
    btnSlower: document.getElementById('btn-slower'),
    btnFaster: document.getElementById('btn-faster'),
    speedLabel: document.getElementById('speed-label')
  };

  const specCtx = DOM.specCanvas.getContext('2d');

  // Web Audio Variables
  let audioCtx = null;
  let propOsc = null;
  let propGain = null;
  let scanOsc = null;
  let scanGain = null;
  let propFadeTimeout = null;

  function initAudioEngine() {
    if (audioCtx) return;
    audioCtx = new (window.AudioContext || window.webkitAudioContext)();

    propOsc = audioCtx.createOscillator();
    propOsc.type = 'sine';
    propGain = audioCtx.createGain();
    propGain.gain.setValueAtTime(0, audioCtx.currentTime);
    propOsc.connect(propGain);
    propGain.connect(audioCtx.destination);
    propOsc.start();

    scanOsc = audioCtx.createOscillator();
    scanOsc.type = 'triangle';
    scanGain = audioCtx.createGain();
    scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
    scanOsc.connect(scanGain);
    scanGain.connect(audioCtx.destination);
    scanOsc.start();
  }

  function evaluatePlanck(nm, temperature) {
    const expTerm = Math.exp(C2 / (nm * temperature));
    if (!isFinite(expTerm) || expTerm <= 1) return 0;
    return C1 / (Math.pow(nm, 5) * (expTerm - 1));
  }

  function getLuminosity(t, r) {
    return (r * r) * Math.pow(t / SUN_TEMP, 4);
  }

  function kelvinToRGB(k) {
    let normalizedK = k / 100;
    let r, g, b;

    if (normalizedK <= 66) {
      r = 255;
      g = Math.max(0, Math.min(255, 99.4708 * Math.log(normalizedK) - 161.1196));
      b = normalizedK <= 19 ? 0 : Math.max(0, Math.min(255, 138.5177 * Math.log(normalizedK - 10) - 305));
    } else {
      r = Math.max(0, Math.min(255, 329.6987 * Math.pow(normalizedK - 60, -0.1332)));
      g = Math.max(0, Math.min(255, 288.1222 * Math.pow(normalizedK - 60, -0.0755)));
      b = 255;
    }

    return { r: Math.round(r), g: Math.round(g), b: Math.round(b) };
  }

  function wavelengthToRGB(nm) {
    let r = 0, g = 0, b = 0;
    if (nm >= 380 && nm < 440) { r = -(nm - 440) / (440 - 380); b = 1.0; }
    else if (nm >= 440 && nm < 490) { g = (nm - 440) / (490 - 440); b = 1.0; }
    else if (nm >= 490 && nm < 510) { g = 1.0; b = -(nm - 510) / (510 - 490); }
    else if (nm >= 510 && nm < 580) { r = (nm - 510) / (580 - 510); g = 1.0; }
    else if (nm >= 580 && nm < 645) { r = 1.0; g = -(nm - 645) / (645 - 580); }
    else if (nm >= 645 && nm <= 750) { r = 1.0; }
    return { r: Math.round(r * 255), g: Math.round(g * 255), b: Math.round(b * 255) };
  }

```

```

}

function stringifyLargeValues(num) {
  if (num === 0) return "0";
  if (num >= 1e12) return (num / 1e12).toFixed(1).replace('.0', '') + " trillion";
  if (num >= 1e9) return (num / 1e9).toFixed(1).replace('.0', '') + " billion";
  if (num >= 1e6) return (num / 1e6).toFixed(1).replace('.0', '') + " million";
  if (num >= 1e3) return (num / 1e3).toFixed(1).replace('.0', '') + " thousand";

  if (num < 1) return parseFloat(num.toPrecision(2)).toString();
  if (num < 10) return num.toFixed(1);
  return num.toFixed(0);
}

function getNiceMax(val) {
  if (val <= 0) return 1.0;
  const exponent = Math.floor(Math.log10(val));
  const fraction = val / Math.pow(10, exponent);

  let niceFraction;
  if (fraction <= 1) niceFraction = 1;
  else if (fraction <= 2) niceFraction = 2;
  else if (fraction <= 2.5) niceFraction = 2.5;
  else if (fraction <= 5) niceFraction = 5;
  else niceFraction = 10;

  return niceFraction * Math.pow(10, exponent);
}

function triggerPropertyTone() {
  if (!appState.isSoundEnabled) return;
  initAudioEngine();
  if (audioCtx.state === 'suspended') audioCtx.resume();

  let peakWavelength = 2.8977719e6 / appState.temp;
  let clampedWavelength = Math.max(appState.xMin, Math.min(appState.xMax, peakWavelength));
  let pct = (clampedWavelength - appState.xMin) / (appState.xMax - appState.xMin);
  let frequency = 1300 * Math.pow((110 / 1300), pct);

  const targetLuminosity = getLuminosity(appState.temp, appState.radius);
  let logL = targetLuminosity > 0 ? Math.log10(targetLuminosity) : -4;
  let scaleAlpha = 0.33 + (logL * 0.15);
  let volumeFactor = Math.max(0.05, Math.min(1.0, scaleAlpha)) * 0.25;

  propOsc.frequency.setValueAtTime(frequency, audioCtx.currentTime);

  if (propFadeTimeout) clearTimeout(propFadeTimeout);
  propGain.gain.setValueAtTime(volumeFactor, audioCtx.currentTime);

  propFadeTimeout = setTimeout(() => {
    propGain.gain.setValueAtTime(propGain.gain.value, audioCtx.currentTime);
    propGain.gain.linearRampToValueAtTime(0, audioCtx.currentTime + 0.5);
  }, 40);
}

function updatePlaybackScanner(dt) {
  if (appState.isPlaying && !appState.isDraggingScanLine) {
    const baseWavelengthStepPerMs = (appState.xMax - appState.xMin) / 7000;
    appState.scanWavelength += baseWavelengthStepPerMs * dt * appState.playbackSpeedMultiplier;

    if (appState.scanWavelength >= appState.xMax) {
      appState.scanWavelength = appState.xMin;
    }
  }

  if (appState.isSoundEnabled && (appState.isPlaying || appState.isDraggingScanLine)) {
    initAudioEngine();
    if (audioCtx.state === 'suspended') audioCtx.resume();

    let pct = (appState.scanWavelength - appState.xMin) / (appState.xMax - appState.xMin);
    let frequency = 1300 * Math.pow((110 / 1300), pct);

    const areaModifier = appState.radius * appState.radius;
    let currentIntensity = evaluatePlanck(appState.scanWavelength, appState.temp) * areaModifier;
  }
}

```

```

    let peakWavelength = 2.8977719e6 / appState.temp;
    let peakCurveIntensity = evaluatePlanck(peakWavelength, appState.temp) * areaModifier;
    if (peakCurveIntensity <= 0) peakCurveIntensity = 1.0;

    let ratio = currentIntensity / peakCurveIntensity;
    let scanVolume = 0;
    if (ratio > 0) {
        let logRatio = Math.log10(ratio);
        scanVolume = 1.0 + (logRatio / appState.audioOrders);
        if (scanVolume < 0) scanVolume = 0;
    }

    scanOsc.frequency.setValueAtTime(frequency, audioCtx.currentTime);
    scanGain.gain.setValueAtTime(scanVolume * 0.25, audioCtx.currentTime);
} else {
    if (scanGain) {
        scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
    }
}
}

function togglePlayback() {
    appState.isPlaying = !appState.isPlaying;
    DOM.btnPlay.innerHTML = appState.isPlaying ? "Pause Spectrum" : "Play Spectrum";
    DOM.btnPlay.style.backgroundColor = appState.isPlaying ? "#94a3b8" : "var(--sound-accent)";
    if (appState.isSoundEnabled) initAudioEngine();
}

function triggerResize() {
    const dpr = window.devicePixelRatio || 1;
    const specRect = DOM.specCanvas.parentElement.getBoundingClientRect();
    DOM.specCanvas.width = (specRect.width) * dpr;
    DOM.specCanvas.height = (specRect.height) * dpr;
}

function drawAppGraphics() {
    renderPhysicalSpheres();
    renderSpectrumChart();
}

function renderPhysicalSpheres() {
    const targetRGB = kelvinToRGB(appState.temp);
    const targetCSSColor = `rgb(${targetRGB.r}, ${targetRGB.g}, ${targetRGB.b})`;
    const computedRadius = Math.max(4, BASE_SUN_RADIUS_PX * appState.radius);

    DOM.starSphere.style.backgroundColor = targetCSSColor;
    DOM.starSphere.style.width = `${computedRadius * 2}px`;
    DOM.starSphere.style.height = `${computedRadius * 2}px`;
    DOM.starSphere.style.boxShadow = `0 0 ${10 + computedRadius * 0.2}px ${targetCSSColor}`;
    DOM.metaDynamic.innerHTML = `${appState.temp.toLocaleString()} K<br>${appState.radius.toFixed(1)} R`;

    DOM.starLabel.style.display = appState.showStarLabel ? "block" : "none";
    DOM.metaDynamic.style.display = appState.showStarLabel ? "block" : "none";
}

function renderSpectrumChart() {
    const canvas = DOM.specCanvas;
    const ctx = specCtx;

    const w = canvas.width;
    const h = canvas.height;
    if (w <= 0 || h <= 0) return;

    ctx.clearRect(0, 0, w, h);

    const dpr = window.devicePixelRatio || 1;
    const pad = { top: 30 * dpr, right: 30 * dpr, bottom: 45 * dpr, left: 110 * dpr };
    const graphW = w - pad.left - pad.right;
    const graphH = h - pad.top - pad.bottom;

    if (graphW <= 0 || graphH <= 0) return;

    const getXCoord = (nm) => {
        let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);

```

```

    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
        return pad.left + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
    }
    return pad.left + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
};

const getWavelengthFromX = (x) => {
    const pct = (x - pad.left) / graphW;
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
        return Math.pow(10, logMin + pct * (logMax - logMin));
    }
    return appState.xMin + pct * (appState.xMax - appState.xMin);
};

// Render By-Eye Strip
if (appState.showByEye) {
    const stripHeight = 16 * dpr;
    const stripY = pad.top / 2 - stripHeight / 2 + 5 * dpr;

    let systemPeakRadiance = 0;
    const step = Math.max(1, (appState.xMax - appState.xMin) / 1000);
    for (let l = appState.xMin; l <= appState.xMax; l += step) {
        let r = evaluatePlanck(l, appState.temp);
        if (r > systemPeakRadiance) systemPeakRadiance = r;
    }
    if (systemPeakRadiance === 0) systemPeakRadiance = 1.0;

    ctx.save();
    ctx.beginPath();
    ctx.rect(pad.left, stripY, graphW, stripHeight);
    ctx.clip();

    for (let x = pad.left; x < pad.left + graphW; x += Math.max(1, dpr)) {
        const currentNM = getWavelengthFromX(x);
        const intensity = evaluatePlanck(currentNM, appState.temp);

        let brightnessFactor = 0;
        if (intensity > 0) {
            let ratio = intensity / systemPeakRadiance;
            let logRatio = Math.log10(ratio);
            brightnessFactor = 1.0 + (logRatio / appState.barLogOrders);
            if (brightnessFactor < 0) brightnessFactor = 0;
        }

        if (currentNM < 380 || currentNM > 750) {
            let grayscaleIntensity = Math.round(230 * brightnessFactor);
            ctx.fillStyle = `rgb(${grayscaleIntensity}, ${grayscaleIntensity}, ${grayscaleIntensity})`;
        } else {
            let hue = wavelengthToRGB(currentNM);
            ctx.fillStyle = `rgb(${Math.round(hue.r * brightnessFactor)}, ${Math.round(hue.g * brightn
            r * brightnessFactor)}, ${Math.round(hue.b * brightnessFactor)})`;
        }
        ctx.fillRect(x, stripY, Math.max(1, dpr) + 0.5, stripHeight);
    }
    ctx.restore();

    ctx.strokeStyle = "#cbd5e1";
    ctx.lineWidth = 1 * dpr;
    ctx.strokeRect(pad.left, stripY, graphW, stripHeight);
}

// Active Star Data
const pointsArray = [];
let maxObservedCurveIntensity = 0;
let peakX = pad.left;
const areaModifier = appState.radius * appState.radius;

for (let x = pad.left; x <= pad.left + graphW; x += Math.max(1, dpr * 2)) {
    const nm = getWavelengthFromX(x);
    const rawIntensity = evaluatePlanck(nm, appState.temp) * areaModifier;

```

```

    pointsArray.push({ x: x, val: rawIntensity });
    if (rawIntensity > maxObservedCurveIntensity) {
        maxObservedCurveIntensity = rawIntensity;
        peakX = x;
    }
}

// Compare Star Data
const comparePointsArray = [];
let compareMaxIntensity = 0;
let comparePeakX = pad.left;

if (appState.compareMode) {
    const compareAreaModifier = appState.compareRadius * appState.compareRadius;
    for (let x = pad.left; x <= pad.left + graphW; x += Math.max(1, dpr * 2)) {
        const nm = getWavelengthFromX(x);
        const rawIntensity = evaluatePlanck(nm, appState.compareTemp) * compareAreaModifier;
        comparePointsArray.push({ x: x, val: rawIntensity });
        if (rawIntensity > compareMaxIntensity) {
            compareMaxIntensity = rawIntensity;
            comparePeakX = x;
        }
    }
}

// Setup Max Values & Scaling
let absoluteMaxIntensity = appState.compareMode ? Math.max(maxObservedCurveIntensity, compareMaxIntensity) : maxObservedCurveIntensity;
if (absoluteMaxIntensity === 0) absoluteMaxIntensity = 1.0;

let referenceIntensity = absoluteMaxIntensity;
if (appState.scaleLocked && appState.lockedReferenceIntensity) {
    referenceIntensity = appState.lockedReferenceIntensity;
} else {
    appState.currentReferenceIntensity = absoluteMaxIntensity;
}

// Apply Relative Scaling if requested
if (appState.isRelative) {
    pointsArray.forEach(pt => pt.val = pt.val / referenceIntensity);
    if (appState.compareMode) {
        comparePointsArray.forEach(pt => pt.val = pt.val / referenceIntensity);
    }
}

// Define Y Axis Top Boundary
let maxGraphY = appState.customYMax;
if (!maxGraphY) {
    if (appState.scaleLocked && appState.lockedMaxGraphY) {
        maxGraphY = appState.lockedMaxGraphY;
    } else {
        if (appState.isRelative) {
            maxGraphY = 1.0;
        } else if (!appState.isLogY) {
            maxGraphY = getNiceMax(absoluteMaxIntensity);
        } else {
            maxGraphY = absoluteMaxIntensity;
        }
    }
}
appState.currentMaxGraphY = maxGraphY;
const minGraphY = appState.isLogY ? maxGraphY * 1e-4 : 0;

if (peakX > pad.left + graphW / 2) {
    DOM.starOverlay.className = 'overlay-left';
} else {
    DOM.starOverlay.className = 'overlay-right';
}

const getYCoord = (intensity) => {
    let pct;
    if (appState.isLogY) {
        let safeVal = Math.max(minGraphY, intensity);
        pct = (Math.log10(safeVal) - Math.log10(minGraphY)) / (Math.log10(maxGraphY) - Math.log10(minGraphY));
    } else {

```

```

    pct = intensity / maxGraphY;
  }
  return pad.top + graphH - (Math.max(0, pct) * graphH);
  // We allow pct > 1 so points extend above chart visually prior to clipping
};

// Draw Axes & Grids
ctx.strokeStyle = '#e2e8f0';
ctx.lineWidth = 1 * dpr;
ctx.fillStyle = '#475569';
ctx.font = `${11 * dpr}px sans-serif`;

ctx.textAlign = 'center';
ctx.textBaseline = 'top';
let xTicks = [];
if (appState.isLogX) {
  xTicks = [appState.xMin];
  let current = Math.pow(10, Math.ceil(Math.log10(appState.xMin)));
  while (current < appState.xMax) {
    if (current > appState.xMin) xTicks.push(current);
    current *= 10;
  }
  xTicks.push(appState.xMax);
} else {
  const range = appState.xMax - appState.xMin;
  const step = Math.pow(10, Math.floor(Math.log10(range))) / 2;
  for (let t = appState.xMin; t <= appState.xMax; t += step) {
    xTicks.push(t);
  }
}

xTicks.forEach(tick => {
  const x = getXCoord(tick);
  if (x < pad.left || x > pad.left + graphW) return;
  ctx.beginPath(); ctx.moveTo(x, pad.top); ctx.lineTo(x, pad.top + graphH); ctx.stroke();
  ctx.fillText(`${Math.round(tick)} nm`, x, pad.top + graphH + 8 * dpr);
});

ctx.textAlign = 'right';
ctx.textBaseline = 'middle';
const drawYLabel = (val, yPos) => {
  ctx.beginPath(); ctx.moveTo(pad.left, yPos); ctx.lineTo(pad.left + graphW, yPos); ctx.stroke();

  let labelStr;
  if (appState.isRelative) {
    labelStr = val.toFixed(2);
  } else {
    labelStr = stringifyLargeValues(val * 1e11);
  }

  let parts = labelStr.split(' ');
  if (parts.length > 1) {
    ctx.fillText(parts[0], pad.left - 10 * dpr, yPos - 6 * dpr);
    ctx.fillText(parts[1], pad.left - 10 * dpr, yPos + 6 * dpr);
  } else {
    ctx.fillText(labelStr, pad.left - 10 * dpr, yPos);
  }
};

if (appState.isLogY) {
  let logStart = Math.ceil(Math.log10(minGraphY));
  let logEnd = Math.floor(Math.log10(maxGraphY));
  for (let o = logStart; o <= logEnd; o++) {
    let v = Math.pow(10, o);
    let y = getYCoord(v);
    drawYLabel(v, y);
  }
} else {
  [0, 0.25, 0.5, 0.75, 1.0].forEach(p => {
    let v = p * maxGraphY;
    let y = getYCoord(v);
    drawYLabel(v, y);
  });
}

```

```

ctx.strokeStyle = '#94a3b8';
ctx.lineWidth = 1.5 * dpr;
ctx.strokeRect(pad.left, pad.top, graphW, graphH);

// Clip graph area for data lines
ctx.save();
ctx.beginPath();
ctx.rect(pad.left, pad.top, graphW, graphH);
ctx.clip();

// Draw Compare Graph (Gray Dashed)
if (appState.compareMode) {
  ctx.beginPath();
  let breakCompare = true;
  comparePointsArray.forEach(pt => {
    if (appState.isLogY && pt.val < minGraphY) {
      breakCompare = true;
    } else {
      let y = getYCoord(pt.val);
      if (breakCompare) {
        ctx.moveTo(pt.x, y);
        breakCompare = false;
      } else {
        ctx.lineTo(pt.x, y);
      }
    }
  });
  ctx.strokeStyle = '#94a3b8';
  ctx.lineWidth = 2 * dpr;
  ctx.setLineDash([6 * dpr, 6 * dpr]);
  ctx.stroke();
  ctx.setLineDash([]); // reset

  // Draw Compare Annotation
  if (appState.showCompareText) {
    let peakVal = appState.isRelative ? (compareMaxIntensity / referenceIntensity) : compareMaxInt
    let peakY = getYCoord(peakVal);

    ctx.fillStyle = '#64748b';
    ctx.font = `bold ${11 * dpr}px sans-serif`;
    ctx.textAlign = 'center';
    ctx.textBaseline = 'bottom';

    let annX = comparePeakX;
    if (annX < pad.left + 50 * dpr) annX = pad.left + 50 * dpr;
    if (annX > pad.left + graphW - 50 * dpr) annX = pad.left + graphW - 50 * dpr;
    let annY = peakY - 10 * dpr;

    if (annY > pad.top) {
      ctx.fillText(`${appState.compareTemp} K, ${appState.compareRadius.toFixed(1)} R☉`, annX, a
    }
  }
}

// Draw Main Data Curve (Black)
ctx.beginPath();
let breakPath = true;
pointsArray.forEach(pt => {
  if (appState.isLogY && pt.val < minGraphY) {
    breakPath = true;
  } else {
    let y = getYCoord(pt.val);
    if (breakPath) {
      ctx.moveTo(pt.x, y);
      breakPath = false;
    } else {
      ctx.lineTo(pt.x, y);
    }
  }
});
ctx.strokeStyle = '#000000';
ctx.lineWidth = 3 * dpr;

```

```

ctx.stroke();

ctx.restore(); // remove clipping

// Audio Scan Indicator Line
if (appState.isSoundEnabled) {
  const scanLineX = getXCoord(appState.scanWavelength);
  if (scanLineX >= pad.left && scanLineX <= pad.left + graphW) {
    ctx.strokeStyle = '#f43f5e';
    ctx.lineWidth = 2 * dpr;
    ctx.beginPath();
    ctx.moveTo(scanLineX, pad.top);
    ctx.lineTo(scanLineX, pad.top + graphH);
    ctx.stroke();

    ctx.beginPath();
    ctx.arc(scanLineX, pad.top, 5 * dpr, 0, Math.PI * 2);
    ctx.fillStyle = '#f43f5e';
    ctx.fill();
  }
}

// Mouse Hover Box (only reads active graph data for simplicity)
if (appState.mouseX !== null && appState.mouseY !== null && !appState.isDraggingScanLine) {
  const mx = appState.mouseX * dpr;
  const my = appState.mouseY * dpr;

  let closestPt = null;
  let minDist = Infinity;

  pointsArray.forEach(pt => {
    let visible = true;
    if (appState.isLogY && pt.val < minGraphY) visible = false;
    if (visible) {
      let py = getYCoord(pt.val);
      if (py >= pad.top && py <= pad.top + graphH) {
        let dist = Math.hypot(pt.x - mx, py - my);
        if (dist < minDist) {
          minDist = dist;
          closestPt = pt;
        }
      }
    }
  });

  if (closestPt && minDist <= 15 * dpr) {
    let py = getYCoord(closestPt.val);

    ctx.beginPath();
    ctx.arc(closestPt.x, py, 5 * dpr, 0, Math.PI * 2);
    ctx.fillStyle = '#000000';
    ctx.fill();
    ctx.strokeStyle = '#ffffff';
    ctx.lineWidth = 1.5 * dpr;
    ctx.stroke();

    const hoverNM = getWavelengthFromX(closestPt.x);
    const stringWavelength = `Wavelength: ${Math.round(hoverNM)} nm`;
    let origVal = appState.isRelative ? closestPt.val * referenceIntensity : closestPt.val;

    const stringIntensity = appState.isRelative
      ? `Rel. Brightness: ${closestPt.val.toFixed(3)}`
      : `Intensity: ${stringifyLargeValues(origVal * 1e11)}`;

    ctx.font = `${11 * dpr}px sans-serif`;
    const textWidth = Math.max(ctx.measureText(stringWavelength).width, ctx.measureText(stringInte
    const boxW = textWidth + 16 * dpr;
    const boxH = 40 * dpr;

    let boxX = closestPt.x + 12 * dpr;
    let boxY = py - 20 * dpr;

    if (boxX + boxW > w) boxX = closestPt.x - boxW - 12 * dpr;
    if (boxY < 0) boxY = 10 * dpr;
  }
}

```

```

        if (boxY + boxH > h) boxY = h - boxH - 10 * dpr;

        ctx.fillStyle = 'rgba(15, 23, 42, 0.92)';
        ctx.strokeStyle = '#475569';
        ctx.lineWidth = 1 * dpr;
        ctx.beginPath(); ctx.rect(boxX, boxY, boxW, boxH); ctx.fill(); ctx.stroke();
        ctx.fillStyle = '#f8fafc';
        ctx.textAlign = 'left';
        ctx.textBaseline = 'top';
        ctx.fillText(stringWavelength, boxX + 8 * dpr, boxY + 6 * dpr);
        ctx.fillText(stringIntensity, boxX + 8 * dpr, boxY + 22 * dpr);
    }
}

ctx.fillStyle = '#111827';
ctx.font = `bold ${12 * dpr}px sans-serif`;
ctx.textAlign = 'center';
ctx.textBaseline = 'top';
ctx.fillText('Wavelength (nm)', pad.left + graphW / 2, pad.top + graphH + 25 * dpr);

ctx.save();
ctx.translate(pad.left - 65 * dpr, pad.top + graphH / 2);
ctx.rotate(-Math.PI / 2);
ctx.fillText(appState.isRelative ? 'Relative Brightness' : 'Spectral Radiance', 0, 0);
ctx.restore();
}

let lastTime = 0;
function renderLoop(timestamp) {
    if (!lastTime) lastTime = timestamp;
    let dt = timestamp - lastTime;
    lastTime = timestamp;

    updatePlaybackScanner(dt);
    drawAppGraphics();
    requestAnimationFrame(renderLoop);
}

function bindEvents() {
    DOM.advancedBtn.addEventListener('click', () => {
        const isHidden = DOM.advancedPanel.style.display === 'none' || DOM.advancedPanel.style.display ===
        DOM.advancedPanel.style.display = isHidden ? 'flex' : 'none';
    });

    DOM.tempSlider.addEventListener('input', (e) => {
        appState.temp = Math.round(Math.exp(parseFloat(e.target.value)));
        DOM.tempInput.value = appState.temp;
        triggerPropertyTone();
    });

    DOM.tempInput.addEventListener('input', (e) => {
        let v = parseInt(e.target.value) || SUN_TEMP;
        v = Math.max(2300, Math.min(55000, v));
        appState.temp = v;
        DOM.tempSlider.value = Math.log(v);
        triggerPropertyTone();
    });

    DOM.radiusSlider.addEventListener('input', (e) => {
        appState.radius = parseFloat(e.target.value);
        DOM.radiusInput.value = appState.radius.toFixed(1);
        triggerPropertyTone();
    });

    DOM.radiusInput.addEventListener('input', (e) => {
        let v = parseFloat(e.target.value) || 1.0;
        v = Math.max(0.1, Math.min(10.0, v));
        appState.radius = v;
        DOM.radiusSlider.value = v;
        triggerPropertyTone();
    });

    // Compare Controls
    DOM.btnCompare.addEventListener('click', () => {

```

```

    appState.compareMode = true;
    appState.compareTemp = appState.temp;
    appState.compareRadius = appState.radius;
    DOM.btnClear.style.display = 'block';
    DOM.compareAdvancedGroup.style.display = 'flex';
  });

  DOM.btnClear.addEventListener('click', () => {
    appState.compareMode = false;
    DOM.btnClear.style.display = 'none';
    DOM.compareAdvancedGroup.style.display = 'none';
    if (appState.scaleLocked) {
      appState.scaleLocked = false;
      appState.lockedMaxGraphY = null;
      appState.lockedReferenceIntensity = null;
      DOM.btnLockScale.innerHTML = "Lock Scale";
      DOM.btnLockScale.classList.remove('active');
    }
  });

  // Advanced Toggles
  DOM.toggleSound.addEventListener('change', (e) => {
    appState.isSoundEnabled = e.target.checked;
    DOM.playbackControls.style.display = appState.isSoundEnabled ? "flex" : "none";
    if (appState.isSoundEnabled) {
      initAudioEngine();
    } else if (appState.isPlaying) {
      togglePlayback();
    }
  });

  DOM.toggleByEye.addEventListener('change', (e) => { appState.showByEye = e.target.checked; });
  DOM.logXCheck.addEventListener('change', (e) => { appState.isLogX = e.target.checked; });
  DOM.logYCheck.addEventListener('change', (e) => { appState.isLogY = e.target.checked; });
  DOM.relativeCheck.addEventListener('change', (e) => { appState.isRelative = e.target.checked; });
  DOM.starLabelCheck.addEventListener('change', (e) => { appState.showStarLabel = e.target.checked; });

  DOM.toggleCompareText.addEventListener('change', (e) => { appState.showCompareText = e.target.checked; });
  DOM.btnLockScale.addEventListener('click', () => {
    appState.scaleLocked = !appState.scaleLocked;
    if (appState.scaleLocked) {
      appState.lockedMaxGraphY = appState.currentMaxGraphY;
      appState.lockedReferenceIntensity = appState.currentReferenceIntensity;
      DOM.btnLockScale.innerHTML = "Unlock Scale";
      DOM.btnLockScale.classList.add('active');
    } else {
      appState.lockedMaxGraphY = null;
      appState.lockedReferenceIntensity = null;
      DOM.btnLockScale.innerHTML = "Lock Scale";
      DOM.btnLockScale.classList.remove('active');
    }
  });

  DOM.xMinInput.addEventListener('change', (e) => {
    let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MIN;
    if(v >= appState.xMax) v = appState.xMax - 1;
    appState.xMin = Math.max(1, v);
    DOM.xMinInput.value = appState.xMin;
  });

  DOM.xMaxInput.addEventListener('change', (e) => {
    let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MAX;
    if(v <= appState.xMin) v = appState.xMin + 1;
    appState.xMax = v;
    DOM.xMaxInput.value = appState.xMax;
  });

  DOM.yMaxInput.addEventListener('change', (e) => {
    let v = parseFloat(e.target.value);
    if (isNaN(v) || v <= 0) {
      appState.customYMax = null;
    } else {
      appState.customYMax = Math.max(1e-10, v * 1e-11);
    }
  });

```

```

});

DOM.btnResetLimits.addEventListener('click', () => {
  appState.xMin = DEFAULT_LAMBDA_MIN;
  appState.xMax = DEFAULT_LAMBDA_MAX;
  appState.customYMax = null;
  DOM.xMinInput.value = appState.xMin;
  DOM.xMaxInput.value = appState.xMax;
  DOM.yMaxInput.value = "";
  if (appState.scaleLocked) DOM.btnLockScale.click();
});

// Playback Events
DOM.btnPlay.addEventListener('click', togglePlayback);
DOM.btnReset.addEventListener('click', () => {
  appState.scanWavelength = appState.xMin;
  appState.isPlaying = false;
  DOM.btnPlay.innerHTML = "Play Spectrum";
  DOM.btnPlay.style.backgroundColor = "var(--sound-accent)";
});
DOM.btnSlower.addEventListener('click', () => {
  appState.playbackSpeedMultiplier = Math.max(0.25, appState.playbackSpeedMultiplier - 0.25);
  DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
});
DOM.btnFaster.addEventListener('click', () => {
  appState.playbackSpeedMultiplier = Math.min(4.0, appState.playbackSpeedMultiplier + 0.25);
  DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
});

// Interactive Canvas Mapping
DOM.specCanvas.addEventListener('mousemove', (e) => {
  const rect = DOM.specCanvas.getBoundingClientRect();
  appState.mouseX = e.clientX - rect.left;
  appState.mouseY = e.clientY - rect.top;

  if (appState.isDraggingScanLine && appState.isSoundEnabled) {
    const dpr = window.devicePixelRatio || 1;
    const padLeft = 110 * dpr;
    const graphW = (rect.width * dpr) - padLeft - (30 * dpr);
    const currentMouseXScaled = appState.mouseX * dpr;

    let pct = (currentMouseXScaled - padLeft) / graphW;
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    let targetWavelength = appState.isLogX
      ? Math.pow(10, logMin + pct * (logMax - logMin))
      : appState.xMin + pct * (appState.xMax - appState.xMin);

    appState.scanWavelength = Math.max(appState.xMin, Math.min(appState.xMax, targetWavelength));
  }
});

DOM.specCanvas.addEventListener('mousedown', (e) => {
  if (!appState.isSoundEnabled) return;
  const rect = DOM.specCanvas.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  const clickXScaled = (e.clientX - rect.left) * dpr;
  const padLeft = 110 * dpr;
  const graphW = (rect.width * dpr) - padLeft - (30 * dpr);

  const getXCoord = (nm) => {
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);
    if (appState.isLogX) {
      return padLeft + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
    }
    return padLeft + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
  };

  if (Math.abs(clickXScaled - getXCoord(appState.scanWavelength)) <= 16 * dpr) {
    appState.isDraggingScanLine = true;
  }
});

```

```

    window.addEventListener('mouseup', () => { appState.isDraggingScanLine = false; });
    DOM.specCanvas.addEventListener('mouseleave', () => {
        appState.mouseX = null;
        appState.mouseY = null;
    });

    window.addEventListener('resize', triggerResize);
}

window.onload = () => {
    bindEvents();
    triggerResize();
    requestAnimationFrame(renderLoop);
};
</script>
</body>
</html>

```

User prompt: 1) Have lock scale always available as an option in the advanced menu 2). When using relative scaling and the compare feature, the annotation for the locked graph is off the top of the display. When the annotation would be off frame, have it shift position to stay visible next to the dashed line. 3). when activating compare mode, default to lock scale. Make a duplicate of the lock scale button to the always-visible options on the left when in compare mode. (continue to have the button in the advanced menu, too)

Response: Here is the updated HTML file incorporating your requests.

I moved the "Lock Scale" button so it's always accessible in the advanced menu, added a duplicate to the main left panel that appears during comparison mode, and set it to lock by default when "Compare" is clicked. Additionally, the annotation text will now detect if it's about to be drawn off the top edge of the graph and safely flip itself down inside the bounding box so it always remains readable.

Save this as an .html file and open it in your browser:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Stellar Simulator (Embed)</title>
  <style>
    :root {
      --bg-color: #ffffff;
      --panel-bg: #f8f9fa;
      --text-color: #111827;
      --accent-color: #2563eb;
      --border-color: #cbd5e1;
      --sound-accent: #f43f5e;
      --compare-accent: #475569;
    }

    body {
      margin: 0;
      padding: 20px;
      font-family: system-ui, -apple-system, sans-serif;
      background-color: #e2e8f0;
      display: flex;
      justify-content: center;
    }

    .simulator-embed {
      width: 1000px;
      height: 375px;
      background-color: var(--bg-color);
      color: var(--text-color);
      border: 1px solid var(--border-color);
      box-shadow: 0 4px 6px rgba(0,0,0,0.1);
      box-sizing: border-box;
      display: flex;
      flex-direction: column;
      position: relative;
    }

    .top-section {
      display: flex;

```

```
    flex: 1;
    padding: 15px;
    gap: 15px;
    height: calc(100% - 45px);
    box-sizing: border-box;
}

.left-controls {
    width: 250px;
    background-color: var(--panel-bg);
    border: 1px solid var(--border-color);
    border-radius: 8px;
    padding: 15px;
    display: flex;
    flex-direction: column;
    gap: 15px;
    box-sizing: border-box;
    overflow-y: auto;
}

.control-group {
    display: flex;
    flex-direction: column;
    gap: 5px;
}

.control-group label {
    font-weight: 600;
    font-size: 0.90rem;
}

.control-group input[type="range"] {
    width: 100%;
    cursor: pointer;
}

.control-group input[type="number"] {
    width: 90px;
    padding: 4px;
    border: 1px solid var(--border-color);
    border-radius: 4px;
    text-align: center;
    font-size: 0.9rem;
    align-self: center;
}

.btn-row {
    display: flex;
    gap: 5px;
}

.btn-main {
    background-color: var(--sound-accent);
    color: white;
    border: none;
    padding: 6px;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
    width: 100%;
}

.btn-compare {
    background-color: var(--compare-accent);
    color: white;
    border: none;
    padding: 6px;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
    flex: 1;
}

.btn-sub {
```

```

    background-color: #e2e8f0;
    border: 1px solid var(--border-color);
    padding: 4px;
    border-radius: 4px;
    flex: 1;
    font-size: 0.8rem;
    cursor: pointer;
    font-weight: 600;
}
.btn-sub.active {
    background-color: var(--compare-accent);
    color: white;
    border-color: var(--compare-accent);
}

#playback-controls {
    display: none;
    flex-direction: column;
    gap: 8px;
    background: #fff;
    border: 1px solid var(--sound-accent);
    padding: 10px;
    border-radius: 6px;
    margin-top: auto;
}

.playback-label {
    font-weight: bold;
    color: var(--sound-accent);
    font-size: 0.85rem;
    text-align: center;
    margin-bottom: 4px;
}

.speed-text { font-size: 0.8rem; text-align: center; padding-top: 2px; }

.main-display {
    flex: 1;
    position: relative;
    border: 1px solid var(--border-color);
    border-radius: 8px;
    background-color: #ffffff;
    box-sizing: border-box;
}

canvas {
    width: 100%;
    height: 100%;
    display: block;
    border-radius: 8px;
    cursor: crosshair;
}

#star-overlay {
    position: absolute;
    background: rgba(255, 255, 255, 0.85);
    backdrop-filter: blur(4px);
    border: 1px solid var(--border-color);
    border-radius: 8px;
    padding: 10px;
    display: flex;
    flex-direction: column;
    align-items: center;
    pointer-events: none;
    transition: left 0.3s, right 0.3s;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.overlay-right { right: 20px; top: 15px; left: auto; }
.overlay-left { left: 95px; top: 15px; right: auto; }

.star-sphere-wrapper {
    width: 80px;
    height: 80px;

```

```

    display: flex;
    align-items: center;
    justify-content: center;
    background-color: #0b0b0c;
    border-radius: 8px;
    overflow: hidden;
    margin-bottom: 5px;
}

.star-sphere {
    border-radius: 50%;
    flex-shrink: 0;
}

.star-label { font-weight: bold; font-size: 0.85rem; }
.star-meta { font-size: 0.75rem; color: #475569; text-align: center; }

.bottom-section {
    padding: 0 15px 15px 15px;
    box-sizing: border-box;
}

#advanced-btn {
    background-color: var(--panel-bg);
    border: 1px solid var(--border-color);
    padding: 6px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: 600;
    text-align: center;
    transition: background-color 0.2s;
    width: 100%;
    font-size: 0.9rem;
}

#advanced-btn:hover { background-color: #e2e8f0; }

.advanced-panel {
    display: none;
    position: absolute;
    bottom: 45px;
    left: 15px;
    width: calc(100% - 30px);
    background-color: #ffffff;
    border: 1px solid var(--border-color);
    border-radius: 6px;
    padding: 10px 15px;
    box-shadow: 0 -4px 15px rgba(0,0,0,0.15);
    box-sizing: border-box;
    z-index: 100;
    flex-wrap: wrap;
    gap: 15px;
    align-items: center;
    font-size: 0.85rem;
}

.toggle-group {
    display: flex;
    align-items: center;
    gap: 6px;
}

.range-input {
    width: 60px;
    padding: 4px;
    border: 1px solid var(--border-color);
    border-radius: 4px;
    text-align: center;
}

button.small-btn {
    background-color: #f1f5f9;
    border: 1px solid var(--border-color);
    padding: 4px 10px;

```

```

        border-radius: 4px;
        cursor: pointer;
        font-weight: 600;
    }
    button.small-btn.active {
        background-color: var(--compare-accent);
        color: white;
        border-color: var(--compare-accent);
    }
</style>
</head>
<body>

<div class="simulator-embed">
  <div class="top-section">
    <div class="left-controls">
      <h3 style="margin: 0; font-size: 1.1rem; border-bottom: 1px solid var(--border-color); padding-bot

    <div class="control-group">
      <label for="temp-slider">Temperature (K)</label>
      <input type="range" id="temp-slider" min="7.7406" max="10.9151" step="0.0001" value="8.6656">
      <input type="number" id="temp-input" min="2300" max="55000" step="100" value="5800">
    </div>

    <div class="control-group">
      <label for="radius-slider">Radius (R☉)</label>
      <input type="range" id="radius-slider" min="0.1" max="10.0" step="0.1" value="1.0">
      <input type="number" id="radius-input" min="0.1" max="10.0" step="0.1" value="1.0">
    </div>

    <div class="control-group btn-row" style="margin-top: 5px;">
      <button id="btn-compare" class="btn-compare">Compare</button>
      <button id="btn-clear" class="btn-sub" style="display: none;">Clear</button>
    </div>

    <div class="control-group btn-row" id="compare-actions-row" style="display: none; margin-top: 2px;">
      <button id="btn-lock-scale-left" class="btn-sub">Lock Scale</button>
    </div>

    <div id="playback-controls">
      <div class="playback-label">Spectrum Audio Scan</div>
      <button id="btn-play" class="btn-main">Play Spectrum</button>
      <div class="btn-row">
        <button id="btn-reset" class="btn-sub">Reset</button>
        <button id="btn-slower" class="btn-sub">Slower</button>
        <button id="btn-faster" class="btn-sub">Faster</button>
      </div>
      <div id="speed-label" class="speed-text">Speed: 1.00x</div>
    </div>

    <div class="main-display">
      <canvas id="spectrumCanvas"></canvas>

      <div id="star-overlay" class="overlay-right">
        <div class="star-sphere-wrapper">
          <div id="star-sphere" class="star-sphere"></div>
        </div>
        <div class="star-label" id="star-label">Target Star</div>
        <div class="star-meta" id="star-meta-dynamic">5,800 K<br>1.0 R☉</div>
      </div>
    </div>
  </div>

  <div class="bottom-section">
    <button id="advanced-btn">Toggle Advanced Display Options</button>
    <div class="advanced-panel" id="advanced-panel">
      <div class="toggle-group" style="font-weight: bold; color: var(--sound-accent);">
        <input type="checkbox" id="toggle-sound">
        <label for="toggle-sound">Sonified</label>
      </div>
      <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
        <input type="checkbox" id="toggle-byeye" checked="">
        <label for="toggle-byeye">"By-Eye" Strip</label>
      </div>
    </div>
  </div>
</div>

```

```

</div>
<div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px;
  <input type="checkbox" id="toggle-logX">
  <label for="toggle-logX">Log X Axis</label>
</div>
<div class="toggle-group">
  <input type="checkbox" id="toggle-logY">
  <label for="toggle-logY">Log Y Axis</label>
</div>
<div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px;
  <input type="checkbox" id="toggle-relative">
  <label for="toggle-relative">Relative Brightness</label>
  <button id="btn-lock-scale" class="small-btn" style="margin-left: 8px;">Lock Scale</button>
</div>
<div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px;
  <input type="checkbox" id="toggle-starlabel" checked>
  <label for="toggle-starlabel">Show Star Text</label>
</div>

<div class="toggle-group" id="compare-advanced-group" style="display: none; margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px;
  <input type="checkbox" id="toggle-comparetext" checked>
  <label for="toggle-comparetext">Compare Text</label>
</div>

<div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px;
  <label>X (nm):</label>
  <input type="number" id="x-min-input" class="range-input" value="10"> to
  <input type="number" id="x-max-input" class="range-input" value="2000">
</div>
<div class="toggle-group">
  <label>Y Max Limit:</label>
  <input type="number" id="y-max-input" class="range-input" placeholder="Auto">
</div>
<button id="btn-reset-limits" class="small-btn">Reset View</button>
</div>
</div>
</div>

<script>
  // App Constants
  const SUN_TEMP = 5800;
  const BASE_SUN_RADIUS_PX = 10;
  const DEFAULT_LAMBDA_MIN = 10;
  const DEFAULT_LAMBDA_MAX = 2000;
  const C1 = 3.741771e16;
  const C2 = 1.438776e7;

  // Dynamic State
  let appState = {
    temp: SUN_TEMP,
    radius: 1.0,
    isLogX: false,
    isLogY: false,
    showByEye: true,
    isRelative: false,
    showStarLabel: true,
    xMin: DEFAULT_LAMBDA_MIN,
    xMax: DEFAULT_LAMBDA_MAX,
    customYMax: null,
    mouseX: null,
    mouseY: null,
    barLogOrders: 2.0,

    // Compare Mode State
    compareMode: false,
    compareTemp: null,
    compareRadius: null,
    showCompareText: true,
    scaleLocked: false,
    lockedMaxGraphY: null,
    lockedReferenceIntensity: null,
    currentMaxGraphY: null,
    currentReferenceIntensity: null,
  };

```

```

// Sonification State
isSoundEnabled: false,
isPlaying: false,
scanWavelength: DEFAULT_LAMBDA_MIN,
playbackSpeedMultiplier: 1.0,
audioOrders: 4.0,
isDraggingScanLine: false
};

// DOM Links Cache
const DOM = {
  tempSlider: document.getElementById('temp-slider'),
  tempInput: document.getElementById('temp-input'),
  radiusSlider: document.getElementById('radius-slider'),
  radiusInput: document.getElementById('radius-input'),
  btnCompare: document.getElementById('btn-compare'),
  btnClear: document.getElementById('btn-clear'),
  compareActionsRow: document.getElementById('compare-actions-row'),
  btnLockScaleLeft: document.getElementById('btn-lock-scale-left'),

  starSphere: document.getElementById('star-sphere'),
  metaDynamic: document.getElementById('star-meta-dynamic'),
  starLabel: document.getElementById('star-label'),
  starOverlay: document.getElementById('star-overlay'),

  specCanvas: document.getElementById('spectrumCanvas'),

  advancedBtn: document.getElementById('advanced-btn'),
  advancedPanel: document.getElementById('advanced-panel'),

  toggleSound: document.getElementById('toggle-sound'),
  toggleByEye: document.getElementById('toggle-byeye'),
  logXCheck: document.getElementById('toggle-logX'),
  logYCheck: document.getElementById('toggle-logY'),
  relativeCheck: document.getElementById('toggle-relative'),
  starLabelCheck: document.getElementById('toggle-starlabel'),

  compareAdvancedGroup: document.getElementById('compare-advanced-group'),
  toggleCompareText: document.getElementById('toggle-comparetext'),
  btnLockScale: document.getElementById('btn-lock-scale'),

  xMinInput: document.getElementById('x-min-input'),
  xMaxInput: document.getElementById('x-max-input'),
  yMaxInput: document.getElementById('y-max-input'),
  btnResetLimits: document.getElementById('btn-reset-limits'),

  // Playback DOM
  playbackControls: document.getElementById('playback-controls'),
  btnPlay: document.getElementById('btn-play'),
  btnReset: document.getElementById('btn-reset'),
  btnSlower: document.getElementById('btn-slower'),
  btnFaster: document.getElementById('btn-faster'),
  speedLabel: document.getElementById('speed-label')
};

const specCtx = DOM.specCanvas.getContext('2d');

// Web Audio Variables
let audioCtx = null;
let propOsc = null;
let propGain = null;
let scanOsc = null;
let scanGain = null;
let propFadeTimeout = null;

function initAudioEngine() {
  if (audioCtx) return;
  audioCtx = new (window.AudioContext || window.webkitAudioContext)();

  propOsc = audioCtx.createOscillator();
  propOsc.type = 'sine';
  propGain = audioCtx.createGain();
  propGain.gain.setValueAtTime(0, audioCtx.currentTime);
  propOsc.connect(propGain);

```

```

propGain.connect(audioCtx.destination);
propOsc.start();

scanOsc = audioCtx.createOscillator();
scanOsc.type = 'triangle';
scanGain = audioCtx.createGain();
scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
scanOsc.connect(scanGain);
scanGain.connect(audioCtx.destination);
scanOsc.start();
}

function evaluatePlanck(nm, temperature) {
  const expTerm = Math.exp(C2 / (nm * temperature));
  if (!isFinite(expTerm) || expTerm <= 1) return 0;
  return C1 / (Math.pow(nm, 5) * (expTerm - 1));
}

function getLuminosity(t, r) {
  return (r * r) * Math.pow(t / SUN_TEMP, 4);
}

function kelvinToRGB(k) {
  let normalizedK = k / 100;
  let r, g, b;

  if (normalizedK <= 66) {
    r = 255;
    g = Math.max(0, Math.min(255, 99.4708 * Math.log(normalizedK) - 161.1196));
    b = normalizedK <= 19 ? 0 : Math.max(0, Math.min(255, 138.5177 * Math.log(normalizedK - 10) - 305.
  } else {
    r = Math.max(0, Math.min(255, 329.6987 * Math.pow(normalizedK - 60, -0.1332)));
    g = Math.max(0, Math.min(255, 288.1222 * Math.pow(normalizedK - 60, -0.0755)));
    b = 255;
  }
  return { r: Math.round(r), g: Math.round(g), b: Math.round(b) };
}

function wavelengthToRGB(nm) {
  let r = 0, g = 0, b = 0;
  if (nm >= 380 && nm < 440) { r = -(nm - 440) / (440 - 380); b = 1.0; }
  else if (nm >= 440 && nm < 490) { g = (nm - 440) / (490 - 440); b = 1.0; }
  else if (nm >= 490 && nm < 510) { g = 1.0; b = -(nm - 510) / (510 - 490); }
  else if (nm >= 510 && nm < 580) { r = (nm - 510) / (580 - 510); g = 1.0; }
  else if (nm >= 580 && nm < 645) { r = 1.0; g = -(nm - 645) / (645 - 580); }
  else if (nm >= 645 && nm <= 750) { r = 1.0; }
  return { r: Math.round(r * 255), g: Math.round(g * 255), b: Math.round(b * 255) };
}

function stringifyLargeValues(num) {
  if (num === 0) return "0";
  if (num >= 1e12) return (num / 1e12).toFixed(1).replace('.0', '') + " trillion";
  if (num >= 1e9) return (num / 1e9).toFixed(1).replace('.0', '') + " billion";
  if (num >= 1e6) return (num / 1e6).toFixed(1).replace('.0', '') + " million";
  if (num >= 1e3) return (num / 1e3).toFixed(1).replace('.0', '') + " thousand";

  if (num < 1) return parseFloat(num.toPrecision(2)).toString();
  if (num < 10) return num.toFixed(1);
  return num.toFixed(0);
}

function getNiceMax(val) {
  if (val <= 0) return 1.0;
  const exponent = Math.floor(Math.log10(val));
  const fraction = val / Math.pow(10, exponent);

  let niceFraction;
  if (fraction <= 1) niceFraction = 1;
  else if (fraction <= 2) niceFraction = 2;
  else if (fraction <= 2.5) niceFraction = 2.5;
  else if (fraction <= 5) niceFraction = 5;
  else niceFraction = 10;

  return niceFraction * Math.pow(10, exponent);
}

```

```

}

function triggerPropertyTone() {
  if (!appState.isSoundEnabled) return;
  initAudioEngine();
  if (audioCtx.state === 'suspended') audioCtx.resume();

  let peakWavelength = 2.8977719e6 / appState.temp;
  let clampedWavelength = Math.max(appState.xMin, Math.min(appState.xMax, peakWavelength));
  let pct = (clampedWavelength - appState.xMin) / (appState.xMax - appState.xMin);
  let frequency = 1300 * Math.pow((110 / 1300), pct);

  const targetLuminosity = getLuminosity(appState.temp, appState.radius);
  let logL = targetLuminosity > 0 ? Math.log10(targetLuminosity) : -4;
  let scaleAlpha = 0.33 + (logL * 0.15);
  let volumeFactor = Math.max(0.05, Math.min(1.0, scaleAlpha)) * 0.25;

  prop0sc.frequency.setValueAtTime(frequency, audioCtx.currentTime);

  if (propFadeTimeout) clearTimeout(propFadeTimeout);
  propGain.gain.setValueAtTime(volumeFactor, audioCtx.currentTime);

  propFadeTimeout = setTimeout(() => {
    propGain.gain.setValueAtTime(propGain.gain.value, audioCtx.currentTime);
    propGain.gain.linearRampToValueAtTime(0, audioCtx.currentTime + 0.5);
  }, 40);
}

function updatePlaybackScanner(dt) {
  if (appState.isPlaying && !appState.isDraggingScanLine) {
    const baseWavelengthStepPerMs = (appState.xMax - appState.xMin) / 7000;
    appState.scanWavelength += baseWavelengthStepPerMs * dt * appState.playbackSpeedMultiplier;

    if (appState.scanWavelength >= appState.xMax) {
      appState.scanWavelength = appState.xMin;
    }
  }

  if (appState.isSoundEnabled && (appState.isPlaying || appState.isDraggingScanLine)) {
    initAudioEngine();
    if (audioCtx.state === 'suspended') audioCtx.resume();

    let pct = (appState.scanWavelength - appState.xMin) / (appState.xMax - appState.xMin);
    let frequency = 1300 * Math.pow((110 / 1300), pct);

    const areaModifier = appState.radius * appState.radius;
    let currentIntensity = evaluatePlanck(appState.scanWavelength, appState.temp) * areaModifier;
    let peakWavelength = 2.8977719e6 / appState.temp;
    let peakCurveIntensity = evaluatePlanck(peakWavelength, appState.temp) * areaModifier;
    if (peakCurveIntensity <= 0) peakCurveIntensity = 1.0;

    let ratio = currentIntensity / peakCurveIntensity;
    let scanVolume = 0;
    if (ratio > 0) {
      let logRatio = Math.log10(ratio);
      scanVolume = 1.0 + (logRatio / appState.audioOrders);
      if (scanVolume < 0) scanVolume = 0;
    }

    scan0sc.frequency.setValueAtTime(frequency, audioCtx.currentTime);
    scanGain.gain.setValueAtTime(scanVolume * 0.25, audioCtx.currentTime);
  } else {
    if (scanGain) {
      scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
    }
  }
}

function togglePlayback() {
  appState.isPlaying = !appState.isPlaying;
  DOM.btnPlay.innerHTML = appState.isPlaying ? "Pause Spectrum" : "Play Spectrum";
  DOM.btnPlay.style.backgroundColor = appState.isPlaying ? "#94a3b8" : "var(--sound-accent)";
  if (appState.isSoundEnabled) initAudioEngine();
}

```

```

function triggerResize() {
  const dpr = window.devicePixelRatio || 1;
  const specRect = DOM.specCanvas.parentElement.getBoundingClientRect();
  DOM.specCanvas.width = (specRect.width) * dpr;
  DOM.specCanvas.height = (specRect.height) * dpr;
}

function drawAppGraphics() {
  renderPhysicalSpheres();
  renderSpectrumChart();
}

function renderPhysicalSpheres() {
  const targetRGB = kelvinToRGB(appState.temp);
  const targetCSSColor = `rgb(${targetRGB.r}, ${targetRGB.g}, ${targetRGB.b})`;
  const computedRadius = Math.max(4, BASE_SUN_RADIUS_PX * appState.radius);

  DOM.starSphere.style.backgroundColor = targetCSSColor;
  DOM.starSphere.style.width = `${computedRadius * 2}px`;
  DOM.starSphere.style.height = `${computedRadius * 2}px`;
  DOM.starSphere.style.boxShadow = `0 0 ${10 + computedRadius * 0.2}px ${targetCSSColor}`;
  DOM.metaDynamic.innerHTML = `${appState.temp.toLocaleString()} K<br>${appState.radius.toFixed(1)} R☉`;

  DOM.starLabel.style.display = appState.showStarLabel ? "block" : "none";
  DOM.metaDynamic.style.display = appState.showStarLabel ? "block" : "none";
}

function renderSpectrumChart() {
  const canvas = DOM.specCanvas;
  const ctx = specCtx;

  const w = canvas.width;
  const h = canvas.height;
  if (w <= 0 || h <= 0) return;

  ctx.clearRect(0, 0, w, h);

  const dpr = window.devicePixelRatio || 1;
  const pad = { top: 30 * dpr, right: 30 * dpr, bottom: 45 * dpr, left: 110 * dpr };
  const graphW = w - pad.left - pad.right;
  const graphH = h - pad.top - pad.bottom;

  if (graphW <= 0 || graphH <= 0) return;

  const getXCoord = (nm) => {
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
      return pad.left + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
    }
    return pad.left + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
  };

  const getWavelengthFromX = (x) => {
    const pct = (x - pad.left) / graphW;
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
      return Math.pow(10, logMin + pct * (logMax - logMin));
    }
    return appState.xMin + pct * (appState.xMax - appState.xMin);
  };

  // Render By-Eye Strip
  if (appState.showByEye) {
    const stripHeight = 16 * dpr;
    const stripY = pad.top / 2 - stripHeight / 2 + 5 * dpr;

    let systemPeakRadiance = 0;
    const step = Math.max(1, (appState.xMax - appState.xMin) / 1000);
    for (let l = appState.xMin; l <= appState.xMax; l += step) {

```

```

        let r = evaluatePlanck(l, appState.temp);
        if (r > systemPeakRadiance) systemPeakRadiance = r;
    }
    if (systemPeakRadiance === 0) systemPeakRadiance = 1.0;

    ctx.save();
    ctx.beginPath();
    ctx.rect(pad.left, stripY, graphW, stripHeight);
    ctx.clip();

    for (let x = pad.left; x < pad.left + graphW; x += Math.max(1, dpr)) {
        const currentNM = getWavelengthFromX(x);
        const intensity = evaluatePlanck(currentNM, appState.temp);

        let brightnessFactor = 0;
        if (intensity > 0) {
            let ratio = intensity / systemPeakRadiance;
            let logRatio = Math.log10(ratio);
            brightnessFactor = 1.0 + (logRatio / appState.barLogOrders);
            if (brightnessFactor < 0) brightnessFactor = 0;
        }

        if (currentNM < 380 || currentNM > 750) {
            let grayscaleIntensity = Math.round(230 * brightnessFactor);
            ctx.fillStyle = `rgb(${grayscaleIntensity}, ${grayscaleIntensity}, ${grayscaleIntensity})`;
        } else {
            let hue = wavelengthToRGB(currentNM);
            ctx.fillStyle = `rgb(${Math.round(hue.r * brightnessFactor)}, ${Math.round(hue.g * brightn
            r * brightnessFactor)}, ${Math.round(hue.b * brightnessFactor)})`;
        }
        ctx.fillRect(x, stripY, Math.max(1, dpr) + 0.5, stripHeight);
    }
    ctx.restore();

    ctx.strokeStyle = "#cbd5e1";
    ctx.lineWidth = 1 * dpr;
    ctx.strokeRect(pad.left, stripY, graphW, stripHeight);
}

// Active Star Data
const pointsArray = [];
let maxObservedCurveIntensity = 0;
let peakX = pad.left;
const areaModifier = appState.radius * appState.radius;

for (let x = pad.left; x <= pad.left + graphW; x += Math.max(1, dpr * 2)) {
    const nm = getWavelengthFromX(x);
    const rawIntensity = evaluatePlanck(nm, appState.temp) * areaModifier;
    pointsArray.push({ x: x, val: rawIntensity });
    if (rawIntensity > maxObservedCurveIntensity) {
        maxObservedCurveIntensity = rawIntensity;
        peakX = x;
    }
}

// Compare Star Data
const comparePointsArray = [];
let compareMaxIntensity = 0;
let comparePeakX = pad.left;

if (appState.compareMode) {
    const compareAreaModifier = appState.compareRadius * appState.compareRadius;
    for (let x = pad.left; x <= pad.left + graphW; x += Math.max(1, dpr * 2)) {
        const nm = getWavelengthFromX(x);
        const rawIntensity = evaluatePlanck(nm, appState.compareTemp) * compareAreaModifier;
        comparePointsArray.push({ x: x, val: rawIntensity });
        if (rawIntensity > compareMaxIntensity) {
            compareMaxIntensity = rawIntensity;
            comparePeakX = x;
        }
    }
}

// Setup Max Values & Scaling
let absoluteMaxIntensity = appState.compareMode ? Math.max(maxObservedCurveIntensity, compareMaxIntens

```

```

if (absoluteMaxIntensity === 0) absoluteMaxIntensity = 1.0;

let referenceIntensity = absoluteMaxIntensity;
if (appState.scaleLocked && appState.lockedReferenceIntensity) {
  referenceIntensity = appState.lockedReferenceIntensity;
} else {
  appState.currentReferenceIntensity = absoluteMaxIntensity;
}

// Apply Relative Scaling if requested
if (appState.isRelative) {
  pointsArray.forEach(pt => pt.val = pt.val / referenceIntensity);
  if (appState.compareMode) {
    comparePointsArray.forEach(pt => pt.val = pt.val / referenceIntensity);
  }
}

// Define Y Axis Top Boundary
let maxGraphY = appState.customYMax;
if (!maxGraphY) {
  if (appState.scaleLocked && appState.lockedMaxGraphY) {
    maxGraphY = appState.lockedMaxGraphY;
  } else {
    if (appState.isRelative) {
      maxGraphY = 1.0;
    } else if (!appState.isLogY) {
      maxGraphY = getNiceMax(absoluteMaxIntensity);
    } else {
      maxGraphY = absoluteMaxIntensity;
    }
  }
}
appState.currentMaxGraphY = maxGraphY;
const minGraphY = appState.isLogY ? maxGraphY * 1e-4 : 0;

if (peakX > pad.left + graphW / 2) {
  DOM.starOverlay.className = 'overlay-left';
} else {
  DOM.starOverlay.className = 'overlay-right';
}

const getYCoord = (intensity) => {
  let pct;
  if (appState.isLogY) {
    let safeVal = Math.max(minGraphY, intensity);
    pct = (Math.log10(safeVal) - Math.log10(minGraphY)) / (Math.log10(maxGraphY) - Math.log10(minGraphY));
  } else {
    pct = intensity / maxGraphY;
  }
  return pad.top + graphH - (Math.max(0, pct) * graphH);
  // We allow pct > 1 so points extend above chart visually prior to clipping
};

// Draw Axes & Grids
ctx.strokeStyle = '#e2e8f0';
ctx.lineWidth = 1 * dpr;
ctx.fillStyle = '#475569';
ctx.font = `${11 * dpr}px sans-serif`;

ctx.textAlign = 'center';
ctx.textBaseline = 'top';
let xTicks = [];
if (appState.isLogX) {
  xTicks = [appState.xMin];
  let current = Math.pow(10, Math.ceil(Math.log10(appState.xMin)));
  while (current < appState.xMax) {
    if (current > appState.xMin) xTicks.push(current);
    current *= 10;
  }
  xTicks.push(appState.xMax);
} else {
  const range = appState.xMax - appState.xMin;
  const step = Math.pow(10, Math.floor(Math.log10(range))) / 2;
  for (let t = appState.xMin; t <= appState.xMax; t += step) {

```

```

        xTicks.push(t);
    }
}

xTicks.forEach(tick => {
    const x = getXCoord(tick);
    if (x < pad.left || x > pad.left + graphW) return;
    ctx.beginPath(); ctx.moveTo(x, pad.top); ctx.lineTo(x, pad.top + graphH); ctx.stroke();
    ctx.fillText(`${Math.round(tick)} nm`, x, pad.top + graphH + 8 * dpr);
});

ctx.textAlign = 'right';
ctx.textBaseline = 'middle';
const drawYLabel = (val, yPos) => {
    ctx.beginPath(); ctx.moveTo(pad.left, yPos); ctx.lineTo(pad.left + graphW, yPos); ctx.stroke();

    let labelStr;
    if (appState.isRelative) {
        labelStr = val.toFixed(2);
    } else {
        labelStr = stringifyLargeValues(val * 1e11);
    }

    let parts = labelStr.split(' ');
    if (parts.length > 1) {
        ctx.fillText(parts[0], pad.left - 10 * dpr, yPos - 6 * dpr);
        ctx.fillText(parts[1], pad.left - 10 * dpr, yPos + 6 * dpr);
    } else {
        ctx.fillText(labelStr, pad.left - 10 * dpr, yPos);
    }
};

if (appState.isLogY) {
    let logStart = Math.ceil(Math.log10(minGraphY));
    let logEnd = Math.floor(Math.log10(maxGraphY));
    for (let o = logStart; o <= logEnd; o++) {
        let v = Math.pow(10, o);
        let y = getYCoord(v);
        drawYLabel(v, y);
    }
} else {
    [0, 0.25, 0.5, 0.75, 1.0].forEach(p => {
        let v = p * maxGraphY;
        let y = getYCoord(v);
        drawYLabel(v, y);
    });
}

ctx.strokeStyle = '#94a3b8';
ctx.lineWidth = 1.5 * dpr;
ctx.strokeRect(pad.left, pad.top, graphW, graphH);

// Clip graph area for data lines
ctx.save();
ctx.beginPath();
ctx.rect(pad.left, pad.top, graphW, graphH);
ctx.clip();

// Draw Compare Graph (Gray Dashed)
if (appState.compareMode) {
    ctx.beginPath();
    let breakCompare = true;
    comparePointsArray.forEach(pt => {
        if (appState.isLogY && pt.val < minGraphY) {
            breakCompare = true;
        } else {
            let y = getYCoord(pt.val);
            if (breakCompare) {
                ctx.moveTo(pt.x, y);
                breakCompare = false;
            } else {
                ctx.lineTo(pt.x, y);
            }
        }
    })
}

```

```

});
ctx.strokeStyle = '#94a3b8';
ctx.lineWidth = 2 * dpr;
ctx.setLineDash([6 * dpr, 6 * dpr]);
ctx.stroke();
ctx.setLineDash([]); // reset

// Draw Compare Annotation
if (appState.showCompareText) {
  let peakVal = appState.isRelative ? (compareMaxIntensity / referenceIntensity) : compareMaxInt
  let peakY = getYCoord(peakVal);

  ctx.fillStyle = '#64748b';
  ctx.font = `bold ${11 * dpr}px sans-serif`;
  ctx.textAlign = 'center';
  ctx.textBaseline = 'bottom';

  let annX = comparePeakX;
  if (annX < pad.left + 50 * dpr) annX = pad.left + 50 * dpr;
  if (annX > pad.left + graphW - 50 * dpr) annX = pad.left + graphW - 50 * dpr;
  let annY = peakY - 10 * dpr;

  // If clipping off top, invert text position so it hangs safely inside the box
  if (annY < pad.top + 5 * dpr) {
    annY = pad.top + 5 * dpr;
    ctx.textBaseline = 'top';
  }

  ctx.fillText(`${appState.compareTemp} K, ${appState.compareRadius.toFixed(1)} R☉`, annX, annY)
}

// Draw Main Data Curve (Black)
ctx.beginPath();
let breakPath = true;
pointsArray.forEach(pt => {
  if (appState.isLogY && pt.val < minGraphY) {
    breakPath = true;
  } else {
    let y = getYCoord(pt.val);
    if (breakPath) {
      ctx.moveTo(pt.x, y);
      breakPath = false;
    } else {
      ctx.lineTo(pt.x, y);
    }
  }
});

ctx.strokeStyle = '#000000';
ctx.lineWidth = 3 * dpr;
ctx.stroke();

ctx.restore(); // remove clipping

// Audio Scan Indicator Line
if (appState.isSoundEnabled) {
  const scanLineX = getXCoord(appState.scanWavelength);
  if (scanLineX >= pad.left && scanLineX <= pad.left + graphW) {
    ctx.strokeStyle = '#f43f5e';
    ctx.lineWidth = 2 * dpr;
    ctx.beginPath();
    ctx.moveTo(scanLineX, pad.top);
    ctx.lineTo(scanLineX, pad.top + graphH);
    ctx.stroke();

    ctx.beginPath();
    ctx.arc(scanLineX, pad.top, 5 * dpr, 0, Math.PI * 2);
    ctx.fillStyle = '#f43f5e';
    ctx.fill();
  }
}

// Mouse Hover Box (only reads active graph data for simplicity)

```

```

if (appState.mouseX !== null && appState.mouseY !== null && !appState.isDraggingScanLine) {
  const mx = appState.mouseX * dpr;
  const my = appState.mouseY * dpr;

  let closestPt = null;
  let minDist = Infinity;

  pointsArray.forEach(pt => {
    let visible = true;
    if (appState.isLogY && pt.val < minGraphY) visible = false;
    if (visible) {
      let py = getYCoord(pt.val);
      if (py >= pad.top && py <= pad.top + graphH) {
        let dist = Math.hypot(pt.x - mx, py - my);
        if (dist < minDist) {
          minDist = dist;
          closestPt = pt;
        }
      }
    }
  });

  if (closestPt && minDist <= 15 * dpr) {
    let py = getYCoord(closestPt.val);

    ctx.beginPath();
    ctx.arc(closestPt.x, py, 5 * dpr, 0, Math.PI * 2);
    ctx.fillStyle = '#000000';
    ctx.fill();
    ctx.strokeStyle = 'ffffff';
    ctx.lineWidth = 1.5 * dpr;
    ctx.stroke();

    const hoverNM = getWavelengthFromX(closestPt.x);
    const stringWavelength = `Wavelength: ${Math.round(hoverNM)} nm`;
    let origVal = appState.isRelative ? closestPt.val * referenceIntensity : closestPt.val;

    const stringIntensity = appState.isRelative
      ? `Rel. Brightness: ${closestPt.val.toFixed(3)}`
      : `Intensity: ${stringifyLargeValues(origVal * 1e11)}`;

    ctx.font = `${11 * dpr}px sans-serif`;
    const textWidth = Math.max(ctx.measureText(stringWavelength).width, ctx.measureText(stringInte
    const boxW = textWidth + 16 * dpr;
    const boxH = 40 * dpr;

    let boxX = closestPt.x + 12 * dpr;
    let boxY = py - 20 * dpr;

    if (boxX + boxW > w) boxX = closestPt.x - boxW - 12 * dpr;
    if (boxY < 0) boxY = 10 * dpr;
    if (boxY + boxH > h) boxY = h - boxH - 10 * dpr;

    ctx.fillStyle = 'rgba(15, 23, 42, 0.92)';
    ctx.strokeStyle = '#475569';
    ctx.lineWidth = 1 * dpr;
    ctx.beginPath(); ctx.rect(boxX, boxY, boxW, boxH); ctx.fill(); ctx.stroke();
    ctx.fillStyle = '#f8fafc';
    ctx.textAlign = 'left';
    ctx.textBaseline = 'top';
    ctx.fillText(stringWavelength, boxX + 8 * dpr, boxY + 6 * dpr);
    ctx.fillText(stringIntensity, boxX + 8 * dpr, boxY + 22 * dpr);
  }
}

ctx.fillStyle = '#111827';
ctx.font = `bold ${12 * dpr}px sans-serif`;
ctx.textAlign = 'center';
ctx.textBaseline = 'top';
ctx.fillText('Wavelength (nm)', pad.left + graphW / 2, pad.top + graphH + 25 * dpr);

ctx.save();
ctx.translate(pad.left - 65 * dpr, pad.top + graphH / 2);
ctx.rotate(-Math.PI / 2);

```

```

    ctx.fillText(appState.isRelative ? 'Relative Brightness' : 'Spectral Radiance', 0, 0);
    ctx.restore();
}

let lastTime = 0;
function renderLoop(timestamp) {
    if (!lastTime) lastTime = timestamp;
    let dt = timestamp - lastTime;
    lastTime = timestamp;

    updatePlaybackScanner(dt);
    drawAppGraphics();
    requestAnimationFrame(renderLoop);
}

function setLockScale(locked) {
    appState.scaleLocked = locked;
    if (locked) {
        appState.lockedMaxGraphY = appState.currentMaxGraphY;
        appState.lockedReferenceIntensity = appState.currentReferenceIntensity;
        DOM.btnLockScale.innerHTML = "Unlock Scale";
        DOM.btnLockScale.classList.add('active');
        DOM.btnLockScaleLeft.innerHTML = "Unlock Scale";
        DOM.btnLockScaleLeft.classList.add('active');
    } else {
        appState.lockedMaxGraphY = null;
        appState.lockedReferenceIntensity = null;
        DOM.btnLockScale.innerHTML = "Lock Scale";
        DOM.btnLockScale.classList.remove('active');
        DOM.btnLockScaleLeft.innerHTML = "Lock Scale";
        DOM.btnLockScaleLeft.classList.remove('active');
    }
}

function toggleLockScale() {
    setLockScale(!appState.scaleLocked);
}

function bindEvents() {
    DOM.advancedBtn.addEventListener('click', () => {
        const isHidden = DOM.advancedPanel.style.display === 'none' || DOM.advancedPanel.style.display === 'flex';
        DOM.advancedPanel.style.display = isHidden ? 'flex' : 'none';
    });

    DOM.tempSlider.addEventListener('input', (e) => {
        appState.temp = Math.round(Math.exp(parseFloat(e.target.value)));
        DOM.tempInput.value = appState.temp;
        triggerPropertyTone();
    });

    DOM.tempInput.addEventListener('input', (e) => {
        let v = parseInt(e.target.value) || SUN_TEMP;
        v = Math.max(2300, Math.min(55000, v));
        appState.temp = v;
        DOM.tempSlider.value = Math.log(v);
        triggerPropertyTone();
    });

    DOM.radiusSlider.addEventListener('input', (e) => {
        appState.radius = parseFloat(e.target.value);
        DOM.radiusInput.value = appState.radius.toFixed(1);
        triggerPropertyTone();
    });

    DOM.radiusInput.addEventListener('input', (e) => {
        let v = parseFloat(e.target.value) || 1.0;
        v = Math.max(0.1, Math.min(10.0, v));
        appState.radius = v;
        DOM.radiusSlider.value = v;
        triggerPropertyTone();
    });

    // Compare Controls
    DOM.btnCompare.addEventListener('click', () => {

```

```

    appState.compareMode = true;
    appState.compareTemp = appState.temp;
    appState.compareRadius = appState.radius;
    DOM.btnClear.style.display = 'block';
    DOM.compareActionsRow.style.display = 'flex';
    DOM.compareAdvancedGroup.style.display = 'flex';

    if (!appState.scaleLocked) {
        setLockScale(true);
    }
});

DOM.btnClear.addEventListener('click', () => {
    appState.compareMode = false;
    DOM.btnClear.style.display = 'none';
    DOM.compareActionsRow.style.display = 'none';
    DOM.compareAdvancedGroup.style.display = 'none';
    if (appState.scaleLocked) {
        setLockScale(false);
    }
});

// Advanced Toggles
DOM.toggleSound.addEventListener('change', (e) => {
    appState.isSoundEnabled = e.target.checked;
    DOM.playbackControls.style.display = appState.isSoundEnabled ? "flex" : "none";
    if (appState.isSoundEnabled) {
        initAudioEngine();
    } else if (appState.isPlaying) {
        togglePlayback();
    }
});

DOM.toggleByEye.addEventListener('change', (e) => { appState.showByEye = e.target.checked; });
DOM.logXCheck.addEventListener('change', (e) => { appState.isLogX = e.target.checked; });
DOM.logYCheck.addEventListener('change', (e) => { appState.isLogY = e.target.checked; });
DOM.relativeCheck.addEventListener('change', (e) => { appState.isRelative = e.target.checked; });
DOM.starLabelCheck.addEventListener('change', (e) => { appState.showStarLabel = e.target.checked; });
DOM.toggleCompareText.addEventListener('change', (e) => { appState.showCompareText = e.target.checked; });

DOM.btnLockScale.addEventListener('click', toggleLockScale);
DOM.btnLockScaleLeft.addEventListener('click', toggleLockScale);

DOM.xMinInput.addEventListener('change', (e) => {
    let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MIN;
    if(v >= appState.xMax) v = appState.xMax - 1;
    appState.xMin = Math.max(1, v);
    DOM.xMinInput.value = appState.xMin;
});

DOM.xMaxInput.addEventListener('change', (e) => {
    let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MAX;
    if(v <= appState.xMin) v = appState.xMin + 1;
    appState.xMax = v;
    DOM.xMaxInput.value = appState.xMax;
});

DOM.yMaxInput.addEventListener('change', (e) => {
    let v = parseFloat(e.target.value);
    if (isNaN(v) || v <= 0) {
        appState.customYMax = null;
    } else {
        appState.customYMax = Math.max(1e-10, v * 1e-11);
    }
});

DOM.btnResetLimits.addEventListener('click', () => {
    appState.xMin = DEFAULT_LAMBDA_MIN;
    appState.xMax = DEFAULT_LAMBDA_MAX;
    appState.customYMax = null;
    DOM.xMinInput.value = appState.xMin;
    DOM.xMaxInput.value = appState.xMax;
    DOM.yMaxInput.value = "";
    if (appState.scaleLocked) setLockScale(false);
});

```

```

});

// Playback Events
DOM.btnPlay.addEventListener('click', togglePlayback);
DOM.btnReset.addEventListener('click', () => {
  appState.scanWavelength = appState.xMin;
  appState.isPlaying = false;
  DOM.btnPlay.innerHTML = "Play Spectrum";
  DOM.btnPlay.style.backgroundColor = "var(--sound-accent)";
});
DOM.btnSlower.addEventListener('click', () => {
  appState.playbackSpeedMultiplier = Math.max(0.25, appState.playbackSpeedMultiplier - 0.25);
  DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
});
DOM.btnFaster.addEventListener('click', () => {
  appState.playbackSpeedMultiplier = Math.min(4.0, appState.playbackSpeedMultiplier + 0.25);
  DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
});

// Interactive Canvas Mapping
DOM.specCanvas.addEventListener('mousemove', (e) => {
  const rect = DOM.specCanvas.getBoundingClientRect();
  appState.mouseX = e.clientX - rect.left;
  appState.mouseY = e.clientY - rect.top;

  if (appState.isDraggingScanLine && appState.isSoundEnabled) {
    const dpr = window.devicePixelRatio || 1;
    const padLeft = 110 * dpr;
    const graphW = (rect.width * dpr) - padLeft - (30 * dpr);
    const currentMouseXScaled = appState.mouseX * dpr;

    let pct = (currentMouseXScaled - padLeft) / graphW;
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    let targetWavelength = appState.isLogX
      ? Math.pow(10, logMin + pct * (logMax - logMin))
      : appState.xMin + pct * (appState.xMax - appState.xMin);

    appState.scanWavelength = Math.max(appState.xMin, Math.min(appState.xMax, targetWavelength));
  }
});

DOM.specCanvas.addEventListener('mousedown', (e) => {
  if (!appState.isSoundEnabled) return;
  const rect = DOM.specCanvas.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  const clickXScaled = (e.clientX - rect.left) * dpr;
  const padLeft = 110 * dpr;
  const graphW = (rect.width * dpr) - padLeft - (30 * dpr);

  const getXCoord = (nm) => {
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);
    if (appState.isLogX) {
      return padLeft + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
    }
    return padLeft + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
  };

  if (Math.abs(clickXScaled - getXCoord(appState.scanWavelength)) <= 16 * dpr) {
    appState.isDraggingScanLine = true;
  }
});

window.addEventListener('mouseup', () => { appState.isDraggingScanLine = false; });
DOM.specCanvas.addEventListener('mouseleave', () => {
  appState.mouseX = null;
  appState.mouseY = null;
});

window.addEventListener('resize', triggerResize);
}

```

```

window.onload = () => {
  bindEvents();
  triggerResize();
  requestAnimationFrame(renderLoop);
};
</script>
</body>
</html>

```

User prompt: When in relative scale, always have the Y-axis extend up to 1.15 or so - high enough that the annotation will appear above the line in compare mode, and also high enough that the thick line is not getting clipped by the top of the graph display

Response: Here is the updated HTML file.

To accomplish this, I increased the top boundary of the graph to 1.15 when in relative mode so the peaks and annotations sit comfortably below the border. I also updated the Y-axis label logic so that it explicitly continues to draw the nice 0, 0.25, 0.5, 0.75, 1.0 markers rather than trying to automatically slice up the new 1.15 maximum into messy decimals.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Stellar Simulator (Embed)</title>
  <style>
    :root {
      --bg-color: #ffffff;
      --panel-bg: #f8f9fa;
      --text-color: #111827;
      --accent-color: #2563eb;
      --border-color: #cbd5e1;
      --sound-accent: #f43f5e;
      --compare-accent: #475569;
    }

    body {
      margin: 0;
      padding: 20px;
      font-family: system-ui, -apple-system, sans-serif;
      background-color: #e2e8f0;
      display: flex;
      justify-content: center;
    }

    .simulator-embed {
      width: 1000px;
      height: 375px;
      background-color: var(--bg-color);
      color: var(--text-color);
      border: 1px solid var(--border-color);
      box-shadow: 0 4px 6px rgba(0,0,0,0.1);
      box-sizing: border-box;
      display: flex;
      flex-direction: column;
      position: relative;
    }

    .top-section {
      display: flex;
      flex: 1;
      padding: 15px;
      gap: 15px;
      height: calc(100% - 45px);
      box-sizing: border-box;
    }

    .left-controls {
      width: 250px;
      background-color: var(--panel-bg);
      border: 1px solid var(--border-color);
      border-radius: 8px;
      padding: 15px;
      display: flex;

```

```
    flex-direction: column;
    gap: 15px;
    box-sizing: border-box;
    overflow-y: auto;
}

.control-group {
    display: flex;
    flex-direction: column;
    gap: 5px;
}

.control-group label {
    font-weight: 600;
    font-size: 0.90rem;
}

.control-group input[type="range"] {
    width: 100%;
    cursor: pointer;
}

.control-group input[type="number"] {
    width: 90px;
    padding: 4px;
    border: 1px solid var(--border-color);
    border-radius: 4px;
    text-align: center;
    font-size: 0.9rem;
    align-self: center;
}

.btn-row {
    display: flex;
    gap: 5px;
}

.btn-main {
    background-color: var(--sound-accent);
    color: white;
    border: none;
    padding: 6px;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
    width: 100%;
}

.btn-compare {
    background-color: var(--compare-accent);
    color: white;
    border: none;
    padding: 6px;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
    flex: 1;
}

.btn-sub {
    background-color: #e2e8f0;
    border: 1px solid var(--border-color);
    padding: 4px;
    border-radius: 4px;
    flex: 1;
    font-size: 0.8rem;
    cursor: pointer;
    font-weight: 600;
}

.btn-sub.active {
    background-color: var(--compare-accent);
    color: white;
    border-color: var(--compare-accent);
}
```

```
#playback-controls {
  display: none;
  flex-direction: column;
  gap: 8px;
  background: #fff;
  border: 1px solid var(--sound-accent);
  padding: 10px;
  border-radius: 6px;
  margin-top: auto;
}

.playback-label {
  font-weight: bold;
  color: var(--sound-accent);
  font-size: 0.85rem;
  text-align: center;
  margin-bottom: 4px;
}

.speed-text { font-size: 0.8rem; text-align: center; padding-top: 2px; }

.main-display {
  flex: 1;
  position: relative;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  background-color: #ffffff;
  box-sizing: border-box;
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
  border-radius: 8px;
  cursor: crosshair;
}

#star-overlay {
  position: absolute;
  background: rgba(255, 255, 255, 0.85);
  backdrop-filter: blur(4px);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 10px;
  display: flex;
  flex-direction: column;
  align-items: center;
  pointer-events: none;
  transition: left 0.3s, right 0.3s;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.overlay-right { right: 20px; top: 15px; left: auto; }
.overlay-left { left: 95px; top: 15px; right: auto; }

.star-sphere-wrapper {
  width: 80px;
  height: 80px;
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: #0b0b0c;
  border-radius: 8px;
  overflow: hidden;
  margin-bottom: 5px;
}

.star-sphere {
  border-radius: 50%;
  flex-shrink: 0;
}
```

```

.star-label { font-weight: bold; font-size: 0.85rem; }
.star-meta { font-size: 0.75rem; color: #475569; text-align: center;}

.bottom-section {
  padding: 0 15px 15px 15px;
  box-sizing: border-box;
}

#advanced-btn {
  background-color: var(--panel-bg);
  border: 1px solid var(--border-color);
  padding: 6px;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
  text-align: center;
  transition: background-color 0.2s;
  width: 100%;
  font-size: 0.9rem;
}

#advanced-btn:hover { background-color: #e2e8f0; }

.advanced-panel {
  display: none;
  position: absolute;
  bottom: 45px;
  left: 15px;
  width: calc(100% - 30px);
  background-color: #ffffff;
  border: 1px solid var(--border-color);
  border-radius: 6px;
  padding: 10px 15px;
  box-shadow: 0 -4px 15px rgba(0,0,0,0.15);
  box-sizing: border-box;
  z-index: 100;
  flex-wrap: wrap;
  gap: 15px;
  align-items: center;
  font-size: 0.85rem;
}

.toggle-group {
  display: flex;
  align-items: center;
  gap: 6px;
}

.range-input {
  width: 60px;
  padding: 4px;
  border: 1px solid var(--border-color);
  border-radius: 4px;
  text-align: center;
}

button.small-btn {
  background-color: #f1f5f9;
  border: 1px solid var(--border-color);
  padding: 4px 10px;
  border-radius: 4px;
  cursor: pointer;
  font-weight: 600;
}

button.small-btn.active {
  background-color: var(--compare-accent);
  color: white;
  border-color: var(--compare-accent);
}
</style>
</head>
<body>

<div class="simulator-embed">

```

```

<div class="top-section">
  <div class="left-controls">
    <h3 style="margin: 0; font-size: 1.1rem; border-bottom: 1px solid var(--border-color); padding-bo

    <div class="control-group">
      <label for="temp-slider">Temperature (K)</label>
      <input type="range" id="temp-slider" min="7.7406" max="10.9151" step="0.0001" value="8.6656">
      <input type="number" id="temp-input" min="2300" max="55000" step="100" value="5800">
    </div>

    <div class="control-group">
      <label for="radius-slider">Radius (R☉)</label>
      <input type="range" id="radius-slider" min="0.1" max="10.0" step="0.1" value="1.0">
      <input type="number" id="radius-input" min="0.1" max="10.0" step="0.1" value="1.0">
    </div>

    <div class="control-group btn-row" style="margin-top: 5px;">
      <button id="btn-compare" class="btn-compare">Compare</button>
      <button id="btn-clear" class="btn-sub" style="display: none;">Clear</button>
    </div>

    <div class="control-group btn-row" id="compare-actions-row" style="display: none; margin-top: 2px;">
      <button id="btn-lock-scale-left" class="btn-sub">Lock Scale</button>
    </div>

    <div id="playback-controls">
      <div class="playback-label">Spectrum Audio Scan</div>
      <button id="btn-play" class="btn-main">Play Spectrum</button>
      <div class="btn-row">
        <button id="btn-reset" class="btn-sub">Reset</button>
        <button id="btn-slower" class="btn-sub">Slower</button>
        <button id="btn-faster" class="btn-sub">Faster</button>
      </div>
      <div id="speed-label" class="speed-text">Speed: 1.00x</div>
    </div>
  </div>

  <div class="main-display">
    <canvas id="spectrumCanvas"></canvas>

    <div id="star-overlay" class="overlay-right">
      <div class="star-sphere-wrapper">
        <div id="star-sphere" class="star-sphere"></div>
      </div>
      <div class="star-label" id="star-label">Target Star</div>
      <div class="star-meta" id="star-meta-dynamic">5,800 K<br>1.0 R☉</div>
    </div>
  </div>
</div>

<div class="bottom-section">
  <button id="advanced-btn">Toggle Advanced Display Options</button>
  <div class="advanced-panel" id="advanced-panel">
    <div class="toggle-group" style="font-weight: bold; color: var(--sound-accent);">
      <input type="checkbox" id="toggle-sound">
      <label for="toggle-sound">Sonified</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
      <input type="checkbox" id="toggle-byeye" checked>
      <label for="toggle-byeye">"By-Eye" Strip</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
      <input type="checkbox" id="toggle-logX">
      <label for="toggle-logX">Log X Axis</label>
    </div>
    <div class="toggle-group">
      <input type="checkbox" id="toggle-logY">
      <label for="toggle-logY">Log Y Axis</label>
    </div>
    <div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px">
      <input type="checkbox" id="toggle-relative">
      <label for="toggle-relative">Relative Brightness</label>
      <button id="btn-lock-scale" class="small-btn" style="margin-left: 8px;">Lock Scale</button>
    </div>
  </div>

```

```

<div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px;
  <input type="checkbox" id="toggle-starlabel" checked>
  <label for="toggle-starlabel">Show Star Text</label>
</div>

<div class="toggle-group" id="compare-advanced-group" style="display: none; margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px;
  <input type="checkbox" id="toggle-comparetext" checked>
  <label for="toggle-comparetext">Compare Text</label>
</div>

<div class="toggle-group" style="margin-left: 10px; border-left: 1px solid #ccc; padding-left: 15px;
  <label>X (nm):</label>
  <input type="number" id="x-min-input" class="range-input" value="10" to
  <input type="number" id="x-max-input" class="range-input" value="2000">
</div>
<div class="toggle-group">
  <label>Y Max Limit:</label>
  <input type="number" id="y-max-input" class="range-input" placeholder="Auto">
</div>
<button id="btn-reset-limits" class="small-btn">Reset View</button>
</div>
</div>
</div>
<script>
  // App Constants
  const SUN_TEMP = 5800;
  const BASE_SUN_RADIUS_PX = 10;
  const DEFAULT_LAMBDA_MIN = 10;
  const DEFAULT_LAMBDA_MAX = 2000;
  const C1 = 3.741771e16;
  const C2 = 1.438776e7;

  // Dynamic State
  let appState = {
    temp: SUN_TEMP,
    radius: 1.0,
    isLogX: false,
    isLogY: false,
    showByEye: true,
    isRelative: false,
    showStarLabel: true,
    xMin: DEFAULT_LAMBDA_MIN,
    xMax: DEFAULT_LAMBDA_MAX,
    customYMax: null,
    mouseX: null,
    mouseY: null,
    barLogOrders: 2.0,

    // Compare Mode State
    compareMode: false,
    compareTemp: null,
    compareRadius: null,
    showCompareText: true,
    scaleLocked: false,
    lockedMaxGraphY: null,
    lockedReferenceIntensity: null,
    currentMaxGraphY: null,
    currentReferenceIntensity: null,

    // Sonification State
    isSoundEnabled: false,
    isPlaying: false,
    scanWavelength: DEFAULT_LAMBDA_MIN,
    playbackSpeedMultiplier: 1.0,
    audioOrders: 4.0,
    isDraggingScanLine: false
  };

  // DOM Links Cache
  const DOM = {
    tempSlider: document.getElementById('temp-slider'),
    tempInput: document.getElementById('temp-input'),
    radiusSlider: document.getElementById('radius-slider'),

```

```

radiusInput: document.getElementById('radius-input'),
btnCompare: document.getElementById('btn-compare'),
btnClear: document.getElementById('btn-clear'),
compareActionsRow: document.getElementById('compare-actions-row'),
btnLockScaleLeft: document.getElementById('btn-lock-scale-left'),

starSphere: document.getElementById('star-sphere'),
metaDynamic: document.getElementById('star-meta-dynamic'),
starLabel: document.getElementById('star-label'),
starOverlay: document.getElementById('star-overlay'),

specCanvas: document.getElementById('spectrumCanvas'),

advancedBtn: document.getElementById('advanced-btn'),
advancedPanel: document.getElementById('advanced-panel'),

toggleSound: document.getElementById('toggle-sound'),
toggleByEye: document.getElementById('toggle-byeye'),
logXCheck: document.getElementById('toggle-logX'),
logYCheck: document.getElementById('toggle-logY'),
relativeCheck: document.getElementById('toggle-relative'),
starLabelCheck: document.getElementById('toggle-starlabel'),

compareAdvancedGroup: document.getElementById('compare-advanced-group'),
toggleCompareText: document.getElementById('toggle-comparetext'),
btnLockScale: document.getElementById('btn-lock-scale'),

xMinInput: document.getElementById('x-min-input'),
xMaxInput: document.getElementById('x-max-input'),
yMaxInput: document.getElementById('y-max-input'),
btnResetLimits: document.getElementById('btn-reset-limits'),

// Playback DOM
playbackControls: document.getElementById('playback-controls'),
btnPlay: document.getElementById('btn-play'),
btnReset: document.getElementById('btn-reset'),
btnSlower: document.getElementById('btn-slower'),
btnFaster: document.getElementById('btn-faster'),
speedLabel: document.getElementById('speed-label')
};

const specCtx = DOM.specCanvas.getContext('2d');

// Web Audio Variables
let audioCtx = null;
let propOsc = null;
let propGain = null;
let scanOsc = null;
let scanGain = null;
let propFadeTimeout = null;

function initAudioEngine() {
  if (audioCtx) return;
  audioCtx = new (window.AudioContext || window.webkitAudioContext)();

  propOsc = audioCtx.createOscillator();
  propOsc.type = 'sine';
  propGain = audioCtx.createGain();
  propGain.gain.setValueAtTime(0, audioCtx.currentTime);
  propOsc.connect(propGain);
  propGain.connect(audioCtx.destination);
  propOsc.start();

  scanOsc = audioCtx.createOscillator();
  scanOsc.type = 'triangle';
  scanGain = audioCtx.createGain();
  scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
  scanOsc.connect(scanGain);
  scanGain.connect(audioCtx.destination);
  scanOsc.start();
}

function evaluatePlanck(nm, temperature) {
  const expTerm = Math.exp(C2 / (nm * temperature));

```

```

    if (!isFinite(expTerm) || expTerm <= 1) return 0;
    return C1 / (Math.pow(nm, 5) * (expTerm - 1));
}

function getLuminosity(t, r) {
    return (r * r) * Math.pow(t / SUN_TEMP, 4);
}

function kelvinToRGB(k) {
    let normalizedK = k / 100;
    let r, g, b;

    if (normalizedK <= 66) {
        r = 255;
        g = Math.max(0, Math.min(255, 99.4708 * Math.log(normalizedK) - 161.1196));
        b = normalizedK <= 19 ? 0 : Math.max(0, Math.min(255, 138.5177 * Math.log(normalizedK - 10) - 305.
    } else {
        r = Math.max(0, Math.min(255, 329.6987 * Math.pow(normalizedK - 60, -0.1332)));
        g = Math.max(0, Math.min(255, 288.1222 * Math.pow(normalizedK - 60, -0.0755)));
        b = 255;
    }
    return { r: Math.round(r), g: Math.round(g), b: Math.round(b) };
}

function wavelengthToRGB(nm) {
    let r = 0, g = 0, b = 0;
    if (nm >= 380 && nm < 440) { r = -(nm - 440) / (440 - 380); b = 1.0; }
    else if (nm >= 440 && nm < 490) { g = (nm - 440) / (490 - 440); b = 1.0; }
    else if (nm >= 490 && nm < 510) { g = 1.0; b = -(nm - 510) / (510 - 490); }
    else if (nm >= 510 && nm < 580) { r = (nm - 510) / (580 - 510); g = 1.0; }
    else if (nm >= 580 && nm < 645) { r = 1.0; g = -(nm - 645) / (645 - 580); }
    else if (nm >= 645 && nm <= 750) { r = 1.0; }
    return { r: Math.round(r * 255), g: Math.round(g * 255), b: Math.round(b * 255) };
}

function stringifyLargeValues(num) {
    if (num === 0) return "0";
    if (num >= 1e12) return (num / 1e12).toFixed(1).replace('.0', '') + " trillion";
    if (num >= 1e9) return (num / 1e9).toFixed(1).replace('.0', '') + " billion";
    if (num >= 1e6) return (num / 1e6).toFixed(1).replace('.0', '') + " million";
    if (num >= 1e3) return (num / 1e3).toFixed(1).replace('.0', '') + " thousand";

    if (num < 1) return parseFloat(num.toPrecision(2)).toString();
    if (num < 10) return num.toFixed(1);
    return num.toFixed(0);
}

function getNiceMax(val) {
    if (val <= 0) return 1.0;
    const exponent = Math.floor(Math.log10(val));
    const fraction = val / Math.pow(10, exponent);

    let niceFraction;
    if (fraction <= 1) niceFraction = 1;
    else if (fraction <= 2) niceFraction = 2;
    else if (fraction <= 2.5) niceFraction = 2.5;
    else if (fraction <= 5) niceFraction = 5;
    else niceFraction = 10;

    return niceFraction * Math.pow(10, exponent);
}

function triggerPropertyTone() {
    if (!appState.isSoundEnabled) return;
    initAudioEngine();
    if (audioCtx.state === 'suspended') audioCtx.resume();

    let peakWavelength = 2.8977719e6 / appState.temp;
    let clampedWavelength = Math.max(appState.xMin, Math.min(appState.xMax, peakWavelength));
    let pct = (clampedWavelength - appState.xMin) / (appState.xMax - appState.xMin);
    let frequency = 1300 * Math.pow((110 / 1300), pct);

    const targetLuminosity = getLuminosity(appState.temp, appState.radius);
    let logL = targetLuminosity > 0 ? Math.log10(targetLuminosity) : -4;

```

```

let scaleAlpha = 0.33 + (logL * 0.15);
let volumeFactor = Math.max(0.05, Math.min(1.0, scaleAlpha)) * 0.25;

propOsc.frequency.setValueAtTime(frequency, audioCtx.currentTime);

if (propFadeTimeout) clearTimeout(propFadeTimeout);
propGain.gain.setValueAtTime(volumeFactor, audioCtx.currentTime);

propFadeTimeout = setTimeout(() => {
  propGain.gain.setValueAtTime(propGain.gain.value, audioCtx.currentTime);
  propGain.gain.linearRampToValueAtTime(0, audioCtx.currentTime + 0.5);
}, 40);
}

function updatePlaybackScanner(dt) {
  if (appState.isPlaying && !appState.isDraggingScanLine) {
    const baseWavelengthStepPerMs = (appState.xMax - appState.xMin) / 7000;
    appState.scanWavelength += baseWavelengthStepPerMs * dt * appState.playbackSpeedMultiplier;

    if (appState.scanWavelength >= appState.xMax) {
      appState.scanWavelength = appState.xMin;
    }
  }

  if (appState.isSoundEnabled && (appState.isPlaying || appState.isDraggingScanLine)) {
    initAudioEngine();
    if (audioCtx.state === 'suspended') audioCtx.resume();

    let pct = (appState.scanWavelength - appState.xMin) / (appState.xMax - appState.xMin);
    let frequency = 1300 * Math.pow((110 / 1300), pct);

    const areaModifier = appState.radius * appState.radius;
    let currentIntensity = evaluatePlanck(appState.scanWavelength, appState.temp) * areaModifier;
    let peakWavelength = 2.8977719e6 / appState.temp;
    let peakCurveIntensity = evaluatePlanck(peakWavelength, appState.temp) * areaModifier;
    if (peakCurveIntensity <= 0) peakCurveIntensity = 1.0;

    let ratio = currentIntensity / peakCurveIntensity;
    let scanVolume = 0;
    if (ratio > 0) {
      let logRatio = Math.log10(ratio);
      scanVolume = 1.0 + (logRatio / appState.audioOrders);
      if (scanVolume < 0) scanVolume = 0;
    }

    scanOsc.frequency.setValueAtTime(frequency, audioCtx.currentTime);
    scanGain.gain.setValueAtTime(scanVolume * 0.25, audioCtx.currentTime);
  } else {
    if (scanGain) {
      scanGain.gain.setValueAtTime(0, audioCtx.currentTime);
    }
  }
}

function togglePlayback() {
  appState.isPlaying = !appState.isPlaying;
  DOM.btnPlay.innerHTML = appState.isPlaying ? "Pause Spectrum" : "Play Spectrum";
  DOM.btnPlay.style.backgroundColor = appState.isPlaying ? "#94a3b8" : "var(--sound-accent)";
  if (appState.isSoundEnabled) initAudioEngine();
}

function triggerResize() {
  const dpr = window.devicePixelRatio || 1;
  const specRect = DOM.specCanvas.parentElement.getBoundingClientRect();
  DOM.specCanvas.width = (specRect.width) * dpr;
  DOM.specCanvas.height = (specRect.height) * dpr;
}

function drawAppGraphics() {
  renderPhysicalSpheres();
  renderSpectrumChart();
}

function renderPhysicalSpheres() {

```

```

const targetRGB = kelvinToRGB(appState.temp);
const targetCSSColor = `rgb(${targetRGB.r}, ${targetRGB.g}, ${targetRGB.b})`;
const computedRadius = Math.max(4, BASE_SUN_RADIUS_PX * appState.radius);

DOM.starSphere.style.backgroundColor = targetCSSColor;
DOM.starSphere.style.width = `${computedRadius * 2}px`;
DOM.starSphere.style.height = `${computedRadius * 2}px`;
DOM.starSphere.style.boxShadow = `0 0 ${10 + computedRadius * 0.2}px ${targetCSSColor}`;
DOM.metaDynamic.innerHTML = `${appState.temp.toLocaleString()} K<br>${appState.radius.toFixed(1)} R☉`;

DOM.starLabel.style.display = appState.showStarLabel ? "block" : "none";
DOM.metaDynamic.style.display = appState.showStarLabel ? "block" : "none";
}

function renderSpectrumChart() {
  const canvas = DOM.specCanvas;
  const ctx = specCtx;

  const w = canvas.width;
  const h = canvas.height;
  if (w <= 0 || h <= 0) return;

  ctx.clearRect(0, 0, w, h);

  const dpr = window.devicePixelRatio || 1;
  const pad = { top: 30 * dpr, right: 30 * dpr, bottom: 45 * dpr, left: 110 * dpr };
  const graphW = w - pad.left - pad.right;
  const graphH = h - pad.top - pad.bottom;

  if (graphW <= 0 || graphH <= 0) return;

  const getXCoord = (nm) => {
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
      return pad.left + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
    }
    return pad.left + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
  };

  const getWavelengthFromX = (x) => {
    const pct = (x - pad.left) / graphW;
    let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
    let logMax = Math.log10(appState.xMax);

    if (appState.isLogX) {
      return Math.pow(10, logMin + pct * (logMax - logMin));
    }
    return appState.xMin + pct * (appState.xMax - appState.xMin);
  };

  // Render By-Eye Strip
  if (appState.showByEye) {
    const stripHeight = 16 * dpr;
    const stripY = pad.top / 2 - stripHeight / 2 + 5 * dpr;

    let systemPeakRadiance = 0;
    const step = Math.max(1, (appState.xMax - appState.xMin) / 1000);
    for (let l = appState.xMin; l <= appState.xMax; l += step) {
      let r = evaluatePlanck(l, appState.temp);
      if (r > systemPeakRadiance) systemPeakRadiance = r;
    }
    if (systemPeakRadiance === 0) systemPeakRadiance = 1.0;

    ctx.save();
    ctx.beginPath();
    ctx.rect(pad.left, stripY, graphW, stripHeight);
    ctx.clip();

    for (let x = pad.left; x < pad.left + graphW; x += Math.max(1, dpr)) {
      const currentNM = getWavelengthFromX(x);
      const intensity = evaluatePlanck(currentNM, appState.temp);

```

```

    let brightnessFactor = 0;
    if (intensity > 0) {
        let ratio = intensity / systemPeakRadiance;
        let logRatio = Math.log10(ratio);
        brightnessFactor = 1.0 + (logRatio / appState.barLogOrders);
        if (brightnessFactor < 0) brightnessFactor = 0;
    }

    if (currentNM < 380 || currentNM > 750) {
        let grayscaleIntensity = Math.round(230 * brightnessFactor);
        ctx.fillStyle = `rgb(${grayscaleIntensity}, ${grayscaleIntensity}, ${grayscaleIntensity})`;
    } else {
        let hue = wavelengthToRGB(currentNM);
        ctx.fillStyle = `rgb(${Math.round(hue.r * brightnessFactor)}, ${Math.round(hue.g * brightn
    }
    ctx.fillRect(x, stripY, Math.max(1, dpr) + 0.5, stripHeight);
}
ctx.restore();

ctx.strokeStyle = "#cbd5e1";
ctx.lineWidth = 1 * dpr;
ctx.strokeRect(pad.left, stripY, graphW, stripHeight);
}

// Active Star Data
const pointsArray = [];
let maxObservedCurveIntensity = 0;
let peakX = pad.left;
const areaModifier = appState.radius * appState.radius;

for (let x = pad.left; x <= pad.left + graphW; x += Math.max(1, dpr * 2)) {
    const nm = getWavelengthFromX(x);
    const rawIntensity = evaluatePlanck(nm, appState.temp) * areaModifier;
    pointsArray.push({ x: x, val: rawIntensity });
    if (rawIntensity > maxObservedCurveIntensity) {
        maxObservedCurveIntensity = rawIntensity;
        peakX = x;
    }
}

// Compare Star Data
const comparePointsArray = [];
let compareMaxIntensity = 0;
let comparePeakX = pad.left;

if (appState.compareMode) {
    const compareAreaModifier = appState.compareRadius * appState.compareRadius;
    for (let x = pad.left; x <= pad.left + graphW; x += Math.max(1, dpr * 2)) {
        const nm = getWavelengthFromX(x);
        const rawIntensity = evaluatePlanck(nm, appState.compareTemp) * compareAreaModifier;
        comparePointsArray.push({ x: x, val: rawIntensity });
        if (rawIntensity > compareMaxIntensity) {
            compareMaxIntensity = rawIntensity;
            comparePeakX = x;
        }
    }
}

// Setup Max Values & Scaling
let absoluteMaxIntensity = appState.compareMode ? Math.max(maxObservedCurveIntensity, compareMaxIntens
if (absoluteMaxIntensity === 0) absoluteMaxIntensity = 1.0;

let referenceIntensity = absoluteMaxIntensity;
if (appState.scaleLocked && appState.lockedReferenceIntensity) {
    referenceIntensity = appState.lockedReferenceIntensity;
} else {
    appState.currentReferenceIntensity = absoluteMaxIntensity;
}

// Apply Relative Scaling if requested
if (appState.isRelative) {
    pointsArray.forEach(pt => pt.val = pt.val / referenceIntensity);
    if (appState.compareMode) {
        comparePointsArray.forEach(pt => pt.val = pt.val / referenceIntensity);
    }
}

```

```

    }
  }

  // Define Y Axis Top Boundary
  let maxGraphY = appState.customYMax;
  if (!maxGraphY) {
    if (appState.scaleLocked && appState.lockedMaxGraphY) {
      maxGraphY = appState.lockedMaxGraphY;
    } else {
      if (appState.isRelative) {
        maxGraphY = 1.15; // Raised slightly so 1.0 values don't clip the graph top edge
      } else if (!appState.isLogY) {
        maxGraphY = getNiceMax(absoluteMaxIntensity);
      } else {
        maxGraphY = absoluteMaxIntensity;
      }
    }
  }
  appState.currentMaxGraphY = maxGraphY;
  const minGraphY = appState.isLogY ? maxGraphY * 1e-4 : 0;

  if (peakX > pad.left + graphW / 2) {
    DOM.starOverlay.className = 'overlay-left';
  } else {
    DOM.starOverlay.className = 'overlay-right';
  }

  const getYCoord = (intensity) => {
    let pct;
    if (appState.isLogY) {
      let safeVal = Math.max(minGraphY, intensity);
      pct = (Math.log10(safeVal) - Math.log10(minGraphY)) / (Math.log10(maxGraphY) - Math.log10(minGraphY));
    } else {
      pct = intensity / maxGraphY;
    }
    return pad.top + graphH - (Math.max(0, pct) * graphH);
    // We allow pct > 1 so points extend above chart visually prior to clipping
  };

  // Draw Axes & Grids
  ctx.strokeStyle = '#e2e8f0';
  ctx.lineWidth = 1 * dpr;
  ctx.fillStyle = '#475569';
  ctx.font = `${11 * dpr}px sans-serif`;

  ctx.textAlign = 'center';
  ctx.textBaseline = 'top';
  let xTicks = [];
  if (appState.isLogX) {
    xTicks = [appState.xMin];
    let current = Math.pow(10, Math.ceil(Math.log10(appState.xMin)));
    while (current < appState.xMax) {
      if (current > appState.xMin) xTicks.push(current);
      current *= 10;
    }
    xTicks.push(appState.xMax);
  } else {
    const range = appState.xMax - appState.xMin;
    const step = Math.pow(10, Math.floor(Math.log10(range))) / 2;
    for (let t = appState.xMin; t <= appState.xMax; t += step) {
      xTicks.push(t);
    }
  }

  xTicks.forEach(tick => {
    const x = getXCoord(tick);
    if (x < pad.left || x > pad.left + graphW) return;
    ctx.beginPath(); ctx.moveTo(x, pad.top); ctx.lineTo(x, pad.top + graphH); ctx.stroke();
    ctx.fillText(`${Math.round(tick)} nm`, x, pad.top + graphH + 8 * dpr);
  });

  ctx.textAlign = 'right';
  ctx.textBaseline = 'middle';
  const drawYLabel = (val, yPos) => {

```

```

ctx.beginPath(); ctx.moveTo(pad.left, yPos); ctx.lineTo(pad.left + graphW, yPos); ctx.stroke();

let labelStr;
if (appState.isRelative) {
  labelStr = val.toFixed(2);
} else {
  labelStr = stringifyLargeValues(val * 1e11);
}

let parts = labelStr.split(' ');
if (parts.length > 1) {
  ctx.fillText(parts[0], pad.left - 10 * dpr, yPos - 6 * dpr);
  ctx.fillText(parts[1], pad.left - 10 * dpr, yPos + 6 * dpr);
} else {
  ctx.fillText(labelStr, pad.left - 10 * dpr, yPos);
}
};

if (appState.isLogY) {
  let logStart = Math.ceil(Math.log10(minGraphY));
  let logEnd = Math.floor(Math.log10(maxGraphY));
  for (let o = logStart; o <= logEnd; o++) {
    let v = Math.pow(10, o);
    let y = getYCoord(v);
    drawYLabel(v, y);
  }
} else {
  if (appState.isRelative) {
    // Ensure labels still mark nice standardized fractional intervals (0 to 1.0) instead of mess
    [0, 0.25, 0.5, 0.75, 1.0].forEach(v => {
      let y = getYCoord(v);
      drawYLabel(v, y);
    });
  } else {
    [0, 0.25, 0.5, 0.75, 1.0].forEach(p => {
      let v = p * maxGraphY;
      let y = getYCoord(v);
      drawYLabel(v, y);
    });
  }
}

ctx.strokeStyle = '#94a3b8';
ctx.lineWidth = 1.5 * dpr;
ctx.strokeRect(pad.left, pad.top, graphW, graphH);

// Clip graph area for data lines
ctx.save();
ctx.beginPath();
ctx.rect(pad.left, pad.top, graphW, graphH);
ctx.clip();

// Draw Compare Graph (Gray Dashed)
if (appState.compareMode) {
  ctx.beginPath();
  let breakCompare = true;
  comparePointsArray.forEach(pt => {
    if (appState.isLogY && pt.val < minGraphY) {
      breakCompare = true;
    } else {
      let y = getYCoord(pt.val);
      if (breakCompare) {
        ctx.moveTo(pt.x, y);
        breakCompare = false;
      } else {
        ctx.lineTo(pt.x, y);
      }
    }
  });
  ctx.strokeStyle = '#94a3b8';
  ctx.lineWidth = 2 * dpr;
  ctx.setLineDash([6 * dpr, 6 * dpr]);
  ctx.stroke();
  ctx.setLineDash([]); // reset

```

```

// Draw Compare Annotation
if (appState.showCompareText) {
  let peakVal = appState.isRelative ? (compareMaxIntensity / referenceIntensity) : compareMaxInt
  let peakY = getYCoord(peakVal);

  ctx.fillStyle = '#64748b';
  ctx.font = `bold ${11 * dpr}px sans-serif`;
  ctx.textAlign = 'center';
  ctx.textBaseline = 'bottom';

  let annX = comparePeakX;
  if (annX < pad.left + 50 * dpr) annX = pad.left + 50 * dpr;
  if (annX > pad.left + graphW - 50 * dpr) annX = pad.left + graphW - 50 * dpr;
  let annY = peakY - 10 * dpr;

  // If clipping off top, invert text position so it hangs safely inside the box
  if (annY < pad.top + 5 * dpr) {
    annY = pad.top + 5 * dpr;
    ctx.textBaseline = 'top';
  }

  ctx.fillText(`${appState.compareTemp} K, ${appState.compareRadius.toFixed(1)} R☉`, annX, annY)
}

// Draw Main Data Curve (Black)
ctx.beginPath();
let breakPath = true;
pointsArray.forEach(pt => {
  if (appState.isLogY && pt.val < minGraphY) {
    breakPath = true;
  } else {
    let y = getYCoord(pt.val);
    if (breakPath) {
      ctx.moveTo(pt.x, y);
      breakPath = false;
    } else {
      ctx.lineTo(pt.x, y);
    }
  }
});

ctx.strokeStyle = '#000000';
ctx.lineWidth = 3 * dpr;
ctx.stroke();

ctx.restore(); // remove clipping

// Audio Scan Indicator Line
if (appState.isSoundEnabled) {
  const scanLineX = getXCoord(appState.scanWavelength);
  if (scanLineX >= pad.left && scanLineX <= pad.left + graphW) {
    ctx.strokeStyle = '#f43f5e';
    ctx.lineWidth = 2 * dpr;
    ctx.beginPath();
    ctx.moveTo(scanLineX, pad.top);
    ctx.lineTo(scanLineX, pad.top + graphH);
    ctx.stroke();

    ctx.beginPath();
    ctx.arc(scanLineX, pad.top, 5 * dpr, 0, Math.PI * 2);
    ctx.fillStyle = '#f43f5e';
    ctx.fill();
  }
}

// Mouse Hover Box (only reads active graph data for simplicity)
if (appState.mouseX !== null && appState.mouseY !== null && !appState.isDraggingScanLine) {
  const mx = appState.mouseX * dpr;
  const my = appState.mouseY * dpr;

  let closestPt = null;
  let minDist = Infinity;

```

```

pointsArray.forEach(pt => {
  let visible = true;
  if (appState.isLogY && pt.val < minGraphY) visible = false;
  if (visible) {
    let py = getYCoord(pt.val);
    if (py >= pad.top && py <= pad.top + graphH) {
      let dist = Math.hypot(pt.x - mx, py - my);
      if (dist < minDist) {
        minDist = dist;
        closestPt = pt;
      }
    }
  }
});

if (closestPt && minDist <= 15 * dpr) {
  let py = getYCoord(closestPt.val);

  ctx.beginPath();
  ctx.arc(closestPt.x, py, 5 * dpr, 0, Math.PI * 2);
  ctx.fillStyle = '#000000';
  ctx.fill();
  ctx.strokeStyle = '#ffffff';
  ctx.lineWidth = 1.5 * dpr;
  ctx.stroke();

  const hoverNM = getWavelengthFromX(closestPt.x);
  const stringWavelength = `Wavelength: ${Math.round(hoverNM)} nm`;
  let origVal = appState.isRelative ? closestPt.val * referenceIntensity : closestPt.val;

  const stringIntensity = appState.isRelative
    ? `Rel. Brightness: ${closestPt.val.toFixed(3)}`
    : `Intensity: ${stringifyLargeValues(origVal * 1e11)}`;

  ctx.font = `${11 * dpr}px sans-serif`;
  const textWidth = Math.max(ctx.measureText(stringWavelength).width, ctx.measureText(stringInte
  const boxW = textWidth + 16 * dpr;
  const boxH = 40 * dpr;

  let boxX = closestPt.x + 12 * dpr;
  let boxY = py - 20 * dpr;

  if (boxX + boxW > w) boxX = closestPt.x - boxW - 12 * dpr;
  if (boxY < 0) boxY = 10 * dpr;
  if (boxY + boxH > h) boxY = h - boxH - 10 * dpr;

  ctx.fillStyle = 'rgba(15, 23, 42, 0.92)';
  ctx.strokeStyle = '#475569';
  ctx.lineWidth = 1 * dpr;
  ctx.beginPath(); ctx.rect(boxX, boxY, boxW, boxH); ctx.fill(); ctx.stroke();
  ctx.fillStyle = '#f8fafc';
  ctx.textAlign = 'left';
  ctx.textBaseline = 'top';
  ctx.fillText(stringWavelength, boxX + 8 * dpr, boxY + 6 * dpr);
  ctx.fillText(stringIntensity, boxX + 8 * dpr, boxY + 22 * dpr);
}

}

ctx.fillStyle = '#111827';
ctx.font = `bold ${12 * dpr}px sans-serif`;
ctx.textAlign = 'center';
ctx.textBaseline = 'top';
ctx.fillText('Wavelength (nm)', pad.left + graphW / 2, pad.top + graphH + 25 * dpr);

ctx.save();
ctx.translate(pad.left - 65 * dpr, pad.top + graphH / 2);
ctx.rotate(-Math.PI / 2);
ctx.fillText(appState.isRelative ? 'Relative Brightness' : 'Spectral Radiance', 0, 0);
ctx.restore();
}

let lastTime = 0;
function renderLoop(timestamp) {

```

```

    if (!lastTime) lastTime = timestamp;
    let dt = timestamp - lastTime;
    lastTime = timestamp;

    updatePlaybackScanner(dt);
    drawAppGraphics();
    requestAnimationFrame(renderLoop);
}

function setLockScale(locked) {
    appState.scaleLocked = locked;
    if (locked) {
        appState.lockedMaxGraphY = appState.currentMaxGraphY;
        appState.lockedReferenceIntensity = appState.currentReferenceIntensity;
        DOM.btnLockScale.innerHTML = "Unlock Scale";
        DOM.btnLockScale.classList.add('active');
        DOM.btnLockScaleLeft.innerHTML = "Unlock Scale";
        DOM.btnLockScaleLeft.classList.add('active');
    } else {
        appState.lockedMaxGraphY = null;
        appState.lockedReferenceIntensity = null;
        DOM.btnLockScale.innerHTML = "Lock Scale";
        DOM.btnLockScale.classList.remove('active');
        DOM.btnLockScaleLeft.innerHTML = "Lock Scale";
        DOM.btnLockScaleLeft.classList.remove('active');
    }
}

function toggleLockScale() {
    setLockScale(!appState.scaleLocked);
}

function bindEvents() {
    DOM.advancedBtn.addEventListener('click', () => {
        const isHidden = DOM.advancedPanel.style.display === 'none' || DOM.advancedPanel.style.display ===
        DOM.advancedPanel.style.display = isHidden ? 'flex' : 'none';
    });

    DOM.tempSlider.addEventListener('input', (e) => {
        appState.temp = Math.round(Math.exp(parseFloat(e.target.value)));
        DOM.tempInput.value = appState.temp;
        triggerPropertyTone();
    });

    DOM.tempInput.addEventListener('input', (e) => {
        let v = parseInt(e.target.value) || SUN_TEMP;
        v = Math.max(2300, Math.min(55000, v));
        appState.temp = v;
        DOM.tempSlider.value = Math.log(v);
        triggerPropertyTone();
    });

    DOM.radiusSlider.addEventListener('input', (e) => {
        appState.radius = parseFloat(e.target.value);
        DOM.radiusInput.value = appState.radius.toFixed(1);
        triggerPropertyTone();
    });

    DOM.radiusInput.addEventListener('input', (e) => {
        let v = parseFloat(e.target.value) || 1.0;
        v = Math.max(0.1, Math.min(10.0, v));
        appState.radius = v;
        DOM.radiusSlider.value = v;
        triggerPropertyTone();
    });

    // Compare Controls
    DOM.btnCompare.addEventListener('click', () => {
        appState.compareMode = true;
        appState.compareTemp = appState.temp;
        appState.compareRadius = appState.radius;
        DOM.btnClear.style.display = 'block';
        DOM.compareActionsRow.style.display = 'flex';
        DOM.compareAdvancedGroup.style.display = 'flex';
    });
}

```

```

        if (!appState.scaleLocked) {
            setLockScale(true);
        }
    });

    DOM.btnClear.addEventListener('click', () => {
        appState.compareMode = false;
        DOM.btnClear.style.display = 'none';
        DOM.compareActionsRow.style.display = 'none';
        DOM.compareAdvancedGroup.style.display = 'none';
        if (appState.scaleLocked) {
            setLockScale(false);
        }
    });

    // Advanced Toggles
    DOM.toggleSound.addEventListener('change', (e) => {
        appState.isSoundEnabled = e.target.checked;
        DOM.playbackControls.style.display = appState.isSoundEnabled ? "flex" : "none";
        if (appState.isSoundEnabled) {
            initAudioEngine();
        } else if (appState.isPlaying) {
            togglePlayback();
        }
    });

    DOM.toggleByEye.addEventListener('change', (e) => { appState.showByEye = e.target.checked; });
    DOM.logXCheck.addEventListener('change', (e) => { appState.isLogX = e.target.checked; });
    DOM.logYCheck.addEventListener('change', (e) => { appState.isLogY = e.target.checked; });
    DOM.relativeCheck.addEventListener('change', (e) => { appState.isRelative = e.target.checked; });
    DOM.starLabelCheck.addEventListener('change', (e) => { appState.showStarLabel = e.target.checked; });
    DOM.toggleCompareText.addEventListener('change', (e) => { appState.showCompareText = e.target.checked; });

    DOM.btnLockScale.addEventListener('click', toggleLockScale);
    DOM.btnLockScaleLeft.addEventListener('click', toggleLockScale);

    DOM.xMinInput.addEventListener('change', (e) => {
        let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MIN;
        if (v >= appState.xMax) v = appState.xMax - 1;
        appState.xMin = Math.max(1, v);
        DOM.xMinInput.value = appState.xMin;
    });

    DOM.xMaxInput.addEventListener('change', (e) => {
        let v = parseFloat(e.target.value) || DEFAULT_LAMBDA_MAX;
        if (v <= appState.xMin) v = appState.xMin + 1;
        appState.xMax = v;
        DOM.xMaxInput.value = appState.xMax;
    });

    DOM.yMaxInput.addEventListener('change', (e) => {
        let v = parseFloat(e.target.value);
        if (isNaN(v) || v <= 0) {
            appState.customYMax = null;
        } else {
            appState.customYMax = Math.max(1e-10, v * 1e-11);
        }
    });

    DOM.btnResetLimits.addEventListener('click', () => {
        appState.xMin = DEFAULT_LAMBDA_MIN;
        appState.xMax = DEFAULT_LAMBDA_MAX;
        appState.customYMax = null;
        DOM.xMinInput.value = appState.xMin;
        DOM.xMaxInput.value = appState.xMax;
        DOM.yMaxInput.value = "";
        if (appState.scaleLocked) setLockScale(false);
    });

    // Playback Events
    DOM.btnPlay.addEventListener('click', togglePlayback);
    DOM.btnReset.addEventListener('click', () => {
        appState.scanWavelength = appState.xMin;

```

```

    appState.isPlaying = false;
    DOM.btnPlay.innerHTML = "Play Spectrum";
    DOM.btnPlay.style.backgroundColor = "var(--sound-accent)";
  });
  DOM.btnSlower.addEventListener('click', () => {
    appState.playbackSpeedMultiplier = Math.max(0.25, appState.playbackSpeedMultiplier - 0.25);
    DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
  });
  DOM.btnFaster.addEventListener('click', () => {
    appState.playbackSpeedMultiplier = Math.min(4.0, appState.playbackSpeedMultiplier + 0.25);
    DOM.speedLabel.innerHTML = `Speed: ${appState.playbackSpeedMultiplier.toFixed(2)}x`;
  });

  // Interactive Canvas Mapping
  DOM.specCanvas.addEventListener('mousemove', (e) => {
    const rect = DOM.specCanvas.getBoundingClientRect();
    appState.mouseX = e.clientX - rect.left;
    appState.mouseY = e.clientY - rect.top;

    if (appState.isDraggingScanLine && appState.isSoundEnabled) {
      const dpr = window.devicePixelRatio || 1;
      const padLeft = 110 * dpr;
      const graphW = (rect.width * dpr) - padLeft - (30 * dpr);
      const currentMouseXScaled = appState.mouseX * dpr;

      let pct = (currentMouseXScaled - padLeft) / graphW;
      let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
      let logMax = Math.log10(appState.xMax);

      let targetWavelength = appState.isLogX
        ? Math.pow(10, logMin + pct * (logMax - logMin))
        : appState.xMin + pct * (appState.xMax - appState.xMin);

      appState.scanWavelength = Math.max(appState.xMin, Math.min(appState.xMax, targetWavelength));
    }
  });

  DOM.specCanvas.addEventListener('mousedown', (e) => {
    if (!appState.isSoundEnabled) return;
    const rect = DOM.specCanvas.getBoundingClientRect();
    const dpr = window.devicePixelRatio || 1;
    const clickXScaled = (e.clientX - rect.left) * dpr;
    const padLeft = 110 * dpr;
    const graphW = (rect.width * dpr) - padLeft - (30 * dpr);

    const getXCoord = (nm) => {
      let logMin = Math.log10(appState.xMin > 0 ? appState.xMin : 1);
      let logMax = Math.log10(appState.xMax);
      if (appState.isLogX) {
        return padLeft + graphW * (Math.log10(Math.max(1, nm)) - logMin) / (logMax - logMin);
      }
      return padLeft + graphW * (nm - appState.xMin) / (appState.xMax - appState.xMin);
    };

    if (Math.abs(clickXScaled - getXCoord(appState.scanWavelength)) <= 16 * dpr) {
      appState.isDraggingScanLine = true;
    }
  });

  window.addEventListener('mouseup', () => { appState.isDraggingScanLine = false; });
  DOM.specCanvas.addEventListener('mouseleave', () => {
    appState.mouseX = null;
    appState.mouseY = null;
  });

  window.addEventListener('resize', triggerResize);
}

window.onload = () => {
  bindEvents();
  triggerResize();
  requestAnimationFrame(renderLoop);
};
</script>

```

</body>
</html>
